

Kirei

Full Fix Plan

Covers 241 verified findings — 157 scheduled, 84 deferred with reasons

Structure Phase 0 Safety → Phase 1 Sweeps → Phase 2 Features → Phase 3 Delete

Batches 4 safety · 6 sweeps · 25 feature · 4 deletion

Estimate ~4–5 focused weeks (2 weeks for the short version)

Rule One batch = one PR · verify each claim before fixing it

Generated 19 July 2026

Kirei — Full Fix Plan

Every one of the **241 verified findings** from the feature-by-feature analysis is accounted for below — either scheduled into a batch, or explicitly deferred with a reason. Nothing is silently dropped.

Sources: `Kirei-Feature-Analysis.pdf` (241 findings, 25 features) and `Kirei-Audit.pdf` (10 ship blockers).

How to use this plan

One batch = one PR. Each batch is sized to be reviewable in a sitting.

Verify before you fix. The audit has a known error rate: 5 findings were refuted in verification, and an earlier audit recommended deleting a route that was still serving production traffic. Open the cited file and confirm the claim holds before changing anything. If it does not hold, tick it off as *refuted* and move on.

Every PR: `npx tsc --noEmit` · `npm test` · `npm run build` · bundle budget. Migrations apply to the LIVE database — review them as production changes.

Order matters. Phase 1 sweeps remove findings that would otherwise be fixed repeatedly in Phase 2. Phase 3 deletions come before Tier D so you do not polish code you are about to remove.

Phase 0 — Safety

~1 day. Carried over from the first audit and still undone. Every item here was verified directly against the production database or code, not merely reported by an agent.

Batch 0.1 — Close the open tables (2h)

`sms_conversations`, `sms_messages` and `report_sessions` have **RLS disabled and zero policies**, and the two SMS tables are published to Supabase Realtime. Verified by querying `pg_class.relrowsecurity` on production.

- ☐ Migration: `ENABLE ROW LEVEL SECURITY` on all three
- ☐ Owner-UNION-active-member policy on each, mirroring the existing pattern
- ☐ `AND NOT public.is_business_worker(business_id)` on the SMS pair
- ☐ Deny-all on `report_sessions` (only the admin client writes it)
- ☐ Add `filter: business_id=eq.<id>` to the Realtime channel in `messages-client.tsx`
- ☐ Verify by re-querying `pg_policies` after applying

Current real exposure is small — 4 conversations, 5 messages, one business, and `report_sessions` is empty. This is a hole to close before customer #2, not a breach that has already happened.

Batch 0.2 — Crons fail closed (1h)

12 of 14 cron routes guard with `if (process.env.CRON_SECRET && auth !== ...)`, so a missing variable removes the check entirely. One of them charges saved cards.

- ☐ Add `requireCron(req)` — 500 when the secret is unset, 401 on mismatch
- ☐ Apply to all 14 routes under `src/app/api/cron/`
- ☐ Confirm `CRON_SECRET` exists in **every** Vercel environment, then redeploy

Batch 0.3 — Worker isolation (3h)

CLAUDE.md documents workers as hard-isolated. `is_business_worker()` is absent from five tables. The web app masks these by path, but mobile queries PostgREST directly with the worker's own JWT.

- ☐ `leads` · `contacts` · `sites` · `recurring_jobs` · `tasks`
- ☐ Fix `tasks` write policies naming roles that do not exist (`admin/manager/staff`) — `editor` currently fails silently
- ☐ Add `recurring_jobs` role gate (viewers can currently delete schedules)
- ☐ CI test enumerating every `business_id` table and asserting RLS + policy + worker clause

Batch 0.4 — Yours, not mine (minutes)

- ☐ Replace `ANTHROPIC_API_KEY` — returns 401, so every AI feature is dead
 - ☐ Set `SENTRY_DSN` — no error monitoring at all right now
 - ☐ Redeploy (env changes do not affect the running deployment)
-

Phase 1 — Cross-cutting sweeps

~4–5 days. The highest-leverage work in the plan. The 241 findings cluster into repeating patterns; fixing each as a sweep repairs many features at once and stops Phase 2 from fixing the same bug a dozen times.

Batch 1.1 — Money totals render \$NaN (1h)

PostgreSQL returns `numeric` as a **string**, so `s + row.total` concatenates: `"0" + "500.00" + "250.00"` → `formatCurrency` → `$NaN`. Correct with 0–1 rows, broken at 2+ — which is exactly why it shipped.

- ☐ `src/components/invoices/invoices-client.tsx:73` — "Paid" KPI on the main invoices page
- ☐ `src/components/customers/customer-detail-client.tsx:494` — "Total spent"
- ☐ `src/components/products/products-client.tsx:80` — "Catalog value" (default-on page)
- ☐ `src/components/invoices/invoice-detail-client.tsx:147` — breaks the paid-confetti trigger
- ☐ Grep for other reducers over money columns before closing this out

Batch 1.2 — Swallowed `error` renders failure as "no data" (1d)

The documented house trap. Callers destructure `{ data }` and ignore `{ error }`, so a failed query renders an empty state. The rational user response is to re-enter the data — which is precisely how the duplicate customer in PR #398 happened.

- ☐ `src/lib/actions/analytics.ts:61–68` — all six queries
- ☐ `src/lib/layout-data.ts:76–81` — plugin flags, then caches the wrong answer for 5 minutes
- ☐ `src/lib/actions/sites.ts:106`, `account-contacts.ts:73`, `billing-profiles.ts:98` — archive actions report success on failure
- ☐ `src/lib/actions/invoices.ts:351` — **the payment insert**
- ☐ `src/app/api/cron/recurring-jobs/route.ts:73`
- ☐ `src/app/(dashboard)/team/page.tsx:30–33`, `src/app/admin/tenants/page.tsx:28`
- ☐ Three dashboard widgets doing `catch { setX([]) }`
- ☐ 19 mobile fetch sites (can be a follow-up PR)
- ☐ Reserve empty states for genuinely empty; show an error state otherwise

Batch 1.3 — Destructive actions with no confirmation (4h)

`AlertDialog` already exists and is used correctly elsewhere **in the same files**. Two of these are severe: one charges a real card, one cascade-deletes every appointment a worker has ever had.

- ☐ `src/components/invoices/invoice-detail-client.tsx:507` — **charges a real card**
- ☐ `src/components/recurring/recurring-invoices-client.tsx:113` — **bills a customer**
- ☐ `src/components/bookings/booking-admin-client.tsx:392` — **cascade-deletes a worker's appointments**
- ☐ `src/components/work-orders/job-portfolio-client.tsx` — 7 sites incl. delete-whole-job at :217
- ☐ `src/components/settings/team-settings.tsx:229` — remove member
- ☐ `expenses-view.tsx:136`, `assets-view.tsx:107`, `kanban-board.tsx:782`, `stripe-settings.tsx:64`
- ☐ Assistant conversation delete

Batch 1.4 — Timezone — dates built with `toISOString()` (1d)

Local dates constructed via UTC. Invisible in the Americas; wrong in Sydney, where your only real customer is.

- ☐ `src/components/schedule/schedule-client.tsx:24-32` + `page.tsx:15` — double shift: the week is not Mon–Sun and **each next/prev click advances 6 days**, so the board walks off the calendar
- ☐ `src/lib/actions/timesheets.ts:41-70` — hours bucketed by UTC day, so 8am Monday Sydney files under the previous week — and that number is exported to payroll CSV
- ☐ `src/app/api/.../bookings/route.ts:165-167` — writes UTC time-of-day into columns the busy-intervals SQL reads as business-local, so the job lands on the wrong day
- ☐ Add unit tests pinned to `TZ=Australia/Sydney`

Batch 1.5 — Number minting races (4h)

`select next_number` → format → `update next+1`, non-atomic, and the counter is bumped **before** the insert that can fail. At 2 users the collision is theoretical; the burned number on a failed insert is certain.

- ☐ Race-safe RPCs `next_invoice_number` / `next_quote_number` already exist and are used in only 2 of 7 places
- ☐ Add a `next_work_order_number` RPC — it does not exist
- ☐ Add a service-role variant (the existing RPCs check `auth.uid()`, null for the admin client)
- ☐ Migrate: `quotes.ts:145`, `work-orders.ts:86`, `work-orders.ts:549`, `schedule.ts:57`, `recurring/generate-invoice.ts:42`, `cron/recurring-jobs:62`, `portal/.../accept-with-deposit:24`

Batch 1.6 — The schema-contract test (2h)

The single highest-value test available: assert that every column named in `.eq()` / `.order()` / `.select()` actually exists in the migrations. Catches a whole class of runtime-only bugs at build time.

- ☐ Parse migrations for table/column names
- ☐ Walk `src/lib/actions/**` for PostgreSQL column references
- ☐ Fail the test on any reference to a column that does not exist
- ☐ Wire into CI

Deliberately not swept: design-system unification (local status-pill copies, `truncate` vs `break-words`, off-palette colours — 12+ features). It is cosmetic, and as a standalone project it is a week that moves nothing. Fold it into each feature's PR in Phase 2 instead.

Phase 2 — Feature by feature

~2–3 weeks. Ordered by commercial importance. Some findings below are already resolved by Phase 1 sweeps — check before starting each batch.

Tier A — the money path

Batch 2.1 — Recurring invoices & recurring jobs

Health: rough · **Verdict:** simplify · **Findings:** 1 blocker · 3 high · 5 medium · 0 low **Estimated:** ~1.2 days (4 scheduled, 5 deferred)

What it does. Two separate, near-identical subsystems sharing one cadence helper. **Recurring jobs** (`/recurring`) are schedules that a nightly cron (19:30 UTC) materialises into work orders: for each active schedule whose `next_occurrence_at` falls inside its `generate_days_ahead` horizon, it mints a WO number, inserts a work order, assigns the default workers, optionally raises an invoice from saved line items and auto-charges the customer's card, then advances the date by cadence. **Recurring invoices** (`/recurring-invoices` , sidebar "Recurring billing") are subscription-style schedules: a 06:00 UTC cron generates an invoice from stored line items, either off-session-charges the saved card via Stripe or emails it with a pay link, advances `next_run_on` , and deactivates past `ends_on` . Both crons catch up missed cycles (capped at 6 and 8 iterations). Cadence is weekly / fortnightly / monthly / quarterly, computed by naive date arithmetic in `src/lib/recurring/cadence.ts` . There is also a manual "Run now" button on recurring invoices. Full MCP tool coverage exists for both.

Why this rating. The happy path works and the code is readable, but this is the one feature that moves money without a human in the loop, and three of its guards are missing: the customer's autopay opt-out is read from the DB and never checked, "Run now" doesn't advance the schedule so it double-bills, and a backdated start date fires six real card charges on day one. On top of that the `recurring_jobs` RLS policy has neither a worker exclusion nor a role check — the only business-data table in the feature that doesn't — so workers and viewers can read and write job schedules including their invoice pricing. Zero tests cover any of it, and the cadence maths silently drifts off month-end.

If you only do three things:

1. Close the two money-and-consent holes before anything else, both small: check `customer.autopay_enabled` in `src/lib/stripe-charge.ts` (one line, after `:51`) so the customer's off switch actually works, and stop 'Run now' double-billing by having either the action advance `next_run_on` or the cron skip cycles already covered by `last_run_at` . These are the only defects here that take money from a real customer wrongly, and the feature is one working Stripe key away from doing it.
2. Fix the RLS policy on `recurring_jobs` to match its sibling — add the `is_business_worker` exclusion and the owner/admin/editor write gate — and add the missing `canEdit(role)` redirect to `src/app/(dashboard)/recurring/page.tsx` . Right now a worker can read every customer's schedule and pricing, and a viewer can delete them; the table predates the worker-isolation rule and was never brought in line.
3. Make the cadence engine honest and prove it with tests: clamp month arithmetic to a `preferred_day_of_month` anchor in `src/lib/recurring/cadence.ts` , wire `preferred_weekday` in or delete the control, and make the recurring-jobs cron check the insert error so a failure retries instead of skipping the visit forever. Then write `src/lib/recurring/tests/cadence.test.ts` — it is a pure function, the tests are cheap, and it is currently wrong for every month-end schedule.

Scheduled

- ☐ **[BLOCKER]** Customer autopay opt-out is ignored — the recurring cron charges saved cards the customer switched off
 - Function · minutes* — `src/lib/stripe-charge.ts:49-51` selects `stripe_customer_id, stripe_payment_method_id, autopay_enabled, allowed_payment_methods`; the eligibility gates at :52-57 check only the two Stripe ids and `customerAllowsCard(allowed_payment_methods)`. `autopay_enabled` is never referenced again through the end of the function (:110). `src/lib/recurring/generate-invoice.ts:77-80` calls `chargeInvoiceToSavedCard` whenever `opts.autoCharge && biz?.stripe_charges_enabled`, where `autoCharge` comes from the schedule's own flag (`src/app/api/cron/recurring-invoices/route.ts:57` `autoCharge: r.auto_charge`; `src/app/api/cron/recurring-jobs/route.ts:120` `autoCharge: r.auto_charge`). Contrast `src/lib/actions/invoices.ts:502-504`, the manual send path, which DOES check `cust?.autopay_enabled && cust?.stripe_payment_method_id` — proving the check was intended and was simply omitted from the shared engine.
 - Impact.** A customer who saves a card and later turns autopay off — via `setCustomerAutopay` (`src/lib/actions/stripe.ts:291`) or the MCP tool (`src/lib/mcp/register-tools.ts:925`) — is still charged off-session every cycle by the daily cron. Chargeback and stored-credential-consent exposure. The manual path honours the flag while the automated path does not, so the inconsistency is invisible in testing.
 - Fix.** In `src/lib/stripe-charge.ts` after line 51 add `if (customer.autopay_enabled === false) return { ok: false, reason: 'not_eligible', message: 'Customer has autopay disabled' };` `generate-invoice.ts:83` then falls through to emailing the pay link, which is correct behaviour. One line, and it also de-duplicates the check now living in `invoices.ts:504`.
- ☐ **[HIGH]** recurring_jobs RLS has no worker exclusion and no role gate — workers and viewers can read and edit job schedules and their pricing
 - Security · hours* — `supabase/migrations/20260419000001_recurring_jobs.sql:33-41` — the sole policy is `FOR ALL USING (business_id IN (businesses WHERE user_id=auth.uid() UNION business_members WHERE user_id=auth.uid() AND status='active'))`, with no `is_business_worker` exclusion and no role filter. Confirmed it is the ONLY policy: `grep` over `supabase/migrations` for 'recurring_jobs' returns hits only in `20260419000001` and in `20260621160000_recurring_billing.sql` lines 4/75/77-80 (the `ALTER TABLE` adding `auto_invoice`, `invoice_line_items`, `auto_charge` — no policy revisit). The sibling pattern exists at `20260621160000_recurring_billing.sql:51` (`AND NOT public.is_business_worker(business_id)`) and :60 (`role IN ('admin','editor')`). `src/app/(dashboard)/recurring/page.tsx:9-40` contains no role lookup and no redirect; `src/app/(dashboard)/recurring-invoices/page.tsx:20-27` does exactly that check.
 - Impact.** A viewer, reachable through the normal web UI with no gate, can create, edit, pause or delete any recurring job schedule including its `invoice_line_items` pricing. A worker is blocked by the page-level path redirect in the web app but is not blocked at the database, so their JWT can read every schedule (customer names, site addresses, line-item prices) and write to the table via any direct PostgREST call. This is the single business-data table that never got the standing `_no_workers` treatment.
 - Fix.** Migration replacing `recurring_jobs_business_access` with the two-policy pattern from `recurring_invoices`: `SELECT` with `AND NOT public.is_business_worker(business_id)`, and `FOR ALL` write restricted to owner plus role `IN ('admin','editor')`. Add the `canEdit(role)` redirect from `recurring-invoices/page.tsx:20-27` to `recurring/page.tsx`. Separately worth fixing: `mobile/app/recurring/index.tsx:29` selects a nonexistent column and that screen is silently broken today.
 - ☐ **[HIGH]** "Run now" does not advance `next_run_on`, so the cron re-bills and re-charges the same cycle hours later
 - Function · hours* — `src/lib/actions/recurring-invoices.ts:113-114` comment: "Does NOT advance `next_run_on` — it's a manual 'bill now'"; line 137 `.update({ last_invoice_id: out.invoiceId, last_run_at: new Date().toISOString() })` — `next_run_on` untouched. `src/app/api/cron/recurring-invoices/route.ts:30-34` selects `.eq('active', true).lte('next_run_on', today)`; the generation loop at :46-58 passes `autoCharge: r.auto_charge` with no reference to `last_run_at` anywhere in the file. Confirmation gate is a native `confirm()` at `src/components/recurring/recurring-invoices-client.tsx:113`.
 -

- **Impact.** Owner clicks Run now on a schedule due today; the 06:00 cron the next morning sees next_run_on still <= today, generates a second invoice for the same period and charges the saved card again. No idempotency key and no duplicate detection exist on this path — generateRecurringInvoice inserts unconditionally.
- **Fix.** Advance next_run_on in runRecurringInvoiceNow when the run covers the current due cycle, or guard the cron by skipping when last_run_at is on/after the next_run_on being processed. Surfacing last_run_at and last_invoice_id on the card would also let the owner see the cycle was already billed.
- ☐ **[HIGH]** recurring-jobs cron ignores the work-order insert error and permanently skips the occurrence
 - *Function · hours* — src/app/api/cron/recurring-jobs/route.ts:73 destructures `const { data: wo } = await (sb as any).from('work_orders').insert({...})` — no error captured. On failure wo is null, so :92 skips assignments and :105 skips auto-billing, but nothing breaks the loop: :129 `generated++` and :130 `occ = advanceOccurrence(occ, r.cadence)` run unconditionally, and :134-142 persists `next_occurrence_at: occ`. The WO number was already burned at :62-68 (read work_order_next_number, write next+1) before the insert. Contrast src/app/api/cron/recurring-invoices/route.ts:63-66, which catches and `break`s with the comment "leave next_run_on as-is so it retries next run".
 - **Impact.** One transient insert failure and that occurrence is gone permanently — the schedule advances past it, generated is incremented, and no error is logged or returned. For a weekly maintenance contract that is a visit the crew never sees, with no signal to anyone. The number counter was already bumped, so the work-order sequence gains a permanent gap pointing at the lost job.
 - **Fix.** Capture error from the insert; on failure console.error and break without incrementing generated, mirroring the invoices cron, so next_occurrence_at is left alone and tomorrow's run retries. Move the number mint to after a successful insert or use an atomic RPC.

Deferred (5) — medium/low on a feature marked *simplify*

- **[MEDIUM]** Monthly and quarterly cadence drift off month-end and never recover — `src/lib/recurring/cadence.ts:8-9` — `case 'monthly': d.setMonth(d.getMonth() + 1); break;` and `case 'quarterly': d.setMonth(d.getMonth() + 3); break;`. The whole file is 12 lines and takes only (current, cadence) — no anchor parameter exists. Both crons call it with exactly two args: `src/app/api/cron/recurring-invoices/route.ts:67` and `src/app/api/cron/recurring-jobs/route.ts:130`. `preferred_day_of_month` exists at `supabase/migrations/20260419000001_recurring_jobs.sql:16` and `20260621160000_recurring_billing.sql:23`, both `INT CHECK (... BETWEEN 1 AND 28)`.
- **[MEDIUM]** `preferred_weekday` and `preferred_day_of_month` are collected, stored and constrained — and read by nothing — `grep -rn 'preferred_weekday|preferred_day_of_month' src/` returns: `src/components/recurring/recurring-jobs-client.tsx:46,47,94,95,310,311,322` (weekday button grid and the `min=1 max=28` day-of-month Input); `src/lib/actions/recurring-jobs.ts:22,23,66,67` (persist); `src/lib/actions/recurring-invoices.ts:20,61` (persist); `src/app/api/agent/route.ts:693,694,1618,1619` (tool schema + write); `src/types/database.ts:377,378,408`. No hit in `src/lib/recurring/cadence.ts`, `src/app/api/cron/recurring-jobs/route.ts`, or `src/app/api/cron/recurring-invoices/route.ts`.
- **[MEDIUM]** Invoice and work-order numbers are minted by non-atomic read-bump under the service role, against the standing rule — `src/lib/recurring/generate-invoice.ts:42-47` — `select invoice_prefix/invoice_next_number`, then a separate `.update({ invoice_next_number: next + 1 })`, then format. The insert follows at :52 and throws at :72 (`if (error || !invoice) throw error ?? new Error(...)`), i.e. after the counter has already been bumped. Same inline pattern at `src/app/api/cron/recurring-jobs/route.ts:62-68` for `work_order_next_number`, with the insert at :73. The rationale is stated in the file header at `generate-invoice.ts:5-7`.
- **[MEDIUM]** Both pages ignore the design system, and the two sibling pages disagree with each other — `grep -c 'ch-'` on both files returns 0; `grep` for `'ui/kirei|EmptyState|StatTile'` returns no matches in `src/components/recurring/recurring-jobs-client.tsx:182` `<h3 className="font-semibold truncate">{s.name}</h3>` and :213 `<span className="truncate">{s.property_address}</span>`; `recurring-invoices-client.tsx:153-154` uses `break-words` for both name and customer. `recurring-jobs-client.tsx:159-164` is a hand-rolled `<button>` with inline Tailwind inside PageHeader actions, where `recurring-invoices-client.tsx:131` uses `<Button>`. Empty states: `recurring-jobs-client.tsx:168-173` is a bare sentence in a Card; `recurring-invoices-client.tsx:140-145` has an icon plus two lines; neither uses the `EmptyState` primitive. `recurring-invoices-client.tsx:113` `if (!confirm(...)) return;` guards Run now, while deletes in both files use `AlertDialog`.
- **[MEDIUM]** Zero tests on the only code path in the product that charges a card unattended — `find src -name '.test.ts' -o -name '.test.tsx'` returns exactly 14 files: `src/lib/tests/pg-filter.test.ts`, `src/lib/assistant/tests/{models,scopes,undo}.test.ts`, `src/lib/booking/tests/{availability,booking-db,time}.test.ts`, `src/lib/content/tests/{live-agents,pipeline,prompts,schedule}.test.ts`, `src/lib/mcp/tests/{collect,invoke,live-api}.test.ts`. Nothing under `src/lib/recurring/` or `src/app/api/cron/`. `src/lib/recurring/cadence.ts` is a 12-line pure function with the confirmed month-end bug; `src/lib/recurring/totals.ts` is a 24-line pure reducer (it does already guard PostgREST string numerics via the `num()` helper at :3-6, so that particular test would be a regression guard, not a bug hunt).

Batch 2.2 — Invoices & payment recording

Health: rough · **Verdict:** keep-core · **Findings:** 0 blocker · 7 high · 0 medium · 3 low **Estimated:** ~1.7 days (10 scheduled)

What it does. Invoices are created from a line-item editor (`invoice-editor.tsx`) that computes subtotal, a percent-or-fixed discount, per-line tax with the discount proportionally allocated before tax, and a total — all in the browser, then written whole to Postgres by `createInvoice`, which mints the number via the race-safe `next_invoice_number` RPC. An invoice can be duplicated, emailed (React-PDF attachment + branded HTML from the per-business template system, plus portal / PDF / Stripe pay links), SMS'd, scheduled, shared via a customer-portal token, and split into progress/deposit child invoices that copy the parent's line items and append a negative "balance due on completion" line. Payments are recorded manually through a dialog (amount, date, method, reference); `addPayment` inserts a `payments` row then recomputes `amount_paid` from

truth (direct payments + children's collections) and derives status paid/partial, rolling the same computation up to the parent — which a DB trigger (`reconcile_parent_invoice`) now also does independently. A daily cron emails payment reminders for past-due invoices. Statuses are draft/sent/partial/paid/overdue/cancelled and, apart from send and payment, are set entirely by hand.

Why this rating. The happy path (create, PDF, email) is solid and well-built. The money path is not: after recording a payment the detail page shows pre-payment figures because client state never syncs, a failed payment insert is swallowed and still reports success, there is no validation on the amount and no way to undo a wrong one. The "Paid" KPI on the list page is string concatenation. Nothing marks an invoice overdue, so the overdue tab, the overdue KPI and the reminder cron disagree with each other. Zero tests cover any of it.

If you only do three things:

1. Fix the payment-recording path as one change: sync the detail component's state after a payment (invoice-detail-client.tsx:60), throw on the swallowed payments insert error (invoices.ts:351), validate the amount is a positive finite number on both sides, and add deletePayment so a mistake is recoverable. Right now the owner cannot record a payment and see it, cannot tell when one silently failed, and cannot undo a wrong one — on the only feature that tracks whether he has been paid.
2. Derive overdue from due_date instead of storing it as a status, so the Overdue tab, the Overdue KPI and the reminder cron finally agree; and fix the reminder cron to send from the correct business with the shared invoice template so reminders carry the Pay-with-card link. Combined, that turns getting paid from a manual chase into something the product does. Also fix the Paid KPI string-concatenation on invoices-client.tsx:73 while in there — it is one Number() call.
3. Delete the JS parent rollup in addPayment (invoices.ts:379-426, ~47 lines). Migration 20260522183034 added a SECURITY DEFINER trigger that recomputes the parent from the identical formula on every child change; the app-side copy is now a redundant second implementation of the same maths that must be kept in sync by hand, and the migration's own header says the trigger exists because the app path kept missing cases. Keep one, and it should be the one that cannot be bypassed.

Scheduled

- ☐ **[HIGH]** Detail page shows stale figures after recording a payment
 - *Function · minutes* — src/components/invoices/invoice-detail-client.tsx:60 `const [invoice, setInvoice] = useState(initial)`; no useEffect syncing `initial` anywhere in the 646-line file. handleAddPayment (:133-153) ends with `router.refresh()` at :150 and never calls setInvoice. src/app/(dashboard)/invoices/[id]/page.tsx:32 renders with no `key`, so React reconciles rather than remounts. The stale state feeds the fact cards (:257-259), the DetailHero status (:181), and the payment history list (:483-494).
 - **Impact.** After 'Payment recorded' the Total/Paid/Balance cards, status pill and payment-history list all still show pre-payment values until a hard reload. Owner reasonably concludes it failed and records again; addPayment has no idempotency, so the second row is accepted and the recompute at :369 sums both.
 - **Fix.** Add `useEffect(() => setInvoice(initial), [initial])` near :60. One line, and it also fixes the identical staleness after chargeSavedCardNow (:511) and scheduleSend (:600), both of which likewise only call router.refresh().
- ☐ **[HIGH]** addPayment ignores the error from the payments insert
 - *Function · minutes* — src/lib/actions/invoices.ts:351 — `await tbl(supabase, "payments").insert({...payment, invoice_id: invoiceId, user_id: user.id, business_id: businessId });` — no destructure, no throw. Contrast :78, :107, :124, :329 in the same file, which all do `if (error) throw error`. Execution falls straight through to the recompute at :357-372 and the function returns void normally.

- **Impact.** A failed insert is indistinguishable from success: the recompute reads the unchanged payments table, writes the same amount_paid back, and the client toasts 'Payment recorded' (invoice-detail-client.tsx:144). Money is not recorded and nobody is told.
- **Fix.** `const { error: payErr } = await tbl(supabase, "payments").insert(...); if (payErr) throw payErr;` immediately before the recompute at :353.
- ☐ **[HIGH]** Payment amount is never validated, client or server
 - *Function · minutes* — src/components/invoices/invoice-detail-client.tsx:137 `const paid = parseFloat(paymentAmount)` fed from the free-text `<Input type="number">` at :535, passed unchecked to addPayment at :138-143; the Record button (:562) is disabled only on `saving`. src/lib/actions/invoices.ts:339 takes `payment: { amount: number; ... }` and performs no finite/positive/ceiling check before the insert at :351.
 - **Impact.** Blank field → NaN → JSON null → NOT NULL violation, swallowed by the previous finding. A fat-fingered 20000 for 2000 writes through and flips status to paid at :370. A negative amount is accepted and silently reduces amount_paid.
 - **Fix.** Guard in addPayment: `if (!Number.isFinite(payment.amount) || payment.amount <= 0) throw new Error(...)`, plus a warn/clamp against `total - amount_paid`. Mirror client-side by disabling the Record button at :562 when the parsed value is not a positive number.
- ☐ **[HIGH]** A recorded payment can never be edited, voided or deleted
 - *Missing · hours* — src/components/invoices/invoice-detail-client.tsx:486-493 renders each payment as a static div (amount, date, method) with no action control. src/lib/actions/invoices.ts exposes only addPayment (:339) and getPayments (:437) for the payments table.
 - **Impact.** Every mistake from the three findings above is permanent from the UI, and the recompute at :369 re-derives from the bad row, so DB-side patching of amount_paid does not stick. Remedy is hand-written SQL against production.
 - **Fix.** Extract :357-372 plus the parent-rollup block (:379-426) into `recalcInvoicePaid(sb, invoiceId, businessId)`, add `deletePayment(paymentId)` that deletes the row and re-runs it, and hang a trash affordance off the payment row at :487 using the AlertDialog already imported at :12.
- ☐ **[HIGH]** "Paid" KPI on the invoices list is string concatenation, not addition
 - *Function · minutes* — src/components/invoices/invoices-client.tsx:73 `.reduce((s, i) => s + i.total, 0)`. invoices.total is `numeric(12,2)` (supabase/migrations/001_initial_schema.sql:92) returned as a string by PostgREST; src/types/database.ts declares it `number` so tsc cannot catch it. The sibling at :69 uses `(i.total - i.amount_paid)` and is accidentally correct, which is why the Outstanding tile works and Paid does not.
 - **Impact.** The 'Paid — all time' StatTile (:137) renders £NaN for any business with two or more paid invoices, sitting next to an Outstanding tile that renders fine — visibly broken on the main invoices page.
 - **Fix.** `reduce((s, i) => s + Number(i.total), 0)` at :73, and `Number(invoice.amount_paid) + paid` at invoice-detail-client.tsx:147.
- ☐ **[HIGH]** Nothing ever marks an invoice overdue, and marking it overdue stops the reminders
 - *Function · hours* — Only writer of 'overdue' is the Smart Organise proposer, src/lib/actions/cleanup.ts:702 `patch: { status: "overdue" }`, reached only via the modal + Apply, plus the manual menu item at src/components/invoices/invoices-client.tsx:223. The dunning cron selects `.in("status", ["sent", "partial"])` at src/app/api/cron/invoice-reminders/route.ts:67 — 'overdue' is absent.
 - **Impact.** Overdue tab (:27) and Overdue KPI (:140, `counts.overdue`) read zero however far past due the book is, contradicting the dashboard. And if the owner does tidy them to overdue, the reminder cron immediately stops chasing exactly those invoices.

- **Fix.** Derive rather than store: `isOverdue = ['sent', 'partial'].includes(status) && due_date < today` for the tab filter, the KPI and the pill; drop the overdue branch at `cleanup.ts:697-707` and the menu item at `invoices-client.tsx:223`. The cron filter is then correct by construction.
- ☐ **[HIGH]** Reminder emails are sent from one hard-coded address for every tenant and carry no pay button
 - *Function · hours* — `src/app/api/cron/invoice-reminders/route.ts:19` `const FROM = process.env.RESEND_FROM_EMAIL ?? "Kirei <noreply@resend.dev>"`, used unchanged at :130 inside the per-business loop with no `replyTo`. Body is hand-assembled at :99-126. Compare the correct path: `src/lib/actions/invoices.ts` imports `buildBusinessFrom` (:7), `invoiceEmailHtml` (:8) and `getResolvedEmailTemplate` (:9) and uses them in `sendInvoiceEmail`.
 - **Impact.** A second tenant's customer receives a payment demand apparently from an unrelated business, with no per-business branding, no portal link, no PDF and no pay-with-card button — on the single email most likely to convert to a payment.
 - **Fix.** Replace FROM with `buildBusinessFrom({ name: biz.name, localPart: "invoices" })` + `replyTo` the business email (`biz.email` is already selected at :45), and swap the inline HTML for `getResolvedEmailTemplate(sb, business_id, "invoice")` + `invoiceEmailHtml` with the same `portalUrl/pdfUrl/payUrl` the send path mints.
- ☐ **[LOW]** List caps at 200 invoices and search only filters what was loaded
 - *UX · hours* — `src/lib/actions/invoices.ts:37` `.limit(filters?.limit ?? 200)` with no offset/cursor. `src/components/invoices/invoices-client.tsx:55-59` filters the in-memory array on number/customer name/email; counts at :61-65 and both KPI reduces at :67-75 are computed over the same loaded slice.
 - **Impact.** Past 200 invoices the oldest silently drops off the page and out of search, and the tab counts and Outstanding/Paid tiles quietly describe only the loaded slice. No warning is shown.
 - **Fix.** Push search + status to the server (`getInvoices` already accepts a status filter) with a debounced query via `ilikeAcross()` from `src/lib/pg-filter.ts`, plus cursor pagination. The leads List view already implements this pattern.
- ☐ **[LOW]** Charging a customer's saved card is confirmed with a browser `confirm()` dialog
 - *UI · minutes* — `src/components/invoices/invoice-detail-client.tsx:507` `if (!confirm( Charge the saved card${customer.stripe_pm_last4 ? `..... ${customer.stripe_pm_last4}` : ""})` for the outstanding balance? `) return;` — an unstyled OS dialog with no amount and no surcharge disclosure, versus the styled `AlertDialog` at :629-640 for delete.
 - **Impact.** The highest-consequence action in the feature gets the least trustworthy confirmation, with no amount breakdown and no card-surcharge disclosure despite surcharging being a supported (and regulated) feature.
 - **Fix.** Swap for an `AlertDialog` stating exact amount, card brand and last4, and any computed surcharge, primary-coloured confirm. Component already imported at :12.
- ☐ **[LOW]** Duplicating an invoice copies the original due date
 - *Function · minutes* — `src/lib/actions/invoices.ts:137-138` — `issue_date: new Date().toISOString().split("T")[0]`, immediately followed by `due_date: invoice.due_date,`. The editor's own default is `addDays(30)` at `src/components/invoices/invoice-editor.tsx:81`.
 - **Impact.** Re-billing a repeat customer by duplicating an older invoice produces a draft issued today but dated due months ago; once sent it is immediately eligible for the dunning cron (`invoice-reminders/route.ts:68`) and the customer can be chased for an invoice they received that morning.
 - **Fix.** Compute `due_date` from today + the business's payment terms, falling back to +30 days, matching `invoice-editor.tsx:81`.

Batch 2.3 — Stripe payments end-to-end (Connect onboarding, checkout, webhook, saved cards/autopay, surcharge, platform fee, receipts)

Health: rough · **Verdict:** keep-core · **Findings:** 0 blocker · 6 high · 1 medium · 2 low **Estimated:** ~4.4 days (9 scheduled)

What it does. Each business links its own Stripe account via Connect Standard onboarding from Settings → Payments (`src/lib/actions/stripe.ts:112`), and customers pay invoices with a direct charge on that connected account while Kirei skims an `application_fee_amount` (default 2%, per-business override). A customer clicks a pay link in an invoice email or the portal, which hits the token-gated `/api/stripe/checkout` route (`src/app/api/stripe/checkout/route.ts:11`) and gets redirected to Stripe's hosted Checkout; on success a webhook (`src/app/api/stripe/webhook/route.ts:11`) records the payment, recomputes `amount_paid` /status on the invoice and any parent invoice, fires a merchant webhook, and emails a branded receipt. Customers can also save a card via a hosted SetupIntent (`/api/stripe/save-card`), after which `sendInvoiceEmail` silently charges them off-session instead of emailing a pay link (`src/lib/actions/invoices.ts:504`). Optional extras: a card surcharge passed to the customer, a quote-accept deposit flow that mints a child invoice, and recurring/auto-billing that reuses the same charge engine. Zero payments have ever run through this in production.

Why this rating. The happy path is well-built and the idempotency/rollup design is genuinely careful, but the money-out half of a payments system does not exist: no refunds, no dispute handling, no failed-async-payment handling. Combined with an unrecoverable webhook failure mode, a payable cancelled invoice, and no post-payment confirmation on the portal page, the first real customer is meaningfully likely to hit something the owner cannot fix from the app.

If you only do three things:

1. Make the webhook resumable and refund-aware. Fix `src/lib/stripe-payments.ts:83` so an already-recorded payment still re-runs `recomputeInvoice/recomputeParent` instead of returning early — that single change removes the 'card charged but invoice unpaid forever' failure mode. Then subscribe to `charge.refunded` and `charge.dispute.created` and reflect them against the invoice. Until refunds exist, taking the first real card payment is a one-way door.
2. Harden the customer-facing pay path. Reject cancelled/draft invoices in `src/app/api/stripe/checkout/route.ts`, wrap the Stripe call in `try/catch` so a failure renders a branded message instead of a 500, and make the portal invoice page read `?paid=1` and `?cancelled=1` so the customer gets confirmation and cannot immediately re-click Pay into a double charge.
3. Build the card-on-file panel on the customer detail page (saved card, autopay toggle, remove card, send save-card link) and stop the webhook auto-enrolling every card update into autopay. Right now the owner can start charging cards automatically but has no dashboard control to stop, which is the one thing that turns a payment problem into an angry phone call.

Scheduled

- ☐ **[HIGH]** No refund, dispute, or failed-payment handling anywhere in the product
 - *Missing · days* — `src/app/api/stripe/webhook/route.ts:31-45` switches on `checkout.session.completed`, `checkout.session.async_payment_succeeded`, `payment_intent.succeeded` and `account.updated`; the default branch at :42-44 is an explicit no-op. A case-insensitive grep for `refund|dispute|chargeback` across `src/` returns exactly one hit: the string literal "billing.refund" in `src/lib/admin/audit.ts:23`. There is no refunds table, no negative-payment path, and no refund action in `src/lib/actions/stripe.ts` (336 lines, verified end to end).

- **Impact.** A merchant refunding a deposit in the Stripe dashboard leaves the Kirei invoice at status 'paid' with amount_paid unchanged, and the payments row intact — revenue reporting is silently wrong with no signal in the app. A chargeback is completely invisible. There is also no way to issue a refund from Kirei at all, so the merchant must always leave the product to do it.
- **Fix.** Subscribe to charge.refunded, charge.dispute.created/closed and checkout.session.async_payment_failed; handle each by writing a negative payments row (or a refunds table) keyed on the refund/dispute id for idempotency, then re-run the invoice recompute. Add a Refund action on the invoice detail calling stripe.refunds.create on the connected account.
- ☐ **[HIGH]** Webhook failure after the payment row inserts leaves the invoice permanently unpaid
 - *Function · hours* — src/lib/stripe-payments.ts:83-85 selects an existing payments row on (business_id, provider_payment_id) and `if (existing) return false;` before doing anything else. The insert is at :97-111; recomputeInvoice at :119 and recomputeParent at :120 run AFTER it, unwrapped in try/catch, so a throw propagates out of recordStripePayment into src/app/api/stripe/webhook/route.ts:46-49 which returns 500. On Stripe's retry, line 83 finds the row written on the first attempt and returns false at :85, so recomputeInvoice never runs and sendPaymentReceipt (:129) never fires.
 - **Impact.** The card is charged and the money is in the merchant's Stripe balance, but the invoice sits at its old status with amount_paid 0, no receipt is sent, and no retry or trigger can heal it. Recovery is manual SQL. recomputeInvoice does two round-trips plus an update, which is the widest failure window in the handler.
 - **Fix.** Make the idempotency check resume rather than abort: when the row already exists, skip only the insert and still run recomputeInvoice/recomputeParent (both are idempotent recomputations from truth), then return 200.
- ☐ **[HIGH]** A cancelled or draft invoice can still be paid, and payment flips it to 'paid'
 - *Function · minutes* — src/app/api/stripe/checkout/route.ts:35-41 selects invoice.status but the only subsequent gates are the token check (:27-32), the Stripe-enabled check (:48-50), the card allow-list (:59-61) and `balance < 0.5` (:63-66) — status is never read. src/lib/stripe-payments.ts:186 then does `const newStatus = newPaid >= total - 0.01 ? "paid" : "partial"` with no status guard, whereas recomputeParent at :204 wraps the same logic in `if (nextStatus !== "cancelled" && nextStatus !== "draft")`, and the migration's SQL does the same at 20260522183034_deposit_parent_reconcile_trigger.sql:48. src/lib/stripe-charge.ts:35-37 also refuses cancelled/draft.
 - **Impact.** Portal tokens are minted with a 90-day expiry (src/lib/actions/stripe.ts:268), and the portal invoice page renders the Pay button for any status that is not paid (page.tsx:247). A customer holding an emailed pay link for a job that was later cancelled can still hand over real money, and the cancelled invoice silently becomes 'paid' — which then needs the refund capability that finding 1 shows does not exist.
 - **Fix.** Reject cancelled/draft in the checkout route with a friendly message, and mirror recomputeParent's status guard inside recomputeInvoice.
- ☐ **[HIGH]** Stripe API errors in the customer-facing checkout and save-card routes surface as a raw 500
 - *Function · hours* — src/app/api/stripe/checkout/route.ts:95-113 calls stripe.checkout.sessions.create with no surrounding try/catch; the only error handling is the `!session.url` check at :115-117. Same shape at src/app/api/stripe/save-card/route.ts:54-61 (customers.create) and :70-83 (sessions.create). The amount floor is a hardcoded `if (balance < 0.5)` at checkout/route.ts:64.
 - **Impact.** This is the first thing a real paying customer touches. Any Stripe-side failure (restricted account, revoked capability, unsupported currency, sub-minimum amount, rate limit) throws unhandled and Next serves a generic 500 on the merchant's branded portal, leaving the customer unsure whether they were charged. With SENTRY_DSN unset the merchant gets no signal at all.
 - **Fix.** Wrap both Stripe calls in try/catch and redirect back to the portal invoice page with an error query param rendering a plain-English message plus the merchant's phone/email. Derive the per-currency minimum instead of the hardcoded 0.5.

- ☐ **[HIGH]** Portal invoice page ignores ?paid=1 — the customer returns from Stripe to an unchanged page with a live Pay button
 - *UX · hours* — src/app/api/stripe/checkout/route.ts:100-101 sets success_url `${baseUrl}/portal/${token}/invoice/${invoice.id}?paid=1` and cancel_url `?cancelled=1`; src/app/api/stripe/save-card/route.ts:75-76 sets ?card_saved=1/?card_cancelled=1. src/app/portal/[token]/invoice/[id]/page.tsx:17 destructures only `{ params }` and the file (315 lines, read in full) never references searchParams. The page is `export const dynamic = "force-dynamic"` (:12) so it re-renders from the DB, and the Pay button at :247-257 is gated only on `!isPaid && balance > 0 && offered.card`. recomputeInvoice (src/lib/stripe-payments.ts:178-187) sums every payments row for the invoice.
 - **Impact.** Payment succeeds, Stripe redirects back, the webhook has not landed, and the customer sees the identical 'Balance due' and 'Pay \$X with card' button they just clicked. Clicking again creates a second session, a second PaymentIntent and a second recorded payment: the customer is double-charged and amount_paid exceeds total.
 - **Fix.** Read searchParams; on ?paid=1 show a success banner and suppress the pay button while refreshing for the webhook, on ?cancelled=1 show a neutral note, on ?card_saved=1 confirm the card. Separately reuse an open Checkout session per invoice so a second click cannot create a second charge.
- ☐ **[HIGH]** Autopay and saved cards have no web UI at all — only MCP tools, and the AI is dead in production
 - *Missing · days* — The three actions exist at src/lib/actions/stripe.ts:275 (getSaveCardLink), :286 (setCustomerAutopay) and :298 (removeSavedCard), and the MCP tool at src/lib/mcp/register-tools.ts:925. Their only other appearances are src/app/api/stripe/webhook/route.ts:161 and the customer-facing src/app/portal/[token]/invoice/[id]/page.tsx:270-284. src/components/invoices/invoice-detail-client.tsx:503-521 has a 'Charge saved card' button but no autopay toggle or card removal. Webhook route.ts:155-162 updates the customer row with `autopay_enabled: true` on every setup-mode session with no conditional.
 - **Impact.** When a customer phones to stop card charges, the owner has no saved-card display, no autopay toggle and no remove-card button in the app. The alternatives are the AI assistant (401 in production) or raw SQL. Worse, a customer who merely clicks the portal's 'Update card' link (page.tsx:274) silently re-enables autopay the merchant had turned off.
 - **Fix.** Add a card-on-file section to the customer detail page (brand/last4/expiry, autopay switch wired to setCustomerAutopay, Remove card with confirmation, Send save-card link). Change webhook handleCardSaved to set autopay_enabled: true only when the setup session metadata explicitly requested enrolment.
- ☐ **[MEDIUM]** Every money-computing function in the payment path is untested
 - *Tests · hours* — find src -name '.test.ts' returns 14 files: 3 under lib/assistant/**tests**, 3 under lib/booking/**tests**, 4 under lib/content/**tests**, 3 under lib/mcp/**tests**, and src/lib/**tests**/pg-filter.test.ts. Nothing covers src/lib/stripe.ts (toStripeAmount :36, computeSurcharge :60, computeApplicationFeeAmount :91), src/lib/payment-methods.ts, src/lib/stripe-payments.ts or src/lib/stripe-charge.ts.
 - **Impact.** These are pure, dependency-free functions deciding how much money is taken and how much is credited, and any refactor can silently change a charge amount with nothing to catch it. The parent/child rollup in recomputeParent has already caused one production incident per the progress-invoice trap in CLAUDE.md and is still unguarded.
 - **Fix.** Add src/lib/**tests**/stripe-amounts.test.ts (to/fromStripeAmount round-trips for 2-decimal, zero-decimal JPY and 3-decimal KWD; computeSurcharge off/percent/fixed/both/rounding; computeApplicationFeeAmount null-to-env-default and 0-no-fee), a payment-methods test for NULL-

inherits vs explicit-list plus the cash opt-in, and a stripe-payments test with a fake sb asserting no double-insert on a duplicate PI, 23505 swallowed, child rollup to parent, and a cancelled parent never flipped to paid.

- ☐ **[LOW]** Payment surfaces hand-roll styles and use native confirm() instead of the design system
- *UI · hours* — src/app/portal/[token]/invoice/[id]/page.tsx:304-314 defines a local StatusBadge with a private colour map for paid/sent/overdue/partial/draft/cancelled instead of the .ch-pill vocabulary or KireiPill; the pay CTA at :248-256 is a raw with inline Tailwind rather than Button.
src/components/settings/stripe-settings.tsx:64 gates Disconnect Stripe with window.confirm, and src/components/invoices/invoice-detail-client.tsx:507 gates charging a real customer's saved card with window.confirm.
- **Impact.** Portal status colours can drift from the rest of the app, and the two most consequential actions in the feature — severing the merchant's Stripe link and taking money off a stored card — are gated by an unstyled browser dialog that looks nothing like the product. Judged against src/components/leads/, this is the least-finished surface in the feature.
- **Fix.** Replace the local StatusBadge with the shared .ch-pill/KireiPill vocabulary, use Button for the pay CTA, and swap both confirm() calls for the existing AlertDialog with the amount and last4 stated in the body.
- ☐ **[LOW]** Card surcharge under-recovers because it is computed on the balance, not the grossed-up total
- *Function · minutes* — src/lib/stripe.ts:60-66 computes `(amount * pct) / 100 + fixed` on whatever amount it is handed. src/app/api/stripe/checkout/route.ts:73-77 passes the bare `balance` (computed at :63) and then charges balance + surcharge as two line items (:82-91). src/lib/stripe-charge.ts:60-65 does the same, with chargeTotal = balance + surcharge at :65.
- **Impact.** Stripe's processing fee applies to the full charged amount, so a merchant setting the surcharge to exactly their cost of acceptance still eats the fee-on-the-fee. Under-recovery is the legally safe direction (over-recovery is what regulators penalise), and the settings warning already tells merchants to stay at or below cost — so this is a rounding-level shortfall, not a compliance risk.
- **Fix.** Either gross up as $((\text{amount} + \text{fixed}) / (1 - \text{pct}/100) - \text{amount})$, or add one line to the surcharge help text noting the fee is calculated on the invoice balance and will slightly under-recover.

Batch 2.4 — Quotes, acceptance & deposits

Health: usable · **Verdict:** keep-core · **Findings:** 0 blocker · 2 high · 4 medium · 2 low **Estimated:** ~2.7 days (8 scheduled)

What it does. A business creates a quote (line items, discount, tax, expiry date), which mints a number atomically via the next_quote_number RPC (src/lib/actions/quotes.ts:68). It can be emailed with a PDF attachment plus a tokenised portal link and a tokenised PDF re-download link (quotes.ts:194-283), or SMS'd (quotes.ts:285-334). The customer opens /portal/[token]/quote/[id], sees the quote, and can either click "Accept" — which only flips the quote's status to accepted and does nothing else (src/app/api/portal/[token]/quote/[id]/accept/route.ts:43-51) — or, if the business has Stripe Connect live and a non-zero deposit_percent, "Accept & pay X% deposit", which converts the quote into a full invoice, mints a deposit child invoice, and hands off to the existing Stripe Checkout route (accept-with-deposit/route.ts:98-184). Separately the business can convert a quote to an invoice manually from the list or detail page. A daily cron at 10:00 UTC (src/app/api/cron/quote-followup/route.ts) emails a nudge for sent quotes expiring within 3 days or expired within 2 days, for businesses that installed the "quote-followup" agent. Nothing ever automatically marks a quote expired — that is manual, or via the Smart Organise proposer at src/lib/actions/cleanup.ts:739.

Why this rating. The happy path works and the dashboard UI is on-system (composes StatTile/KireiTabs/KireiPill/DetailHero correctly). But the two moments that matter commercially are broken: when a customer accepts a quote nobody is told (no webhook, no email, no notification — the `quote.accepted` webhook event is declared in src/types/database.ts:1233 and dispatched from nowhere in the codebase), and

the automated follow-up email — the only thing in the product that chases a \$327k pipeline — has no accept link in it and is sent from the wrong business's address. Conversion to invoice has no idempotency guard on the server and races on the invoice counter. Zero tests touch any of it.

If you only do three things:

1. Make acceptance an event, not a status change. In `accept/route.ts:43-51` and `accept-with-deposit/route.ts:127-130`, dispatch the already-declared `quote.accepted` webhook and email the business owner. Right now the highest-value moment in the product happens in total silence, and a declared webhook event fires from nowhere in the codebase.
2. Fix the follow-up cron: send from `buildBusinessFrom` instead of the global `RESEND_FROM_EMAIL` (`route.ts:19`), put a portal accept link in the email body (it currently says 'reply to this email'), and guard the null-name crash at :107 that can kill an entire day's run for every business. This is the only automated lever on a \$327k open pipeline and it currently has no button and the wrong sender.
3. Make conversion safe: add the `business_id` filter and an `invoice_id` idempotency guard to `convertQuoteToInvoice` (`quotes.ts:142`) and its MCP twin (`register-tools.ts:739`), and move all three invoice-number mints onto an atomic increment. Then write the two test files that lock the deposit arithmetic and the conversion guard in place.

Scheduled

- ☐ **[HIGH]** Customer accepts a quote and nobody is told
 - *Missing · hours* — `src/app/api/portal/[token]/quote/[id]/accept/route.ts:43-51` is exactly as described - one `UPDATE` to `status='accepted'` then `return NextResponse.json({ ok: true })`, no `dispatchWebhook`, no `sendEmail`, no notification insert. `accept-with-deposit/route.ts:127-130` likewise only updates the quote row. Grep for 'quote.accepted' across `src/` returns exactly two hits, both declarations: `src/types/database.ts:1233` (`WebhookEvent` union) and :1256 (`ALL_WEBHOOK_EVENTS` picker label). No dispatch site exists. Contrast confirmed at `src/lib/actions/quotes.ts:79` (`dispatchWebhook(businessId, "quote.created", data)`) and :281 (`dispatchWebhook(businessId, "quote.sent", ...)`).
 - **Impact.** A business owner subscribing to `quote.accepted` in the webhook UI gets an event that can never fire, and there is no email or in-app notification either. Acceptance is discovered only by opening `/quotes` and noticing a pill changed. For the one dependent roofing business with \$327k of open quotes, a Saturday acceptance sits unseen.
 - **Fix.** After the successful update in `accept/route.ts`, call `dispatchWebhook(link.business_id, 'quote.accepted', {...})` and send a notification email to the business via `buildBusinessFrom` + the template engine. Mirror it in `accept-with-deposit/route.ts` after line 127/129.
- ☐ **[HIGH]** `convertQuoteToInvoice` has no idempotency guard and no `business_id` filter
 - *Function · hours* — `src/lib/actions/quotes.ts:142` reads `await tbl(supabase, "quotes").select("*").eq("id", quoteId).single()` with no `business_id` predicate; `getQuote` at :53-54 does filter, confirming the inconsistency. Nothing between :142 and the insert at :158 checks `quote.invoice_id`, and :185 unconditionally overwrites `invoice_id` with the new invoice. The MCP tool at `register-tools.ts` (`convert_quote_to_invoice`) reads `.select("*").eq("id", args.quote_id).eq("business_id", ctx.businessId).maybeSingle()` - `business_id` IS filtered; only the `invoice_id` check is absent. Also confirmed: no unique constraint on `invoices(business_id, number)` exists in `supabase/migrations`, so a duplicate persists silently. UI-only protection confirmed at `quotes-client.tsx` (converting state) and the `accept-with-deposit` route implements the reuse check the action lacks at lines 91-96.
 - **Impact.** Double-click on a slow connection, or the AI assistant calling `convert_quote_to_invoice` twice in a chain, produces two invoices for one job and orphans the first (`invoice_id` is overwritten at :185). Converting a quote whose deposit already created a parent invoice at `accept-with-deposit:101` produces a

second parent. RLS still blocks cross-business reads for a single-business user, so the business_id gap only bites a user who is a member of both businesses.

- **Fix.** Add `.eq("business_id", businessId)` to quotes.ts:142, and early-return the existing invoice when quote.invoice_id is set and the row still exists - lift the reuse check already written at accept-with-deposit/route.ts:91-96 into a shared helper and call it from quotes.ts and register-tools.ts.
- ☐ **[MEDIUM]** Follow-up cron sends from the wrong business and has no accept link
- *Function · hours* — src/app/api/cron/quote-followup/route.ts:19 - `const FROM = process.env.RESEND_FROM_EMAIL ?? "Kirei <noreply@resend.dev>"`, module-scope, used verbatim at :133 inside the per-business loop that starts at :58. No call to buildBusinessFrom and no getResolvedEmailTemplate anywhere in the file (grep clean). The email body at :106-129 has no link element at all; :126-128 is the prose 'Simply reply to this email'. Contrast src/lib/actions/quotes.ts:270 (buildBusinessFrom), :263 (getResolvedEmailTemplate), and :238-260 which mints/reuses a portal token and builds `${base}/portal/${token}/quote/${id}`. Cron is registered in vercel.json (path /api/cron/quote-followup).
- **Impact.** Any business enabling the quote-followup agent sends chase emails from a From address that is not theirs, and per-business email templates are ignored. The follow-up gives the customer no way to accept - the portal accept flow and the whole deposit path behind it are unreachable from the one automated nudge in the product.
- **Fix.** Build FROM per iteration with buildBusinessFrom({ name: biz.name, localPart: 'quotes' }); mint/reuse a portal token as quotes.ts:238-260 does and add a 'Review & accept' button to /portal/{token}/quote/{id}; route the body through getResolvedEmailTemplate(sb, business_id, 'quote').
- ☐ **[MEDIUM]** Customers can accept but cannot decline
- *Missing · hours* — src/app/portal/[token]/quote/[id]/page.tsx:169-193 - the action Card renders only AcceptAndDepositButton (:174) and AcceptQuoteButton (:180); the else branch at :188-193 is a static 'Quote accepted' card. No decline control anywhere in the file. `ls src/app/api/portal/[token]/quote/[id]/` returns exactly two entries: accept, accept-with-deposit. The rejected status is business-settable only: quotes-client.tsx:26 tab, register-tools.ts set_quote_status enum, and both portal routes only read it (accept/route.ts:39, accept-with-deposit:59).
- **Impact.** Declined-in-practice quotes stay 'sent' forever, inflating the Open pipeline figure at quotes-client.tsx:60 and continuing to trigger follow-up emails to customers who already went elsewhere. No loss-reason is ever captured.
- **Fix.** Add POST /api/portal/[token]/quote/[id]/decline mirroring accept/route.ts, with an optional free-text reason column, a secondary 'Not proceeding' button on the portal page, and a notification to the business (same dispatch gap as finding 1 - fix both together).
- ☐ **[MEDIUM]** Nothing ever marks a quote expired, so the expiry date is decorative
- *Function · hours* — Grep for 'expired' across src/lib and src/app/api returns only: cleanup.ts:747 (the manual Smart Organise proposer, `patch: { status: "expired" }`), the MCP/agent status enums (register-tools.ts:699, :718; api/agent:769,793; api/mobile/agent:86; api/v1/agent:190), and the two portal routes reading it (accept:39, accept-with-deposit:59). No cron writes it; quote-followup/route.ts only reads expiry_date (:39-40, :69-70) and never updates status. quotes-client.tsx:27 renders the Expired tab unconditionally; :60 sums pipeline from status==='sent'; :62-64 computes counts.accepted / (accepted+rejected+expired). page.tsx:46 treats any non-accepted/rejected/expired quote as acceptable, so a long-expired 'sent' quote is still acceptable at its original price.
- **Impact.** A quote that lapsed months ago is still status 'sent': counted in Open pipeline, still bindable by the customer at stale pricing through the portal, and absent from the acceptance-rate denominator so the rate reads better than reality.
- **Fix.** Extend the quote-followup cron (it already computes today's boundary dates) with an expiry sweep: update sent quotes with expiry_date < today - grace to 'expired'. Run it business-wide, not gated on the agent install, since the status governs portal acceptability.

- ☐ **[MEDIUM]** Zero tests on the money path
 - *Tests · hours* — `find src -name "*.test.ts"` returns exactly 14 files: pg-filter, assistant/{models,scopes,undo}, booking/{availability,booking-db,time}, content/{live-agents,pipeline,prompts,schedule}, mcp/{collect,invoke,live-api}. None reference quotes, conversion, deposits, or the portal. The uncovered logic is real: the deposit arithmetic at accept-with-deposit/route.ts:85, the reusable-child predicate at :138-141 (`c.status !== "paid" && Number(c.amount_paid ?? 0) < Number(c.total ?? 0) - 0.01`), and the stale-parent recovery at :91-96.
 - **Impact.** Any refactor of the deposit route is a blind change against a live Stripe Connect account. The reuse predicate and the rounding are the kind of logic that fails quietly on edge inputs (odd percentages, zero-total children).
 - **Fix.** Lift the arithmetic and the reuse predicate out of the route handler into pure functions and cover them in src/lib/**tests**/quote-deposit.test.ts. Add a convertQuoteToInvoice test asserting idempotency and cross-business refusal once finding 2's guard lands.
- ☐ **[LOW]** Invoice numbers are minted by read-modify-write in three places, twice in one request
 - *Function · hours* — quotes.ts:145-153 selects invoice_prefix/invoice_next_number then writes back next+1. register-tools.ts mintNumber does the same, with the rationale comment at :55-64. accept-with-deposit/route.ts:24-33 defines mintInvoiceNumber with the same pattern and its own comment at :25, called at :99 and :145. next_invoice_number(uuid) does exist and is atomic (supabase/migrations/20260511020100_perf_atomic_number_mint.sql:14-45, single UPDATE ... RETURNING) and is used by src/lib/actions/invoices.ts:71 and :237 - so quotes.ts is the one cookie-bound path that could use it and does not. createQuote at quotes.ts:66-69 does use next_quote_number. No unique index on invoices(business_id, number) exists, so a collision would persist silently rather than erroring.
 - **Impact.** Under concurrent accepts, two invoices can carry the same number; with no unique constraint the DB will not catch it, and duplicate numbers are painful to unwind for tax reporting. Realistic exposure today is very low given invoice volume.
 - **Fix.** Switch quotes.ts:145-153 to the next_invoice_number RPC. For the service-role paths, either add a p_business_id variant that skips the auth.uid() check or replace the read-bump with a single `update businesses set invoice_next_number = invoice_next_number + 1 ... returning`. Adding a unique index on (business_id, number) would also surface collisions instead of hiding them.
- ☐ **[LOW]** Portal quote page hand-rolls a status badge with different colours than the dashboard, and errors via alert()
 - *UI · minutes* — src/app/portal/[token]/quote/[id]/page.tsx:212-221 defines a local StatusBadge whose map has `draft: "bg-amber-500/15 text-amber-600..."`. quotes-client.tsx:32-38 STATUS_GRADIENT has `draft: "violet"` and :180 renders `<KireiPill tone={quote.status} />`. Neither uses .ch-pill, which is defined in globals.css:455-473. src/components/customer-portal/accept-quote-button.tsx:18 is `alert(err.error || "Failed to accept")`; the sibling accept-and-deposit-button.tsx:18/26/40 uses useState error + `<p className="text-xs text-destructive">`. quotes-client.tsx:71-72 uses sonner toasts.
 - **Impact.** The only Kirei surface a paying client ever sees shows a browser-chrome alert box on failure and a status colour that contradicts what the business sees on the same quote. Against the leads workspace, which composes the shared primitives throughout, this page is a fork.
 - **Fix.** Replace the local StatusBadge with KireiPill (tone={quote.status}), and replace alert() in accept-quote-button.tsx with the inline error pattern its sibling accept-and-deposit-button.tsx already uses.

Tier B — the job

Batch 2.5 — Customers, sites & contacts

Health: rough · **Verdict:** keep-core · **Findings:** 0 blocker · 3 high · 7 medium · 0 low **Estimated:** ~3.6 days (10 scheduled)

What it does. Customers are the spine of Kirei: `customers` rows (business-scoped, soft-archivable) that invoices, quotes, work orders, reports, leads and portal tokens all FK to. `/customers` renders a client-filtered card list (search + Active/Archived/All tabs + KPI tiles + multi-select bulk archive/delete + CSV import), backed by `getCustomers()` which pulls up to 1000 rows through a 30s `unstable_cache`. `/customers/[id]` is a hub: a DetailHero, six stat tiles, and eight tabs (Properties, Contacts, Jobs, Invoices, Quotes, Reports, Billing profiles, Notes, plus a conditional Onboarding tab), each fed by ~11 parallel server fetches. "Properties" are actually `sites` rows surfaced through a compatibility shim (`customer-hub.ts`) that maps the old `customer_properties` / `customer_contacts` shapes onto the new `sites` / `account_contacts` tables. `/sites/[id]` is a separate read/write hub for one site (address, access notes, assets, jobs, billing profile) with a read-only contacts tab. `/contacts` is a third, unrelated CRM surface over the `contacts` table with lifecycle stages (lead/contact/customer) and a promote-to-customer action. Search everywhere in this feature is client-side string matching — the `.or()` PostgREST trap is genuinely fixed, every server-side search path routes through `ilikeAcross()`.

Why this rating. The happy path works and the list/detail pages are visually on-system, but three defects sit on the primary path: the state/region field is silently discarded on every customer save, the "Total spent" figure on the customer hub is computed by string concatenation and goes wrong the moment a customer has two paid invoices, and archive actions on sites/contacts/billing profiles ignore their database error so the UI reports success on failure. On top of that the contact model is split across three tables with a dead-end instruction in the UI pointing users between them, and there is not one test file covering any of it.

If you only do three things:

1. Fix the three silent-correctness bugs in one small PR: add `state` to the customer-form submit payload (customer-form.tsx:99-118), coerce the money reducers with `Number()` (customer-detail-client.tsx:494-495), and check `error` in the three archive actions (sites.ts:112, account-contacts.ts:76, billing-profiles.ts:100). All are minutes of work and all currently lie to the user about what was saved.
2. Collapse the three contact stores into one. Point the customer hub's Contacts tab at the `contacts` table, wire the already-written `linkContactToSite` into the site page so site contacts can actually be created, then delete `account_contacts` and the contact half of `customer-hub.ts`. This is the deletion the owner is looking for — it removes a table, a shim and a dead-end instruction while ADDING the per-site contact capability competitors have.
3. Make archiving reversible and make `/contacts` look like the rest of the product: add an Unarchive action on customers, an `includeArchived` path plus restore for contacts, and rewrite `contacts-client.tsx` on the kirei primitives (`KireiAvatar`, `KireiTabs`, `KireiPill`, `EmptyState`, `StatTile`) with break-words instead of truncate, matching the leads workspace bar.

Scheduled

- ☐ **[HIGH]** Customer state/region is silently discarded on every save
 - *Function · minutes* — `src/components/customers/customer-form.tsx:53` declares `state:` `z.string().optional()`, :87 seeds `state: customer?.state ?? ""`, and :233/:240 wire it into `AddressFields` (read at :228-244). The payload object at :100-120 lists name, email, phone, secondary_phone, company, contact_role, website, tax_number, account_type, preferred_contact, address,

city, postcode, country, notes, allowed_payment_methods — `state` is absent. The payload is passed verbatim to `updateCustomer/createCustomer` (:121-123), and `src/lib/actions/customers.ts:52-53` declares `state` as an accepted optional column, so nothing downstream re-adds it. Column exists: `supabase/migrations/20260507000002_address_state.sql:9`.

- **Impact.** Any user entering a state/region (NSW, CA) sees it accepted and lost. Because `updateCustomer` sends a partial payload, an existing state value is not overwritten — but it can never be set or changed from the UI. Address is incomplete on every PDF/email for customers created through this form.
- **Fix.** Add `state: data.state || null`, to the payload object at `src/components/customers/customer-form.tsx:112`, next to `city`.
- ☐ **[HIGH]** "Total spent" stat concatenates strings instead of summing money
- *Function · minutes* — `src/components/customers/customer-detail-client.tsx:494` `reduce((s, i) => s + i.total, 0)` with no `Number()`. `invoices.total` is `numeric(12,2)` (`supabase/migrations/001_initial_schema.sql:92`) and `getInvoices` (`src/lib/actions/invoices.ts:19-44`) selects `total/amount_paid` raw with no coercion. The repo documents the string-return behaviour for these exact columns at `src/lib/emails/invoice.ts:75-79`. Line 495 (`outstanding`) is accidentally safe because `i.total - i.amount_paid` coerces before the add. Value renders via `formatCurrency` (`src/lib/utils.ts:8-15`) → `Intl.NumberFormat.format`, which coerces a single-invoice string fine but yields NaN once two are concatenated.
- **Impact.** A customer with 2+ paid invoices shows a NaN/garbage "Total spent" tile (`customer-detail-client.tsx:574`). Single-paid-invoice customers happen to render correctly, which makes the bug intermittent and hard for the owner to attribute.
- **Fix.** Coerce both reducers: `s + Number(i.total ?? 0)` and `s + (Number(i.total ?? 0) - Number(i.amount_paid ?? 0))`. Apply the same fix at `src/components/invoices/invoices-client.tsx:69` and :73 in the same pass.
- ☐ **[HIGH]** Site contacts are read-only and point users at a page that writes to a different table
- *Function · days* — `src/lib/actions/sites.ts:44-51` `getSiteContactsFull` joins `site_contacts` → `contacts(*)`; the FK is `contact_id UUID NOT NULL REFERENCES public.contacts(id)` (`supabase/migrations/20260418000001_account_site_portfolio.sql:103`). The site UI renders the list then a footer reading "Manage contacts on the account page" (`src/components/sites/site-detail-client.tsx:298-322`) with no add/link control. The account page's Contacts tab goes `customer-hub.ts:119-131` `createCustomerContact` → `account-contacts.ts:50-58`, which inserts into `account_contacts` — a different table (import confirmed at `customer-hub.ts:20-22`). `linkContactToSite` exists at `sites.ts:121-126`; a repo-wide grep finds zero callers (only `getSiteContactsFull` is consumed, by `src/app/(dashboard)/sites/[id]/page.tsx:18`).
- **Impact.** A user follows the on-screen instruction, adds a site manager on the account page, and it never appears on the site. Site contacts can only be created by direct SQL. This is the per-site tenant/super/building-manager workflow ServiceM8 and Jobber sell, and it is inert.
- **Fix.** Additive first: add a "Link contact" picker on the site page calling the existing `linkContactToSite` / `unlinkContactFromSite`, and correct the misleading footer text at `site-detail-client.tsx:320-322`. Any consolidation of `account_contacts` into `contacts` is a separate migration with its own review.
- ☐ **[MEDIUM]** Archive actions swallow their database error and report success anyway
- *Function · minutes* — `src/lib/actions/sites.ts:106-109` `archiveSite` — `await tbl(...).update({archived:true})...` with no destructure of `error` (and no `revalidatePath`, unlike its siblings at :90 and :102). `src/lib/actions/account-contacts.ts:73-76` `archiveContact` and `src/lib/actions/billing-profiles.ts:98-101` `archiveBillingProfile` are identical. Every neighbouring function in those files does `if (error) throw error`. Callers: `src/components/customers/customer-`

detail-client.tsx:513-527 removes the row from local state and fires `toast.success("Deleted")` (:527) unconditionally; `deleteCustomerProperty/deleteCustomerContact` (src/lib/actions/customer-hub.ts:107-110, :151-154) just await the archive helpers.

- **Impact.** On an RLS denial or transient failure the card vanishes with a success toast and reappears on next load. Silent, unexplained divergence between what the user believes and what is stored.
- **Fix.** `const { error } = await tbl(...); if (error) throw error;` in all three functions — matching sites.ts:27, account-contacts.ts:56, billing-profiles.ts:28. Add the missing `revalidatePath` to `archiveSite` while there.
- ☐ **[MEDIUM]** Nothing can be unarchived — and the confirmation dialog promises it can
- *Missing · hours* — src/components/customers/customers-client.tsx:324 promises "You can find them under the Archived tab and unarchive any of them later"; the row dropdown at :265-278 offers View/New invoice/New quote/Archive (:270-274, gated on `!customer.archived`)/Delete only. A repo-wide grep for `archived: false` outside create paths and for "unarchive" returns nothing but that dialog string — no unarchive exists in components or in src/lib/mcp/register-tools.ts. For contacts, src/lib/actions/contacts.ts:28-39 `listContacts` hard-codes `.eq("archived", false)` with no `includeArchived` option, and src/components/contacts/contacts-client.tsx:359 says "You can restore it from the database if needed."
- **Impact.** Archiving is a one-way door the UI describes as reversible. Archived contacts are unreachable from the product entirely — only recoverable by direct SQL.
- **Fix.** Add an "Unarchive" dropdown item in customers-client.tsx calling `updateCustomer(id, { archived: false })` when `customer.archived`. Give `listContacts` an `includeArchived` flag plus an Archived filter and an unarchive action. Add matching MCP coverage per the standing rule.
- ☐ **[MEDIUM]** `getCustomers` caches by business only, so a warmed cache can serve rows RLS would deny
- *Security · hours* — src/lib/actions/customers.ts:19-33 wraps the query in `unstable_cache` keyed `[ customers- $\{businessId\}$ - $\{includeArchived\}$  ]`, tag `customers- $\{businessId\}$` , revalidate 30 — no user or role component. The Supabase client is created outside the cache callback (:14), so nothing dynamic forces a bail-out; on a hit the query never runs and `customers_no_workers` never applies. There is no role check in the action. The worker restriction is a redirect in src/app/(dashboard)/layout.tsx:60-66 based on `x-pathname` — a UI guard on page navigation, not an auth gate on the action.
- **Impact.** A worker who invokes the action within the 30s window after an owner loads /customers receives the full customer list with emails and phones. Worker hard-isolation is the boundary this product sells to crews.
- **Fix.** Include the caller's role (or drop caching here — the select is already bounded at 1000 rows). Given RLS is the only gate everywhere else, caching an RLS-scoped result under a role-blind key is the wrong trade.
- ☐ **[MEDIUM]** /contacts is off the design system and is the visual outlier of the whole feature
- *UX · hours* — src/components/contacts/contacts-client.tsx: hand-rolled initials avatar at :224-226 (not KireiAvatar), hand-rolled stage filter buttons at :190-203 (not KireiTabs), hand-rolled empty state at :208-218 (not the EmptyState primitive), coloured stage dots defined at :35-39 and rendered at :234 (not KireiPill/.ch-pill). `truncate` on name (:231), company (:241) and email (:243), against the owner's stated break-words preference. No StatTile row. src/app/(dashboard)/contacts/page.tsx:7 wraps in `<div data-theme="console" className="bg-background text-foreground rounded-lg p-6 sm:p-10 min-h-full">`, nesting padding inside the shell's own p-6 — CLAUDE.md forbids per-page outer wrappers on list pages.
- **Impact.** Contacts reads as a different product from /customers and /leads next to it in the sidebar. On mobile, truncation clips emails and company names to unreadable stubs.

- **Fix.** Recompose contacts-client.tsx on the kirei primitives the way customers-client.tsx does (KireiAvatar, KireiTabs, KireiPill, EmptyState, a StatTile lifecycle row), swap truncate for break-words, and delete the wrapper div in contacts/page.tsx.
- ☐ **[MEDIUM]** Customer detail turns every failure into a 404
 - *Function · hours* — src/app/(dashboard)/customers/[id]/page.tsx:52-84: one `try` around `Promise.all` of eleven fetches, with `} catch { notFound(); }` at :82-84 and no error inspection. Four fetches carry their own `.catch(() => [])` (:60-63) and loadOnboarding swallows its own errors (:34-36), so the bare catch fires only for getCustomer, getInvoices, getQuotes, getBusiness, getWorkOrders or getReports. getCustomer uses `.single()` (src/lib/actions/customers.ts:46) and throws on no row, so a genuine 404 and a DB timeout are indistinguishable.
 - **Impact.** A transient Supabase error tells the user their customer does not exist. With SENTRY_DSN unset, nothing is logged and the owner never learns it happened.
 - **Fix.** Switch getCustomer to `.maybeSingle()` and call notFound() only when it returns null; rethrow everything else so the error boundary renders a real error state.
- ☐ **[MEDIUM]** No duplicate prevention on customer create — leads have it, customers don't
 - *Missing · hours* — src/lib/actions/customers.ts:55-70 `createCustomer` inserts with no lookup on email or phone. src/lib/actions/contacts.ts:136-177 `promoteContactToCustomer` inserts a customer unconditionally when contact.customer_id is null. Contrast the leads path (generated identity_key + unique index, per CLAUDE.md). Existing mitigation is after-the-fact only: src/lib/actions/cleanup.ts:137-176 clusters customers by email > phone > name+address and proposes merges with FK relinking.
 - **Impact.** Duplicate customer accounts accumulate, splitting invoice history for the same person. The existence of a bespoke merge proposer suggests this is already occurring.
 - **Fix.** Add a soft dedup check in createCustomer using the same clustering keys as proposeCustomerCleanup — return the existing row or surface a "possible duplicate" confirmation in customer-form.tsx before insert.
- ☐ **[MEDIUM]** Zero tests across the entire feature
 - *Tests · hours* — Test files under src/: src/lib/**tests**/pg-filter.test.ts; src/lib/assistant/**tests**/{models,scopes,undo}.test.ts; src/lib/booking/**tests**/{availability,booking-db,time}.test.ts + setup.ts; src/lib/content/**tests**/{live-agents,pipeline,prompts,schedule}.test.ts; src/lib/mcp/**tests**/{collect,invoke,live-api}.test.ts. None touch customers, sites, contacts, account-contacts, customer-hub or billing-profiles.
 - **Impact.** The three confirmed bugs above (dropped state, string-concat money, swallowed archive errors) are all trivially unit-testable and all shipped.
 - **Fix.** Extract the totalSpent/outstanding reducers from customer-detail-client.tsx into a pure helper and unit-test string-numeric coercion; add a payload round-trip test asserting the customer-form submit object covers every column its zod schema declares (catches the missing `state` and the next omission); add a DB-backed, auto-skipping archive test asserting the three archive actions reject on failure.

Batch 2.6 — Schedule & dispatch

Health: rough · **Verdict:** keep-core · **Findings:** 0 blocker · 3 high · 5 medium · 1 low **Estimated:** ~3.9 days (9 scheduled)

What it does. A weekly job calendar over the `work_orders` table. `/schedule` server-renders one Mon–Sun week (`src/app/(dashboard)/schedule/page.tsx:36`) plus all member profiles and customers, then hands off to a client component with two views: a 7-column "Week" grid of status-coloured job cards, and a "Dispatch" board that is a worker-rows × 7-day-columns matrix with an "Unassigned" row. Clicking any cell or card opens a modal that creates/edits a work order (title, date, start/end time, customer with address autofill, status, multi-worker assignment, description) and can delete it behind a confirm dialog. On the dispatch board you can drag a job

card to another worker/day cell, which calls `rescheduleJob` to change `scheduled_date` and the assignment set. Week navigation refetches client-side via the `getScheduledJobs` server action. There is no month view, no time-grid/hour axis, no route or travel planning, no recurring-job surface, and no conflict detection — worker reminder emails are handled separately by `/api/cron/reminders`.

Why this rating. The date arithmetic is broken for every user east of UTC — which is literally the only dependent customer (a Sydney roofing business with 98 jobs). I executed the code under TZ=Australia/Sydney: the rendered week is shifted one day, "Today" lands on a week that does not contain today, and each next/prev click advances only 6 days so the week boundary walks backwards through the calendar. Separately, the dispatch board's drag-to-reassign adds a second worker instead of moving the job, and its drag doesn't initiate in Firefox or on touch at all. The modal, delete confirm and reminder cron are solid; the surrounding date and dispatch logic is not.

If you only do three things:

1. Fix the timezone date handling. Extract a single `ymd(date: Date)` local-date formatter into `src/lib/schedule/dates.ts`, replace all six `toISOString().split("T")[0]` calls across `page.tsx`, `schedule-client.tsx` and `dispatch-board.tsx`, add `setHours(0,0,0,0)` to `goToday`, and cover it with tests pinned to Australia/Sydney. Nothing else on this page can be trusted until the week window is correct — and the one paying-adjacent user is in Sydney.
2. Make assignment writes safe and correct. Check the errors in `setJobAssignments` (`schedule.ts:184-200`) so a failed insert throws instead of silently wiping the crew off a job, and fix the drag-drop union at `dispatch-board.tsx:98` so moving a job between workers actually moves it. These are two small edits that stop the feature from quietly lying about what it saved.
3. Cut the Dispatch board and rebuild worker swimlanes into the Week view using `@dnd-kit`. Its native HTML5 drag is dead in Firefox and on every tablet, it duplicates the Week view's data, and `leads-board.tsx` already carries the working pattern plus a comment explaining why native DnD was abandoned. Deleting ~190 lines and reusing the proven approach is less work than repairing it, and it fixes the mobile grid at the same time.

Scheduled

- ☐ **[HIGH]** Timezone round-trip corrupts the week window; navigation drifts 1 day per click
 - *Function · hours* — `src/app/(dashboard)/schedule/page.tsx:15-16` and `:29-30` call `monday.toISOString().split("T")[0]` on a local-midnight Date; `schedule-client.tsx:24-28` (`addDays`), `:30-32` (`weekDays`), `:120-127` (`goToday`); `dispatch-board.tsx:19-23` repeats `addDays` verbatim. Reproduced under TZ=Australia/Sydney: `addDays("2026-07-20", 0)` returns "2026-07-19"; chaining `addDays(w,7)` five times from 2026-07-20 yields 2026-08-19 instead of 2026-08-24, i.e. 5 days lost. Double shift confirmed: `page.tsx` sends a start already one day early, then `weekDays` shifts each column back again, so grid and queried range are also offset from each other.
 - **Impact.** For the one dependent Sydney roofing business the queried range is Sun-Sat instead of Mon-Sun, so the current week's Sunday jobs fall outside the fetched range and never render while the prior Sunday leaks in. Each next/prev click slides the window an extra day, so within a few clicks the board no longer aligns to a working week. Columns stay internally consistent, so nothing looks broken — the dispatcher simply cannot see some jobs.
 - **Fix.** Add `src/lib/schedule/dates.ts` with a local-date key helper `( ${d.getFullYear()}-${pad(d.getMonth()+1)}-${pad(d.getDate())} )` and replace all five `toISOString().split("T")[0]` sites across `page.tsx`, `schedule-client.tsx` and `dispatch-board.tsx`. Add `setHours(0,0,0,0)` in `goToday`. Confirmed no existing shared helper to reuse.
- ☐ **[HIGH]** Dispatch drag-to-reassign adds a second worker instead of moving the job
 - *Function · hours* — `src/components/schedule/dispatch-board.tsx:98` — `const nextAssignees = profileId === "" ? [] : Array.from(new Set([...existingIds.filter((p) => !!p), profileId]))`. `existingIds` is read from `job.work_order_assignments` at `:95` and the source row is never subtracted.

Success toast at :101 reads "Job moved".

- **Impact.** Dragging a job from Dave's row to Sam's leaves it assigned to both. `work_orders` are narrowed to a worker by assignment (`my_member_profile_ids`), so both crews see it on mobile and both can turn up to the site. The UI reports a move, so the dispatcher gets no signal. Recovery means opening the modal and unticking the original worker.
- **Fix.** Track the source row id in state alongside `dragJobId` (`dispatch-board.tsx:59`); on drop compute `existingIds.filter(p => p !== sourceRowId).concat(profileId)`. Optionally make shift-drag the explicit add-without-removing gesture and reword the toast to distinguish move from assign.
- ☐ **[HIGH]** Week grid renders 7 columns on a phone and empty days render as blank space
 - *UI · hours* — `src/components/schedule/schedule-client.tsx:214` — `grid gap-3 flex-1`
`${displayDays.length > 1 ? "grid-cols-7" : "grid-cols-1"}` with no responsive prefix. `selectedDay` initialises to null (:92) and `displayDays` falls back to all 7 days (:137), so first paint at any width is seven columns; the mobile day-tab row (:186) only collapses to one day after a tap. The empty-day Add CTA (:235) is `hidden sm:flex` and the secondary add button (:253) is `hidden sm:block`. Job titles are line-clamp-2 (:306), the address is `truncate` (:322), against the documented break-words preference.
 - **Impact.** A tradie opening `/schedule` on a 375px phone gets seven ~45px unlabelled columns, and any day without jobs is an empty void with no way to add one — he must first discover the day-tab row, which nothing prompts. This is the default view of the schedule on the device most used in the field.
 - **Fix.** Initialise `selectedDay` to today's local date key so mobile opens single-day; change the grid to `grid-cols-1 sm:grid-cols-7`; drop `hidden sm:` from both add CTAs and add a full-width mobile Add job button; replace `truncate` on the address (:322) with `break-words`.
- ☐ **[MEDIUM]** `setJobAssignments` deletes then inserts with no error checking — a failed insert silently wipes all assignments
 - *Function · hours* — `src/lib/actions/schedule.ts:184-200` — the delete at :185-187 and the insert at :191-199 are both awaited with no `error` destructured. Every other query in the same file checks: :37, :87, :124, :145, :165 all `if (error) throw error`. Return type is `Promise`, so callers `createScheduledJob` (:91), `updateScheduledJob` (:128) and `rescheduleJob` (:168) cannot detect failure either.
 - **Impact.** If the insert fails after the delete commits, every worker is stripped off the job, the action returns normally, and the UI toasts "Job moved" / "Job updated". The crew loses the job from their mobile list with nothing surfaced to anyone. Same failure class as the unchecked-error trap in `CLAUDE.md`.
 - **Fix.** Destructure and throw on both statements, mirroring `register-tools.ts:1042`. Add `.eq("business_id", businessId)` to the delete at line 186 for parity. Ideally move both into one Postgres function so a failed insert rolls the delete back.
- ☐ **[MEDIUM]** Native HTML5 drag has no `dataTransfer` payload and no touch support
 - *Function · days* — `src/components/schedule/dispatch-board.tsx:159-161` — `draggable onDragStart={() => setDragJobId(job.id)}` with no `e.dataTransfer.setData` call. Contrast `src/components/leads/leads-board.tsx:4-13`, whose header comment records abandoning native DnD ("fired inconsistently across browsers"), and :18-22 importing `DndContext/DragOverlay/PointerSensor`. `@dnd-kit/core ^6.3.1`, `@dnd-kit/sortable ^10.0.0`, `@dnd-kit/utilities ^3.2.2` are already dependencies (`package.json:18-20`).
 - **Impact.** Drag-to-dispatch is expected not to initiate in Firefox and definitively does nothing on touch devices — dragging just scrolls. No fallback affordance exists on the card (no 'move to' menu), so on those clients the board is view-only. Reduced from high because the week view remains fully functional and is what loads by default.
 - **Fix.** Port the board to `@dnd-kit` using the `leads-board.tsx` pattern: `PointerSensor` covers mouse and touch, `DragOverlay` gives the preview. As an interim, add an assign/reschedule menu on the card so a non-drag path exists.
- ☐ **[MEDIUM]** Feature composes none of the design system — hand-rolled avatars and status colours duplicated across three files

- *UI · hours* — grep for "ui/kirei" and for "ch-pill" across src/components/schedule/ both return zero hits. AVATAR_COLORS plus avatarColor() and initials() are byte-identical in three files: schedule-client.tsx:65-77, dispatch-board.tsx:39-47, job-modal.tsx:17-28. Status colour is defined twice in incompatible shapes — STATUS_STYLE (card/badge/label) at schedule-client.tsx:16-22 and STATUS_DOT (dot only) at dispatch-board.tsx:11-17. Empty days render as dashed placeholder buttons (schedule-client.tsx:233-238); no EmptyState anywhere.
- **Impact.** The schedule reads as a different product from the redesigned leads workspace, and status colour is not learnable across the app because these maps do not derive from .ch-pill. Adding or renaming a status means editing four separate maps, which is how one of them drifts.
- **Fix.** Replace the three copies of avatarColor/initials with KireiAvatar; replace STATUS_STYLE and STATUS_DOT with the shared .ch-pill class keyed on status; use EmptyState for a week with zero jobs. Mostly deletion.
- ☐ **[MEDIUM]** Zero tests on the feature, including the date helpers that are demonstrably wrong
 - *Tests · hours* — All test files are under src/lib/{assistant,booking,content,mcp}/**tests/** — 14 files, none referencing schedule or dispatch. src/lib/content/**tests**/schedule.test.ts is the Content Studio cadence planner, confirming the name collision flagged. src/lib/booking/**tests**/time.test.ts exists and is the in-repo precedent for isolated date/time helper tests.
 - **Impact.** The week-boundary drift is a one-assertion failure nothing would have caught, and any fix ships with no regression guard. The date helpers are pure functions currently trapped inside client components, which is the only reason they are untested.
 - **Fix.** Extract addDays/weekBounds/weekDays/formatDateHeader into src/lib/schedule/dates.ts and add dates.test.ts run under BOTH TZ=Australia/Sydney and TZ=America/Los_Angeles (the bug is invisible in the latter, so both are needed): assert addDays(d,0)==d, that weekBounds returns a Monday, that 7 chained next-week calls advance exactly 49 days, and that goToday's window contains today on all seven weekdays at both 08:00 and 16:00 local. Then an actions test asserting setJobAssignments throws rather than silently wiping when the insert fails.
- ☐ **[MEDIUM]** Work-order number is minted read-modify-write and burned on failure
 - *Function · hours* — src/lib/actions/schedule.ts:57-66 — reads work_order_prefix/work_order_next_number as `const { data: biz }` with no error, builds the number at :63, writes num + 1 in a separate unchecked statement at :64-66, then inserts at :68-85. The insert's error is only thrown at :87, by which point the counter is already incremented. Contrast the race-safe next_invoice_number / next_quote_number RPCs from migration 20260511020100_perf_atomic_number_mint.sql; CLAUDE.md confirms no next_work_order_number RPC exists.
 - **Impact.** Two dispatchers scheduling simultaneously can mint the same WO number, and a failed insert leaves a permanent gap in the sequence, which matters when reconciling job numbers against invoices. Low frequency at 2 active users, real at 20.
 - **Fix.** Add a next_work_order_number(uuid) RPC mirroring the existing two and call it here, minting only after the insert succeeds. At minimum check the select error rather than silently falling back to 1.
- ☐ **[LOW]** scope_of_work is collected in state but has no input
 - *UX · minutes* — src/components/schedule/job-modal.tsx:58 declares `const [scopeOfWork, setScopeOfWork] = useState(job?.scope_of_work ?? "")` and :95 passes scope_of_work into the payload, but setScopeOfWork is never called anywhere in the file. The only textarea (:264-269) is bound to description, under a Label at :263 reading "Description / Scope".
 - **Impact.** A dispatcher reads "Description / Scope" and assumes that box is the scope of work; it is not — scope is only editable on the work-order detail page. Reads as a half-finished field.
 - **Fix.** Delete the scopeOfWork state and the payload key and rename the label to "Description", since the work-order detail page owns scope. Adding a real bound textarea is the alternative if scope should be editable from dispatch.

Batch 2.7 — Customer portal (token-gated)

Health: rough · **Verdict:** keep-core · **Findings:** 0 blocker · 2 high · 6 medium · 2 low **Estimated:** ~5.3 days (10 scheduled)

What it does. A business mints a long random token (`cust_` + 24 random bytes, `customer-portal.ts:37/84`) tied to one customer, and shares the resulting `/portal/<token>` URL by copy, WhatsApp, SMS or email from the `PortalLinkButton` dialog on the customer page. That one URL opens a hub listing every invoice, quote, job, appointment and report for that customer, plus a computed outstanding balance. From the hub the customer can open an invoice (line items, totals, payment history, bank details, "Pay with card" via Stripe Checkout, save-card/autopay, PDF download), a quote (line items, "Accept quote" and optionally "Accept & pay X% deposit"), a site report, an SEO report, a contract (native e-signature), or an onboarding form. Every page and API route re-validates the token against `customer_portal_tokens` (revoked_at, expires_at) and then queries through the service-role admin client scoped by `business_id` + `customer_id` — RLS is bypassed here, so those two `.eq()` filters are the entire security model. It works and the scoping is correct everywhere I checked, including the money path (`api/stripe/checkout/route.ts:22-40`).

Why this rating. Token scoping is genuinely solid — every one of the 7 pages and 6 API routes re-validates and filters by both `business_id` and `customer_id`. But the portal shows customers documents the business has not sent (drafts), lets them pay a draft invoice, accepts a quote with zero audit trail and no way to decline, ignores quote expiry dates entirely, and has zero test coverage. For the one dependent user with \$327k of open quotes, quote acceptance is the highest-stakes action in the product and it is a single unconfirmed button press recorded as nothing but `status='accepted'`.

If you only do three things:

1. Stop showing customers unsent work. Add a status filter to the four hub queries (`portal/[token]/page.tsx:40-59`), `notFound()` drafts on the invoice and quote detail pages, and reject draft invoices in `api/stripe/checkout/route.ts`. This is a few hours of work and it is the difference between a portal you can confidently send to a customer and one you can't.
2. Make quote acceptance defensible. Capture `accepted_at` / `name` / `IP` / `user-agent` in `api/portal/[token]/quote/[id]/accept/route.ts` (the contracts sign route at `api/portal/[token]/contract/[id]/sign/route.ts:41-43` already does exactly this — copy it), enforce `expiry_date` in both the accept routes and `quote/[id]/page.tsx:46`, and add a Decline button so lost quotes leave the pipeline. \$327k of open quotes currently accept on one unconfirmed tap and record nothing.
3. Extract the token gate into `src/lib/portal/resolve-token.ts` and test it. The same 6-line check is copy-pasted across 13 files with zero tests; since these routes use the service-role client, those two `.eq()` filters are the only thing standing between customer A and customer B's invoices. One helper plus one test file removes an entire class of future breach.

Scheduled

- ☐ **[HIGH]** Draft invoices, quotes and reports are visible — and payable — in the portal
 - *Function · hours* — `src/app/portal/[token]/page.tsx:40-59` — all four queries filter only `.eq("business_id")` and `.eq("customer_id")` plus `.order()`; no `.neq("status","draft")` anywhere. Only the appointments query (line 66) filters status at all (`.neq("status","cancelled")`). `invoice/[id]/page.tsx:44` is `if (!invoice) notFound();` — existence only. `quote/[id]/page.tsx:42` likewise. `src/app/api/stripe/checkout/route.ts:35-41` selects status but never branches on it; the only gates are token validity (27-32), business Stripe enablement (48), per-customer card allow-list (59), and balance `>= 0.5` (64) — a draft invoice with a balance reaches `stripe.checkout.sessions.create` at line 95. `page.tsx:335` does carry a `draft:` entry in the `StatusBadge` colour map.

- **Impact.** A business drafting or editing an invoice or quote has that intermediate state visible on the customer's live hub, and the customer can click 'Pay with card' on a draft invoice (invoice/[id]/page.tsx:247-256 gates the pay button on balance and offered.card only, not status).
- **Fix.** Add .neq("status", "draft") to the invoices and quotes queries at page.tsx:40-49, require reports.status = 'complete' at 55-59, notFound() on draft in the two detail pages, and reject invoice.status === 'draft' in api/stripe/checkout/route.ts after line 41.
- ☐ **[HIGH]** Quote acceptance has no audit trail, no decline path, and ignores expiry_date
 - *Missing · days* — accept/route.ts:43-45 writes exactly `{ status: "accepted", updated_at: ... }` — no signer name, IP, UA or accepted_at. Contrast contract/[id]/sign/route.ts:40-42 which captures `ip` from x-forwarded-for and `userAgent` (sliced to 300 chars) plus a typed/drawn method and an explicit consent check at line 16. quote/[id]/page.tsx:46: `const isAcceptable = quote.status !== "accepted" && quote.status !== "rejected" && quote.status !== "expired"` — expiry_date is rendered at line 83 but never compared to today. The route's guards (accept/route.ts:36-41) are status-only for both accept and accept-with-deposit (accept-with-deposit/route.ts:59-61). Directory listing of src/app/api/portal/[token]/quote/[id]/ contains only `accept` and `accept-with-deposit` — no decline/reject route exists.
 - **Impact.** A quote displaying a past 'Expires' date is still acceptable at that price indefinitely. The only record of who accepted and when is an updated_at timestamp — thin evidence for a business carrying \$327k of open quotes. Customers who decline have no button, so lost quotes never leave 'sent'.
 - **Fix.** Capture accepted_at / accepted_by_name / ip / user_agent in accept/route.ts (mirror sign/route.ts:40-42). Treat expiry_date < today as not acceptable in both the page (line 46) and both accept routes, showing an 'expired — contact us' state. Add a decline route + button writing status='rejected'.
- ☐ **[MEDIUM]** Deposit acceptance mints invoice numbers with a non-atomic read-increment
 - *Function · hours* — accept-with-deposit/route.ts:24-33 — mintInvoiceNumber selects invoice_next_number then issues a separate update with next+1; the comment at line 25 states the atomic RPC is unusable because it checks auth.uid(). Called at line 99 (parent) and line 145 (deposit child). The reuse guard at line 88 reads `quote.invoice_id` from the row fetched at line 52-57, before any update, so two concurrent requests both see null and both take the create branch at 98-127. `grep -rn invoices supabase/migrations/*.sql | grep -i 'unique|index' | grep -i number` returns nothing.
 - **Impact.** Concurrent accepts can mint duplicate invoice numbers and create two parent+deposit pairs for the same quote, billing the customer twice for one deposit.
 - **Fix.** Add a SECURITY DEFINER service-role variant of next_invoice_number(business_id) and call it from mintInvoiceNumber; add a unique index on invoices(business_id, number); re-read quote.invoice_id inside a conditional update rather than trusting the pre-fetched row.
- ☐ **[MEDIUM]** Zero tests on the entire portal — including the token gate and the money path
 - *Tests · days* — `find src -name '*.test.ts*'` returns exactly the 14 files the finding lists (assistant/models,scopes,undo; booking/availability,booking-db,time; content/live-agents,pipeline,prompts,schedule; mcp/collect,invoke,live-api; **tests/pg-filter**). None under src/app/portal, src/app/api/portal, or covering src/lib/actions/customer-portal.ts. `grep -rln customer_portal_tokens src/app` returns 22 paths, each re-implementing the same select/revoked_at/expires_at block (e.g. page.tsx:18-24, invoice/[id]/page.tsx:21-27, accept/route.ts:14-24, checkout/route.ts:23-32).
 - **Impact.** The only thing separating one customer's financial history from another's is a hand-copied .eq("customer_id") in 22 places, with RLS bypassed (createAdminClient) and no test to catch a dropped filter.
 - **Fix.** Extract src/lib/portal/resolve-token.ts returning {business_id, customer_id} | null, migrate the call sites, and test revoked/expired/unknown/valid. Add a route test asserting customer A's token cannot accept customer B's quote and that an already-accepted quote returns {already:true}.
- ☐ **[MEDIUM]** An expired or revoked link shows a bare 404 with no explanation or way back

- *UX · hours* — `notFound()` on both revoked and expired at `page.tsx:23-24`, `invoice/[id]/page.tsx:26-27`, `quote/[id]/page.tsx:28-29`, `report/[id]/page.tsx:23-24`, `seo-report/[id]/page.tsx:21-22` — identical to the unknown-token case. The API routes do differentiate: `accept/route.ts:22-24` returns 410 'Link expired' vs 404 'Invalid link', as does `accept-with-deposit/route.ts:47-50` — but the customer only ever sees the page. `customer-portal.ts:90` mints `expires_at: now + 90 * 86_400_000` and `portal-link-button.tsx:28` calls `getShareLinks`, so shared links do die at 90 days.
- **Impact.** On day 91 the customer clicking the link in their emailed invoice gets Next's default 404 — no business name, no contact details, no explanation.
- **Fix.** Render a branded expired/revoked page (business name, logo, phone, email are already fetched for the hub) and reserve `notFound()` for tokens that do not exist.
- ☐ **[MEDIUM]** The portal ignores the Connected Hub design system entirely
- *UI · days* — `page.tsx:303-307` hard-codes emerald `#34d399/#047857`, violet `#a78bfa/#6d28d9`, blue `#60a5fa/#1d4ed8` in `SummaryCard` against a teal-accented product; `page.tsx:91` hand-codes a `linear-gradient(135deg, #3a847e 0%, #1f4f4a 100%)` logo fallback. `truncate` (not break-words, contrary to the owner's stated preference and unlike `page.tsx:258` which does use break-words for appointment notes) at `page.tsx:98` business name, `:223` job title, `:278` report title.
- **Impact.** The customer-facing surface reads as a different product from the app, and the same record's status renders in different colours depending on which portal page you are on. Long job titles clip mid-word on mobile, where these links are mostly opened.
- **Fix.** One on `.ch-pill` replacing the four local copies, `StatTile` from `src/components/ui/kirei/` in place of `SummaryCard`, and break-words at `page.tsx:98/223/278`.
- ☐ **[MEDIUM]** `createPortalLink` defaults to a token that never expires, and builds its URL from raw env
- *Security · minutes* — `src/lib/actions/customer-portal.ts:38-40` — `const expires_at = options.expires_in_days ? new Date(...) : null;` so an omitted (or 0) value stores NULL, and the token gates check `if (link.expires_at && ...)` so NULL means forever. Line 52 reads `process.env.NEXT_PUBLIC_APP_URL || ""` even though `appUrl()` is imported at line 7 and used correctly by `getShareLinks` at line 95. Confirmed caller: `src/app/api/agent/route.ts:1646` `createPortalLink(input.customer_id, { expires_in_days: input.expires_in_days ?? null })` — the AI omitting the arg mints a permanent link. The URL bug is masked at `portal-link-button.tsx:58` `(r.url.startsWith("http") ? r.url : window.location.origin + r.url)`, which is the only other caller.
- **Impact.** An agent-minted link grants permanent read access to a customer's full financial history, and returns a bare path to the user if `NEXT_PUBLIC_APP_URL` is unset in that environment.
- **Fix.** Default to 90 days when `expires_in_days` is null/undefined; replace the line 52 env read with `appUrl()`.
- ☐ **[MEDIUM]** No visibility or management of issued links; Rotate is destructive with no confirmation
- *UX · hours* — `listPortalTokens` is defined at `customer-portal.ts:13` and `grep -rn listPortalTokens src/` returns only that definition — no caller. `last_used_at` is written at `page.tsx:27-29` and `grep -rn last_used_at src/` shows the only read is `api-keys-settings.tsx:176`, which renders the `api_keys` table's column, not this one. `portal-link-button.tsx:50-66` — `rotate()` revokes then mints in one click; the trigger at line 146 is a ghost-variant `Button` with no confirm dialog and no undo.
- **Impact.** The business cannot see how many live links exist, whether the customer ever opened one, or revoke a specific one. One click on a low-contrast 'Rotate' silently breaks the link already sitting in the customer's inbox. The 'customer viewed your quote' signal that Jobber and ServiceM8 surface is captured but never shown.
- **Fix.** Render `listPortalTokens` in the dialog (created / last viewed / expires / revoke each), put Rotate behind a confirm naming the consequence, and surface `last_used_at` as 'Last viewed ...' on the customer page.
- ☐ **[LOW]** Partial-payment line is hidden by a string comparison on a numeric column

- *Function · minutes* — `src/app/portal/[token]/page.tsx:162` — `{inv.amount_paid > 0 && inv.amount_paid < inv.total && (`. Both columns are numeric(12,2) per `supabase/migrations/001_initial_schema.sql:92-93`. The codebase's own defensive pattern confirms these arrive as strings: `src/lib/emails/invoice.ts:78-85` defines `const num = (v) => typeof v === "number" ? v : parseFloat(String(v ?? 0))` and applies it to `total/amount_paid/subtotal`, and CLAUDE.md records the NaN incident as a production trap. `"500.00" > 0` coerces and is true; `"500.00" < "1200.00"` is a lexical string compare and is false.
- **Impact.** The 'paid' line under a partially-paid invoice never renders on the portal hub. Cosmetic inconsistency with the detail page, most visible in the deposit flow.
- **Fix.** `const paid = Number(inv.amount_paid ?? 0); const total = Number(inv.total);` then compare — the pattern already used at `invoice/[id]/page.tsx:48`.
- ☐ **[LOW]** Hub blocks on a write and fetches every record the customer has ever had
- *Perf · minutes* — `src/app/portal/[token]/page.tsx:27-29` — `await tbl(sb, "customer_portal_tokens").update({last_used_at: ...})` runs before the `Promise.all` at line 31 and before any data fetch. Lines 39-67: invoices, quotes, work_orders, reports and appointments each carry `.eq().order` but no `.limit()`, unlike the app-side list getters which default to `.limit(200)` per CLAUDE.md.
- **Impact.** Every hub load serialises a round-trip for a write nothing reads, and a customer with 98 jobs downloads all of them on a phone. Grows linearly with no pagination path.
- **Fix.** Fire-and-forget the touch (`void tbl(...).update(...).then(() => {})`, the `link_my_member_profile` pattern) and add `.limit(50)` per list.

Batch 2.8 — Work orders / jobs

Health: rough · **Verdict:** keep-core · **Findings:** 0 blocker · 1 high · 6 medium · 3 low **Estimated:** ~6.0 days (10 scheduled)

What it does. Work orders are the field-services core: a job record (number, title, customer, site, property address, scheduled date/time window, status draft→assigned→in_progress→submitted→reviewed→completed) created at `/work-orders/new`, optionally prefilled from a work-order template, with multi-worker assignment via `work_order_assignments`. The detail page (`/work-orders/[id]`) is a single 1,839-line "job portfolio" with sections for overview/scope, an event timeline, before/during/after photos, start-stop time clocks, materials, documents, drawn signatures, and linked quotes/invoices — including a one-click "Invoice unbilled" that turns logged hours plus billable materials into a draft invoice. Workers submit a job with notes, which emails owners/admins. Owners can mint a share token exposing a public customer-facing job page at `/jobs/[token]` plus a downloadable PDF, both scoped to `customer_visible` photos, documents and timeline events. Everything is mirrored as MCP tools so the AI assistant can drive it.

Why this rating. The in-app job lifecycle works and is genuinely deep (time, materials, signatures, costing, share links). But both customer-facing outputs — the public share page and the work-order PDF — order `job_photos` by a column that does not exist, so they render zero photos. For a product whose pitch is photo-driven field submissions, the two surfaces a customer actually sees are silently broken, and the list page's photo counter is dead for the same class of reason.

If you only do three things:

1. Fix the `captured_at` column name in `src/app/jobs/[token]/page.tsx:44` and `src/app/api/pdf/work-order/[id]/route.ts:52` so customer share links and job PDFs actually show photos — this is a two-line change that restores the product's headline feature on both customer-facing surfaces, and it should ship today.
2. Fix the money path in `invoiceUnbilledForWorkOrder` (`src/lib/actions/work-orders.ts:476-591`): use the `next_invoice_number` RPC instead of the read-modify-write, apply the business tax rate instead of hardcoded zero, and require an hourly rate instead of defaulting to \$0. Then add the guard against concurrent open

timers in job-time.ts:39 so hours can't be double-billed.

3. Give the jobs list the treatment leads just got: a search box (via ilikeAcross, checking error not just data), a scheduled-date filter, pagination past 200, and a real photo count sourced from job_photos rather than the dead work_orders.photos column.

Scheduled

- ☐ **[HIGH]** Public job share page and work-order PDF show zero photos — they order by a non-existent column
 - *Function · minutes* — Verified directly. src/app/jobs/[token]/page.tsx:44 — `.eq("customer_visible", true).order("captured_at", { ascending: true })` on job_photos. src/app/api/pdf/work-order/[id]/route.ts:52 — `.from("job_photos").select("*").eq("work_order_id", id).order("captured_at", { ascending: true })`. The table definition at supabase/migrations/20260418000001_account_site_portfolio.sql:279-294 has `taken_at TIMESTAMPTZ NOT NULL DEFAULT NOW()` (line 290) and no captured_at; the index at line 295 is on (work_order_id, taken_at). A repo-wide grep for captured_at returns only these two job_photos call sites plus seo_keyword_snapshots usages (20260708120000_seo_foundation.sql:57). src/types/database.ts:1080-1094 JobPhoto likewise has taken_at, no captured_at. Both call sites swallow the error: page.tsx:55 `(photosRes.data ?? [])`, route.ts:66 `(photosRes.data ?? [])` — neither destructures `error`. src/lib/actions/job-photos.ts:26 correctly orders by taken_at, which is why the in-app grid works.
 - **Impact.** On both customer-facing surfaces the photo arrays come back empty, so the before/during/after sections render nothing and the PDF ships with no imagery. Silent — no error surfaces to the owner. Correcting severity blocker→high: the pages still load and every other section (timeline, signatures, documents) renders, there is no data loss, and I found no evidence in the repo that share links are actively in use. It is still the single highest-value fix in this feature and it is a one-word change.
 - **Fix.** Change `captured_at` to `taken_at` at page.tsx:44 and route.ts:52. Then destructure `{ data, error }` at both sites and log/throw — the reason this survived is that the error was coalesced away.
- ☐ **[MEDIUM]** Photos column on the work-orders list is permanently empty — the column isn't selected
 - *Function · hours* — src/lib/actions/work-orders.ts:27-38 — the slim select lists id, business_id, user_id, number, title, customer_id, property_address, status, scheduled_date, start_time, end_time, site_id, share_token, share_enabled_at, recurring_job_id, assigned_to, assigned_to_email, assigned_to_profile_id, created_at, updated_at, customers(...). No `photos`. The in-code comment at lines 25-26 confirms it was removed deliberately for perf. src/app/(dashboard)/work-orders/page.tsx feeds this straight into WorkOrdersClient. src/components/work-orders/work-orders-client.tsx:131 renders a `Photos` header and :166-169 renders `(wo.photos?.length ?? 0) > 0 ? ... : "-"`, so the cell is always "-".
 - **Impact.** A dead column on every row of the jobs list. Correcting high→medium: it is a missing signal in a nice-to-have column, not incorrect data or a broken workflow — nothing is mis-stated to the user, the count is simply absent. The finding's secondary point is right and worth keeping: work_orders.photos is the legacy JSONB store (see supabase/migrations/20260601000001_backfill_job_photos_from_work_orders.sql and 20260623120000_sync_wo_photos_to_job_photos.sql, which make job_photos canonical), so re-adding it to the select would be both slow and wrong.
 - **Fix.** Fetch a job_photos count map alongside getWorkOrders and pass it to the client, or drop the column. Do not re-add work_orders.photos to the slim select.
- ☐ **[MEDIUM]** "Invoice unbilled" mints invoice numbers with a read-modify-write and charges zero tax
 - *Function · hours* — Both halves verified. Number mint: src/lib/actions/work-orders.ts:549-554 selects `invoice_prefix, invoice_next_number` then issues a separate `.update({ invoice_next_number: (business?.invoice_next_number ?? 1) + 1 })` — two statements, no RPC. src/lib/actions/invoices.ts:71 and :237 both call `.rpc("next_invoice_number", { p_business_id:`

`businessId }}`, so the race-safe path exists and is simply not used here. Tax: the labour line at `work-orders.ts:521-529` and each material line at `:532-540` are built with `tax_rate: 0, ... tax_amount: 0, total: subtotal`, and the insert at `:561-562` sets `tax_total: 0, total: subtotal`. Rate default: `src/components/work-orders/job-portfolio-client.tsx:1732 useState("")` and `:1739 const rate = parseFloat(hourlyRate) || 0`, with the input at `:1776` defaulting to placeholder "0.00" — so submitting blank produces a \$0 labour line.

- **Impact.** Correcting high→medium for two reasons the finding omits. (1) The invoice is inserted with `status: "draft"` (`work-orders.ts:557`), so it lands in the owner's editor for review before anything is sent — the zero-tax and zero-rate outputs are visible and correctable, not silently mailed. (2) The duplicate-number race needs two concurrent invoicers; the business has 2 active users in 30 days, so it is a latent correctness bug rather than a live one. Still real: the defaults are wrong for an AU business that owes GST, and an owner who trusts the generated draft under-bills 10%.
- **Fix.** Swap the mint for `supabase.rpc("next_invoice_number", { p_business_id: businessId })` as `invoices.ts:71` does. Read the business default tax rate and compute `tax_amount/tax_total` per line. Prefill the hourly-rate input from the business or quoting-agent baseline instead of defaulting to 0, and reject an empty rate when there is billable time.
- ☐ **[MEDIUM]** Deleting a work order orphans its photos in storage and the confirmation dialog claims otherwise
 - *Function · hours* — `src/lib/actions/work-orders.ts:428-432`: `supabase.storage.from("work-order-photos").list( ${user.id}/${id} )` then `.remove(files.map(f => ${user.id}/${id}/${f.name} ))` — keyed on the deleting user's uid. Upload path is `src/components/work-orders/job-portfolio-client.tsx:951-955`: `const path = ${currentUserId}/${workOrderId}/${crypto.randomUUID()}.${ext}` followed by getPublicUrl(path). src/lib/actions/job-photos.ts:78-84 deleteJobPhoto removes only the row. The bucket is public — supabase/migrations/010_work_orders.sql:53 creates it with true and :63 grants an unconditional public SELECT policy. Dialog copy verified at work-orders-client.tsx:204: "This will permanently delete the order and all associated photos."`
 - **Impact.** Correcting high→medium and correcting one overstatement in the finding: it is not true that "an owner deleting a job removes zero image files in practice" — when the deleter is also the uploader the prefix matches and cleanup works, which is the common single-operator case. It fails specifically for photos uploaded by a different user (a worker's photos deleted by the owner). Those files stay in a public bucket forever. The `job_photos` rows themselves do go away — the table is defined ON DELETE CASCADE (migration `20260418000001:281-282`) — so the dialog's claim is half-true; the records die, the image files survive at unguessable-but-public UUID URLs. A retention/privacy gap, not an exposure of the kind that leaks broadly.
 - **Fix.** Read `job_photos.url` for the work order before deleting, derive the storage keys from those URLs, and remove exactly those. Mirror it in `deleteJobPhoto`. Then the dialog copy becomes true.
- ☐ **[MEDIUM]** No search, no date filter, and a silent 200-row ceiling on the jobs list
 - *Missing · days* — Read `src/components/work-orders/work-orders-client.tsx` end to end. TABS at `:21-28` are the seven statuses; filtering at `:54-57` is `tab === "all" ? workOrders : workOrders.filter(w => w.status === tab)`. There is no input element in the file — the only interactive controls are the tabs, the row links, the per-row dropdown (`:180-196`) and the delete `AlertDialog`. `src/lib/actions/work-orders.ts:38` `.limit(filters?.limit ?? 200)` with no offset/cursor, and the client receives no total count so it cannot warn.
 - **Impact.** Confirmed at medium. The roofing business is at 98 jobs, so the 200 cap has not bitten yet but is roughly one busy year away, and when it does the older rows simply disappear with no notice. The absence of search is the live pain today. The comparison to the just-shipped leads workspace is fair — `src/components/leads/` has Board/List/Calendar; jobs has a status tab strip.

- **Fix.** Server-side search across title, number, customer name and address using `ilikeAcross()` from `src/lib/pg-filter.ts` (never an interpolated `.or()`), plus a scheduled-date range filter and either pagination or an explicit "showing first 200" notice when the cap is reached.
- ☐ **[MEDIUM]** Photo upload has no size cap, no compression, no per-file progress, and fails opaquely mid-batch
 - *UX · days* — `src/components/work-orders/job-portfolio-client.tsx:944-966`. `for (const file of Array.from(files))` uploads each raw File sequentially; `if (up.error) throw up.error` aborts the whole loop on the first failure; the catch at :963 emits a single `toast.error(e instanceof Error ? e.message : "Upload failed")`. No file.size check and no resize anywhere in the function. Feedback is the single `uploading` boolean driving one spinner on the button at :987-990. Note the sibling download path at :938-941 does track per-file progress (`setZip({ done, total })`) — so the pattern the owner wants exists in the same component, ten lines away.
 - **Impact.** Confirmed at medium. A worker uploading a batch on 4G sees one undifferentiated spinner and, on a mid-batch failure, a message that gives no way to tell which files landed — the natural recovery is to re-select everything and duplicate the successes. Directly contradicts the stated preference for visual feedback on everything, and the adjacent zip-download code proves the standard.
 - **Fix.** Downscale client-side before upload, reject oversized files, upload with bounded concurrency, and show "n of N" using the same progress shape as `runZip` at :938. On partial failure, name the failed files and keep the successes.
- ☐ **[MEDIUM]** The entire jobs feature has zero test coverage, including the money path
 - *Tests · days* — Enumerated every test file outside `node_modules` and mobile: `src/lib/assistant/tests/{models,scopes,undo}.test.ts`, `src/lib/booking/tests/{availability,booking-db,time}.test.ts`, `src/lib/content/tests/{live-agents,pipeline,prompts,schedule}.test.ts`, `src/lib/mcp/tests/{collect,invoke,live-api}.test.ts`, `src/lib/tests/pg-filter.test.ts`. Nothing touches `work-orders.ts`, `job-time.ts`, `job-materials.ts` or `job-photos.ts`. `invoiceUnbilledForWorkOrder` spans `work-orders.ts:476-591` and does produce a real invoice row.
 - **Impact.** Confirmed at medium. The concrete argument carries it: the `captured_at` bug in finding 1 is exactly the class a schema-contract test catches, and it shipped and stayed shipped. The line-item arithmetic in `invoiceUnbilledForWorkOrder` is pure and trivially testable once extracted.
 - **Fix.** Extract the line-item builder out of the "use server" file the way `src/lib/recurring/totals.ts` was, then test hours rounding, travel inclusion, materials subtotal and empty-input rejection. Separately, add a test asserting every column name used in `.order().eq()` against `job_photos`, `job_timeline_events`, `job_documents` and `job_signatures` exists in the migration — that one test catches the entire `captured_at` class.
- ☐ **[LOW]** Work-order numbers use the same racy read-modify-write mint
 - *Function · hours* — `src/lib/actions/work-orders.ts:86-95`: selects `work_order_prefix`, `work_order_next_number`, builds the number string, then a separate `.update({ work_order_next_number: (biz?.work_order_next_number ?? 1) + 1 })`. (The finding cited 90-98; the block is at 86-95 — close enough that the claim stands.) I grepped the whole repo including `supabase/migrations` for `next_work_order_number` and got zero hits, confirming the RPC does not exist.
 - **Impact.** Correcting medium→low. The mechanism is real but the exposure is not: 2 active users in the last 30 days makes a genuine collision essentially theoretical, and unlike an invoice number a WO number carries no tax-document sequencing obligation. Worth fixing when the invoice-mint fix is made, not on its own.
 - **Fix.** Add a `next_work_order_number(uuid)` RPC mirroring `supabase/migrations/20260511020100_perf_atomic_number_mint.sql` and call it from `createWorkOrder`.
- ☐ **[LOW]** Nothing prevents multiple concurrent running timers on the same job

- *Function · hours* — `src/lib/actions/job-time.ts:37-46` — `startTimeEntry` inserts `started_at` with no `ended_at` and performs no lookup for an existing open entry. `deleteTimeEntry` at `:122-129` selects only `work_order_id` and deletes with no `invoice_id` check. Both code claims are accurate. However the stated failure scenario does not hold: (a) I grepped `mobile/` for `job_time_entries` and got zero hits — the mobile app has no timer at all, so "a worker taps Start twice on a phone" cannot happen; (b) the web UI at `src/components/work-orders/job-portfolio-client.tsx:1233` computes `const open = entries.find(e => !e.ended_at)` and at `:1255-1264` renders the Stop button instead of the three Start buttons whenever `open` is set, with `disabled={busy}` on all of them.
- **Impact.** Correcting medium→low and stripping the false premise. Double-billing via parallel timers requires two simultaneous browser sessions on the same job — not the routine field scenario described. The residual real gap is the smaller one the finding mentions in passing: `deleteTimeEntry` will happily delete an entry that has already been stamped with `invoice_id` by `invoiceUnbilledForWorkOrder` (`work-orders.ts:571-575`), silently detaching billed time from its invoice.
- **Fix.** Add the open-entry guard in `startTimeEntry` as defence in depth, and block `deleteTimeEntry` when `invoice_id` is set — that second one is the part with a real path to it.
- ☐ **[LOW]** Destructive actions on the job detail page use native `window.confirm`
- *UX · hours* — grep for `confirm(` in `src/components/work-orders/job-portfolio-client.tsx` returns seven hits, not four: `:217` (delete work order), `:292` (revoke share link), `:1069` (delete photo), `:1289` (delete time entry), `:1377` ("Remove?"), `:1470` ("Delete?"), `:1547` (delete signature). The contrast is real — `src/components/work-orders/work-orders-client.tsx:199-213` uses a styled `AlertDialog` with a busy state.
- **Impact.** Correcting medium→low. This is an inconsistency against the Connected Hub design system, not a defect: the dialogs work, they are blocking, and the destructive action is correctly gated. It matters most at `:217` where a whole job is deleted. Worth noting the finding undercounted — there are seven bare confirms, so a fix should sweep all of them.
- **Fix.** Replace all seven with the `AlertDialog` pattern from `work-orders-client.tsx:199-213`, including the `busy` pending state.

Tier C — growth & trust

Batch 2.9 — AI assistant (/assistant + /api/assistant + shared MCP tool registry)

Health: rough · **Verdict:** keep-core · **Findings:** 1 blocker · 2 high · 5 medium · 3 low **Estimated:** ~4.0 days (11 scheduled)

What it does. A full-page chat at `/assistant` that can drive the entire app. The route (`src/app/api/assistant/route.ts`) runs a 15-iteration tool loop against Claude, streaming NDJSON events (text, thinking, tool_start/done/error, done) to the client. Its tools are not hand-written: `src/lib/mcp/collect.ts` runs the MCP `registerTools` registry through a collector to get ~199 tool specs as plain data, and `src/lib/mcp/invoke.ts` validates args against each tool's zod shape and dispatches in-process. Those handlers run as SERVICE ROLE and bypass RLS, so `src/lib/assistant/scopes.ts` — role to API scopes, workers denied outright — is the only access gate on this path. The conversation wire format is a real Anthropic block array (tool_use/tool_result preserved), which is what gives it cross-turn memory within a session. It also has: model + effort pickers (Opus 4.8 / Sonnet 5 / Haiku 4.5, `src/lib/assistant/models.ts`), prompt caching with the volatile snapshot deliberately placed after the cache breakpoint, image attach + phone camera capture (`src/components/assistant/images.ts` downscales to 1568px client-side), voice input that shows a transcript for confirmation rather than auto-sending, browser TTS, per-message Undo backed by row snapshots (`src/lib/assistant/undo.ts`, 19 allow-listed tools), and a webhook bridge (`src/lib/assistant/tool-webhooks.ts`) because MCP tools don't fire `dispatchWebhook`. It is dead in production today — `ANTHROPIC_API_KEY 401s`.

Why this rating. The core loop is the best-engineered code in this repo — the comments name the exact bug each design choice prevents, and the scope gate is genuinely thought through. But three things undercut it: the OLD assistant it was built to replace is still shipped and mounted on every dashboard page, the persistence round-trip throws away the very tool blocks the feature exists to preserve (and can write a message shape that permanently 400s a resumed conversation), and the role resolution feeding the security gate fails open. Plus it is 401-dead in prod, so none of this has been exercised by a real user.

If you only do three things:

1. Delete the old assistant. Remove `src/app/api/agent/route.ts` (2151 lines) and `src/components/agent/agent-panel.tsx`, and drop from `dashboard-shell.tsx:82`. Right now the assistant every user actually sees is the broken one; this is one deletion that both fixes the product and removes the largest single file in the AI surface.
2. Fix the persistence round-trip. Persist every message of a turn with its true role instead of collapsing to two rows (`src/lib/actions/assistant.ts:157`, `src/components/assistant/assistant-chat.tsx:520`). This restores cross-turn memory across reloads — the feature's whole reason for existing — and eliminates the message shape that permanently 400s a resumed conversation.
3. Fail closed on role. `src/lib/role.ts:36` returns 'viewer' when the member lookup errors or comes back empty, and on this one path that grants full-business read through a client that bypasses RLS. Check the error, return the most-restricted role, and verify the cookie's businessId server-side before building `invokeCtx`.

Scheduled

- ☐ **[BLOCKER]** Role resolution fails open to viewer, and the assistant's tools bypass RLS
 - *Security · hours* — `src/lib/role.ts` `getMyRoleCached` destructures only `{ data: biz }` and `{ data: member }` — neither error is read — and ends `return (member?.role ?? "viewer") as Role;`. `src/app/api/assistant/route.ts` calls `getActiveBizId` then `getMyRoleCached` and builds `const invokeCtx = { businessId, userId: user.id, scopes }`. `src/lib/assistant/scopes.ts` gives a viewer `[...READ_SCOPES, "agent:access"]`. `src/lib/mcp/invoke.ts` forwards them as `extra.authInfo.extra` to handlers running on the admin client. `src/lib/active-business.ts` returns the cookie with the comment 'Trust the cookie — RLS enforces actual access at the query level', which is false on this route.
 - **Impact.** Any authenticated Kirei user can set `active_business_id` to another business's UUID, resolve to 'viewer', receive every read scope, and have service-role tools query that `business_id` with RLS bypassed — cross-tenant read of customers, invoices, quotes and leads.
 - **Fix.** In `/api/assistant/route.ts` verify the resolved `businessId` belongs to the caller (`businesses.user_id = user.id` OR an active `business_members` row) before building `invokeCtx`; do not trust the cookie on a service-role path. Separately in `src/lib/role.ts` destructure and check both errors and return 'worker' when the member lookup errors or is empty and the user is not the owner.
- ☐ **[HIGH]** The old, broken assistant is still shipped and mounted on every dashboard page
 - *Function · hours* — `src/components/layout/dashboard-shell.tsx:13-16` lazily imports `AgentPanel` and line 82 renders in the shell, so it is on every dashboard route. `src/components/agent/agent-panel.tsx` declares `interface DisplayMessage { ... content: string }` and `interface ApiMessage { role; content: string }`; the send path builds `[...apiHistory, { role: "user", content: trimmed }]` and appends `{ role: "assistant", content: finalText }` after streaming — text only, no tool blocks. It fetches `/api/agent`, and `src/app/api/agent/route.ts` exists at 2151 lines. The project task list marks '#73 PR4: retire /api/agent' completed, which it is not on this branch.
 - **Impact.** Two assistants ship. The one every user meets (floating button, all pages) keeps only flattened text across turns, so an ID a tool found in turn 1 is gone by turn 3; the rebuilt block-preserving one is a page you must navigate to. Plus ~2100 lines of duplicated tool loop to maintain.

- **Fix.** Delete `src/app/api/agent/route.ts` and `src/components/agent/agent-panel.tsx`; drop the dynamic import (`dashboard-shell.tsx:13-16`) and the render (line 82). If a floating entry point is wanted, make it a link to `/assistant`. Keep `src/components/agent/use-voice-capture.ts` — `assistant-chat.tsx` imports it.
- ☐ **[HIGH]** Persistence drops every tool block, so cross-turn memory dies on reload
- *Function · hours* — `src/lib/actions/assistant.ts` `saveAssistantTurn` inserts exactly two rows: one `role: "user"` at seq `n` and one `role: "assistant"` at seq `n+1`. `src/components/assistant/assistant-chat.tsx` computes `const assistantBlocks = returnedHistory[returnedHistory.length - 1].content` and passes only that as `assistantContent`. `openConversation` rebuilds with `setApiHistory(rows.map((r) => ({ role: r.role, content: r.content })))`.
- **Impact.** Intermediate `assistant tool_use` messages and their `tool_result` replies are never written to `assistant_messages`. Works in-session, silently regresses on reload — the reopened conversation has prose but no record of what was looked up, so the model re-searches. The exact defect the rebuild existed to fix, reintroduced one layer down.
- **Fix.** Have `saveAssistantTurn` accept the full slice of new `Anthropic.MessageParam[]` the turn produced (the route already returns the whole `messages` array; diff it against what was sent) and insert one row per message with incrementing seq instead of collapsing to two.
- ☐ **[MEDIUM]** A turn that hits the iteration cap writes a message that permanently 400s the conversation on reload
- *Function · minutes* — `src/app/api/assistant/route.ts`: `for (let iteration = 1; iteration <= MAX_ITERATIONS; iteration++)` with `MAX_ITERATIONS = 15`; it does `messages.push({ role: "user", content: results })` (`tool_result` blocks) and then only sends an error event when `iteration === MAX_ITERATIONS` before falling out of the loop — so the final element has role "user". `assistant-chat.tsx` takes `returnedHistory[returnedHistory.length - 1].content` with no role check and saves it as the assistant row. The route guards the mirror-image case server-side ('a `tool_use` with no `tool_result` is a 400 that would wedge the conversation for good').
- **Impact.** An assistant row whose content is `tool_result` blocks with no preceding `tool_use` is rejected by the Messages API, so reopening that conversation and sending anything 400s forever, with no recovery but deleting the chat.
- **Fix.** In `assistant-chat.tsx` pick the last element whose role `=== "assistant"`, falling back to `[{ type: "text", text: finalText }]`. Fixing finding #2 removes this as a side effect.
- ☐ **[MEDIUM]** "Restore lead" undo always fails — `leads.identity_key` is a generated column
- *Function · minutes* — `src/lib/assistant/undo.ts` registers `delete_lead: { table: "leads", idArg: "lead_id", op: "delete" }`, and `snapshotRow` does `.select("*")` on the admin client, so the snapshot carries every column. `src/lib/actions/assistant.ts`'s delete branch re-inserts wholesale: `tbl(supabase, entry.table).insert(before)`, throwing `Couldn't restore the ${entry.table} row on error`. `supabase/migrations/20260503000002_lead_dedup.sql:57-58` declares `ADD COLUMN identity_key TEXT GENERATED ALWAYS AS (public.lead_identity_key(email, phone, name, address)) STORED`, which Postgres refuses an explicit value for (428C9).
- **Impact.** The Undo button renders, the user clicks it to recover a deleted lead — the case where undo matters most — and it errors every time.
- **Fix.** Strip generated columns before re-insert in the delete branch: minimally `delete before.identity_key` for leads, better a per-table skip-list declared alongside `UndoSpec`. Audit the other undo tables for the same pattern.
- ☐ **[MEDIUM]** Deleting a conversation is instant, irreversible and unconfirmed
- *UX · hours* — `src/components/assistant/assistant-chat.tsx` renders a Trash2 button inside the conversation row with `opacity-0 group-hover:opacity-100` and `className="w-3 h-3"`, `onClick` calling `void removeConversation(c.id)` after `stopPropagation`. `removeConversation` awaits

deleteAssistantConversation with no dialog. `src/lib/actions/assistant.ts` deleteAssistantConversation is a hard `.delete()` on `assistant_conversations`, cascading to `assistant_messages`.

- **Impact.** A 12px hover-revealed icon sits inside the row you click to open the chat; one mis-click destroys the conversation and its `change_log` — the only record of what the assistant did to real rows and the only thing Undo can read.
- **Fix.** Wrap in the AlertDialog used elsewhere, or soft-delete with a `deleted_at` column plus an undo toast. Given `change_log` is a write audit trail, soft-delete is the better answer.
- ☐ **[MEDIUM]** On mobile there is no conversation list, no New chat, and no way to switch chats
 - *UI · hours* — `src/components/assistant/assistant-chat.tsx`: outer container is `<div className="flex gap-4 h-[calc(100vh-11rem)] min-h-[28rem]">`; the sidebar is `<aside className="hidden md:flex w-56 shrink-0 flex-col gap-2">` and contains the only New chat button, the entire conversation map and every delete control.
 - **Impact.** On a phone the assistant is one ephemeral thread — no history, no new chat, no switching, no delete. The fixed 100vh-based height also fights mobile browser chrome.
 - **Fix.** Move the conversation list into a sheet/drawer behind a header button below md, and put New chat in the control bar rather than only in the hidden aside. Replace the `h-calc` with `flex-1` inside a `min-h-0` column.
- ☐ **[MEDIUM]** The route loop — the part that writes to real records — has no test at all
 - *Tests · hours* — `src/lib/assistant/tests/` contains exactly `models.test.ts`, `scopes.test.ts`, `undo.test.ts`; `src/lib/mcp/tests/` contains `collect.test.ts`, `invoke.test.ts`, `live-api.test.ts`. A repo-wide `find src -name "*.test.ts"` returns 14 files, with nothing for `src/app/api/assistant/route.ts` and nothing for `src/lib/role.ts`.
 - **Impact.** The tests that exist guard the pure, already-careful modules; the tool loop, the persistence round-trip and the role fallback — where all four serious defects above live — are unguarded.
 - **Fix.** Three targeted tests: (1) route-loop with a mocked Anthropic client asserting the returned message array is well-formed after a `MAX_ITERATIONS` exit and after a `max_tokens` truncation (catches #3); (2) `role.ts` asserting a member-lookup error resolves to the most-restricted role, not viewer (#4); (3) an undo round-trip that snapshots and re-inserts a leads row, which fails today on `identity_key` (#5).
- ☐ **[LOW]** Opening a saved conversation fails silently
 - *UX · minutes* — `src/components/assistant/assistant-chat.tsx` `openConversation` wraps `getAssistantConversation` in a try/catch whose body is only the comment `'// Falls through to an empty chat rather than a broken one.'` `src/lib/actions/assistant.ts` `getAssistantConversation` throws on `convErr`, on `Conversation not found`, and on `msgErr` — all three land in that empty catch.
 - **Impact.** The user taps a chat, the spinner runs and stops, nothing changes — indistinguishable from a slow network, a permissions problem or a deleted row. No message, no retry.
 - **Fix.** Set an error state and render it in the message pane with a Retry button, matching how the send path already surfaces the server's own message.
- ☐ **[LOW]** Attached photos are stored as base64 in Postgres and re-fetched whole on every open
 - *Perf · days* — `src/components/assistant/images.ts` `toImageBlock` emits `source: { type: "base64", data: img.data }`; `assistant-chat.tsx` builds `userBlocks` from those and passes `userContent: userBlocks` to `saveAssistantTurn`, which writes into `assistant_messages.content` JSONB. `src/lib/actions/assistant.ts` `getAssistantConversation` reads with `.select("*")` ordered by seq and no limit. `images.ts` sets `MAX_EDGE = 1568`, `MAX_IMAGES_PER_MESSAGE = 4`, `canvas.toDataURL(OutType, 0.85)`.
 - **Impact.** Conversation rows grow unboundedly with photo turns and the whole thing reloads through one server-action payload — slow on cellular, which is the scenario the camera button exists for. Real but not yet reached by anyone.

- **Fix.** When it matters: upload attachments to a Storage bucket and persist a reference, rehydrating base64 only for turns sent to the API. Cheaper interim step is paginating `getAssistantConversation` and stripping image blocks from older rows.
- ☐ **[LOW]** The chat surface hand-rolls everything instead of composing the design system
- *UI · hours* — `src/app/(dashboard)/assistant/page.tsx` uses `ch-page-header`, `ch-page-title`, `ch-page-subtitle` AND `ch-empty` — contradicting the finding's 'only design-system class is `ch-empty`'. Verified true: `assistant-chat.tsx` uses raw ☐s for the app's Select, use the `kirei EmptyState`, replace `truncate` with `line-clamp-2/break-words` on titles, and wire the existing `renameAssistantConversation` to a menu item plus a filter input above the list. Drop the inaccurate `ch-*` sentence from the report.

Batch 2.10 — Auth, signup & first-run onboarding

Health: rough · **Verdict:** keep-core · **Findings:** 1 blocker · 2 high · 7 medium · 0 low **Estimated:** ~3.3 days (10 scheduled)

What it does. A new customer hits `/auth/register`, which POSTs `name/email/password` to `/api/auth/signup` — a public route that uses the Supabase SERVICE ROLE key to create the user with `email_confirm:true` (no verification email ever sent), then signs them in client-side and pushes to `/dashboard`. The dashboard layout (`src/app/(dashboard)/layout.tsx:39`) finds zero businesses and redirects to `/onboarding`, a 3-step wizard: business details + industry preset picker + logo, then optional bank details, then a "you're all set" screen. Finishing inserts a businesses row and best-effort applies an industry preset (writes `business_agent_installs` rows that decide which sidebar modules exist). The user then lands on a completely empty dashboard — no sample data, no getting-started checklist, no guided next action. Invited team members take a variant path through `/api/activate-invite`. Password reset exists as pages but does not function.

Why this rating. The happy path (register → onboard → dashboard) works, but the two flanking paths are broken or unsafe: password reset can never complete because the middleware bounces the recovery session to `/dashboard`, and the signup endpoint is an unauthenticated, unrate-limited service-role user factory. The submit control on both login and register is a `div`, so keyboard users cannot sign up at all. And the destination — the dashboard — has nothing in it, which is the actual conversion problem: 13 users, 2 active.

If you only do three things:

1. Fix the password reset blocker — one-line exemption for `/auth/reset-password` in `src/lib/supabase/middleware.ts:13`. Right now a customer who forgets their password can never set a new one, and the reset email doubles as a permanent passwordless login link. Minutes of work, and it is currently a data-loss-grade support problem waiting for the first paying customer.
2. Make the first five minutes lead somewhere. Delete step 2 of the onboarding wizard (bank details — every field is optional and it is the narrowest point of the funnel), and add a first-run getting-started checklist to `src/components/dashboard/dashboard-client.tsx` driven off the stats already fetched: add a customer, send a quote, create a job, connect Stripe. Today the wizard ends on 'start creating invoices right away' and drops the user on a screen of zeros with no link to do so — this is the most likely code-level cause of 13 users and 2 active.
3. Close the signup endpoint and fix the submit buttons. Add `rateLimit()` from `src/lib/booking/public.ts` to `/api/auth/signup` (every other public route already uses it, and this one holds the service-role key), and replace the `motion.div` submit controls on login and register with real

so pressing Enter works. These are the cheapest two fixes with the largest blast radius: one stops anonymous account spam against your Supabase bill, the other stops silently losing every keyboard user on the very first screen.

Scheduled

- ☐ **[BLOCKER]** Password reset is impossible — middleware redirects the recovery session to `/dashboard`

- *Function · minutes* — `src/lib/supabase/middleware.ts:13` `const isAuthRoute = pathname.startsWith("/auth");`;:108-112 `if (claims && isAuthRoute) { url.pathname = "/dashboard"; return NextResponse.redirect(url); }`. `src/app/auth/forgot-password/page.tsx:26` `redirectTo: ${NEXT_PUBLIC_APP_URL}/auth/callback?next=/auth/reset-password`. `src/app/auth/callback/route.ts:11-13`. `src/app/auth/reset-password/page.tsx:32-36` checks `getSession`, meaning it expects to be reached WITH a session — exactly the state middleware bounces.
- **Impact.** No user can ever set a new password from a reset link; the entire reset-password UI (98 lines) is unreachable. The emailed link instead acts as a passwordless sign-in that leaves the forgotten/compromised password live. With 13 users this has not generated support noise yet, but it blocks account recovery for every future customer.
- **Fix.** Exempt the route in `middleware.ts:13`: `const isAuthRoute = pathname.startsWith("/auth") && pathname !== "/auth/reset-password"`. Verify by requesting a reset and confirming the page renders in the 'ok' state.
- ☐ **[HIGH]** `/api/auth/signup` is a public, unrate-limited service-role user factory with no email verification
- *Security · hours* — `src/lib/supabase/middleware.ts:46` `pathname === "/api/auth/signup"` inside `isPublicRoute`, which returns at :68 before any auth. `src/app/api/auth/signup/route.ts:18-29` builds a `SUPABASE_SERVICE_ROLE_KEY` admin client and calls `admin.createUser({ ..., email_confirm: true })`. No captcha, origin check, or rate limit anywhere in the 36-line file. `src/lib/booking/public.ts:55` exports `rateLimit(key, limit, windowMs)` used by other public endpoints.
- **Impact.** Unlimited scripted account creation against the production Supabase project — inflated auth rows and MAU billing, and confirmed accounts on addresses the registrant never proved they own.
- **Fix.** Add `rateLimit(ip, 3, 60_000)` from `src/lib/booking/public.ts` at the top of the POST handler (returning 429), matching the pattern in `/api/f/[slug]/submit`. Email verification is a separate, larger decision — do the rate limit now.
- ☐ **[HIGH]** The submit control on login and register is a `motion.div` — keyboard users cannot sign up or sign in
- *UI · minutes* — `src/components/ui/kirei/animated-press.tsx:17-32` — `<motion.div ref={ref} whileTap=... {...rest}>`, typed `HTMLMotionProps<"div">`. `src/app/auth/login/page.tsx:131-137` and `src/app/auth/register/page.tsx:188-194` use it as the sole submit control with `onClick={handleSubmit(onSubmit) as unknown as () => void}` — the cast itself signals the type mismatch.
- **Impact.** Pressing Enter in the password field does nothing on either the sign-in or sign-up form — the most common submission gesture. Tab never reaches the control and screen readers see a plain div. This is the first and last screen of the conversion funnel.
- **Fix.** Replace `AnimatedPress` with `<button type="submit">` in both files using `src/components/ui/button.tsx`, or add an `as` prop to `AnimatedPress` so it can render `motion.button` with `type="submit"`. Remove the redundant `onClick` so the form's `onSubmit` is the single path.
- ☐ **[MEDIUM]** Onboarding creates a duplicate business every time it is submitted
- *Function · hours* — `src/app/onboarding/page.tsx:95-101` `(supabase.from("businesses") as any).upsert({ user_id: user.id, ...step1Data, ...data, logo_url, industry_preset })`. `supabase/migrations/007_multi_business.sql:4` `ALTER TABLE public.businesses DROP CONSTRAINT IF EXISTS businesses_user_id_key;`. `src/app/(dashboard)/layout.tsx:39` redirects TO `/onboarding` when `allBusinesses` is empty; nothing redirects away. Counter-evidence: `src/app/onboarding/page.tsx:278` `disabled={saving}`.
- **Impact.** A user who revisits `/onboarding` after setup silently gains a second empty business. It appears in the `BusinessSwitcher`, and since `layout.tsx:44` falls back to `allBusinesses[0]` when no cookie is set, they can land in the empty one and believe their data is gone.

- **Fix.** At the top of the onboarding page, query for an existing owned business and redirect to /dashboard if one exists. Change .upsert to .insert so intent is explicit. The button-disable half of the proposed fix is already in place.
- ☐ **[MEDIUM]** The first five minutes end on a wall of zeros — no sample data, no getting-started checklist
 - *Missing · days* — src/app/(dashboard)/dashboard/page.tsx (52 lines, no zero-state branch). src/components/dashboard/dashboard-client.tsx:214-223 (invoice empty state WITH CTA) and :263-267 (schedule empty state, no CTA). No checklist component exists in src/components/dashboard/. src/app/onboarding/page.tsx:293-296 ends on 'Your workspace is ready' and a bare Go-to-dashboard button.
 - **Impact.** A new owner lands on live-stats widgets showing zeros with no guided next step beyond a single invoice CTA. The onboarding wizard promises 'start creating professional invoices right away' and then links nowhere in particular.
 - **Fix.** Add a first-run branch in dashboard-client.tsx when the stats show zero invoices and zero jobs: a 4-step checklist (add customer, send quote, create job, connect Stripe) built from CardListRow/EmptyState in src/components/ui/kirei/, ticked off from the stats already fetched. Prefer this over seeded sample data the user must then delete.
- ☐ **[MEDIUM]** Onboarding asks for bank account details before the user has seen the product
 - *UX · hours* — src/app/onboarding/page.tsx:36-43 (step2Schema, all optional); :243-285 (step 2 render, Back + Finish only); :252, :265, :270, :231 (UK placeholders/default).
 - **Impact.** The second thing Kirei asks a stranger for is bank and sort-code details, with no apparent way past it, at the narrowest point of the funnel. Because every field is optional it collects nothing from anyone cautious — pure drop-off.
 - **Fix.** Minimum: add a 'Skip for now' link beside Finish setup and label the section optional. Better: drop step 2 entirely (bank details already live in Settings) and localise the placeholders/country default away from UK.
- ☐ **[MEDIUM]** Onboarding is off-brand — hardcoded blue against the app's teal accent, no design-system primitives
 - *UI · hours* — src/app/onboarding/page.tsx:132 `bg-blue-500 text-white`; :136 `bg-blue-500`; :161 `border-blue-500 ring-1 ring-blue-500/40 bg-blue-500/5 ... hover:border-blue-300`. src/app/globals.css:142 `--primary: 191 38% 36%; /* deep teal */`. src/app/auth/register/page.tsx:131 `bg-blue-50 border-blue-200`, :134 `text-blue-800`, :135 `text-xs text-blue-600 truncate`.
 - **Impact.** Signup and onboarding read as a different product from the teal app behind them. A long invited email is clipped mid-string in the banner, so the user cannot confirm the address they were told to register with.
 - **Fix.** Replace every blue-* token with the primary/accent CSS variables, rebuild the step rail and industry cards on KireiPill/CardListRow/FormSection, and change `truncate` to `break-words` at register/page.tsx:135.
- ☐ **[MEDIUM]** Logo upload silently fails and ignores its own stated 2MB limit
 - *Function · minutes* — src/app/onboarding/page.tsx:61-66 `const file = e.target.files?.[0]; if (!file) return; setLogoFile(file); setLogoPreview(URL.createObjectURL(file));`; :85-91 `const { error: uploadError } = await supabase.storage.from("logos").upload(...); if (!uploadError) { ... logo_url = urlData.publicUrl; }`; :195 UI text 'PNG, JPG or SVG, max 2MB'.
 - **Impact.** A user who uploads an oversized file sees the local preview render, gets the success screen, and ends up with no logo — after which every invoice and quote PDF goes out unbranded with no explanation.
 - **Fix.** Validate size (2MB) and MIME type in handleLogoChange before setState, with a toast on rejection; add an else branch on uploadError that toasts the failure.
- ☐ **[MEDIUM]** auth/callback does not validate its `next` parameter, unlike the login page

- *Security · minutes* — `src/app/auth/callback/route.ts:7` `const next = searchParams.get("next") ?? "/" ;`
`:13 return NextResponse.redirect( ${origin}${next} )` — no validation. Contrast
`src/app/auth/login/page.tsx:33-34` `const nextParam = /^\/(?!\/)/.test(nextParamRaw) ?`
`nextParamRaw : ""` with the comment 'so this can't be used as an open redirect'.
- **Impact.** A crafted next value beginning with @ redirects off-origin after a successful code exchange. The guard already written for the login page is simply absent on the OAuth/recovery entry point.
- **Fix.** Apply the same `/^\/(?!\/)/` test in `callback/route.ts:7`, defaulting to `"/dashboard"` on failure. Extract it as a shared `safeNext()` helper so login and callback cannot drift again.
- ☐ **[MEDIUM]** Zero tests cover the entire signup, auth and onboarding path
 - *Tests · hours* — find `src -name '.test.ts'` returns 14 files: `src/lib/lib/tests/pg-filter.test.ts`,
`src/lib/assistant/tests/{models,scopes,undo}.test.ts`, `src/lib/booking/tests/{availability,booking-`
`db,time}.test.ts`, `src/lib/content/tests/{live-agents,pipeline,prompts,schedule}.test.ts`,
`src/lib/mcp/tests/{collect,invoke,live-api}.test.ts`. None reference auth, onboarding, or middleware.
 - **Impact.** The conversion path is the least-tested code in the repo, and it is the path where a regression costs new customers rather than annoying an existing one. The reset-password blocker sat live and invisible precisely because nothing asserts on it.
 - **Fix.** Start with the highest-value one: `src/lib/supabase/tests/middleware.test.ts` asserting `updateSession` does NOT redirect `/auth/reset-password` when claims are present, and DOES redirect `/auth/login`. Add a `presets.test.ts` covering `presetInstallRows` vs `OPTIONAL_PLUGINS`. Skip the DB-backed onboarding test until a staging database exists.

Batch 2.11 — Mobile app (React Native / Expo) — mobile/

Health: rough · **Verdict:** simplify · **Findings:** 0 blocker · 6 high · 4 medium · 0 low **Estimated:** ~8.6 days (6 scheduled, 4 deferred)

What it does. An Expo SDK 54 / expo-router app (78 source files, ~12.7k lines) that talks directly to the same Supabase Postgres as the web app — no API layer, RLS does the filtering. It has two personalities driven by `useActiveBusiness().role`: workers get a hard-isolated Jobs/Tasks/Profile shell (deep-link guard in `app/_layout.tsx:65-70` plus `WORKER_BLOCKED_SEGMENTS` in `src/lib/permissions.ts:43`), and owners/admins get a Home dashboard, Sales, Tasks, Schedule and roughly 20 back-office screens (customers, invoices, quotes, leads, work orders, products, reports, recurring, team, analytics, messages, settings, an AI chat at `app/agent/index.tsx` hitting `/api/mobile/agent`). The worker's job screen (`app/job/[id].tsx`) does maps deep-link, call/email contacts, batch photo upload to the work-order-photos bucket, a free-text note, and three status buttons. There is a full light/dark theme system (`src/lib/theme.ts` mutates `colors` in place) and local-only push reminders (`src/lib/notifications.ts`). It is NOT at feature parity with the web despite CLAUDE.md saying so — contracts, forms, onboarding, content, SEO, expenses, inventory, timesheets, assets, bookings, contacts, sites and Stripe payment collection have no mobile surface at all.

Why this rating. It launches, looks good, and the worker flow is coherent. But the owner-side screens re-implement money-writing logic that the web spent production fires getting right — racy invoice/quote number minting in 5 places, a status button that flips an invoice to "paid" without touching `amount_paid` or writing a payments row, and the exact PostgREST `.or()` injection that PR #398 fixed on web (17 sites) still live in the lead→quote converter where it silently creates duplicate customers. On top of that, a field-services app has zero offline support and 19 fetch sites swallow `error`, so a tradie in a dead zone sees "You have no invoices yet" rather than "offline". Zero tests in mobile/.

If you only do three things:

1. Fix the money and data-integrity bugs before anything cosmetic: the `.or()` injection in `app/leads/[id].tsx:122` that creates duplicate customers, the five racy number mints (swap to `next_invoice_number/next_quote_number` RPCs), and `setStatus('paid')` in `app/invoices/[id].tsx:80` writing no `amount_paid` and no payments row. These are a day's work total and each one corrupts real records for the one customer who depends on this.
2. Make failure visible, then make it survivable. Step one is mechanical: destructure `{ data, error }` at all 19 fetch sites and render a real error/offline state instead of `EmptyState` — today an offline phone claims the business has no invoices. Step two is a persisted react-query cache (MMKV is already a dependency, unused) so a cold launch in a dead zone still shows last-known jobs. Offline is the defining feature of every competitor's field app.
3. Pick a lane on the job screen and finish it: add a start/stop timer and a materials picker to `app/job/[id].tsx` (the tables and MCP tools already exist), and delete the mobile back-office screens that duplicate the web. The worker flow is what makes Kirei worth having on a phone; the phone-sized copy of the owner dashboard is where all the bugs live and is the reason every web PR now owes a mobile PR.

Scheduled

- ☐ **[HIGH]** Lead→quote conversion interpolates lead email/phone into `.or()` and swallows the error, creating duplicate customers
 - *Function · hours* — `mobile/app/leads/[id].tsx:122-126` is verbatim as claimed: `const filters = [lead.email && email.eq.${lead.email} , lead.phone && phone.eq.${lead.phone} ].filter(Boolean).join(",")` then `.or(filters).limit(1).maybeSingle()` destructuring only `{ data: existing }`. Lines 128-139 insert a new customer when `customerId` is null, with `country: "Australia"` hardcoded at :133. There is no pg-filter equivalent anywhere in mobile/ (grep for `ilikeAcross` returns nothing).
 - **Impact.** Confirmed data-integrity defect: any lead whose captured phone or email contains a comma or a period-delimited segment that breaks PostgREST's or-grammar 400s the lookup, the error is discarded, and a duplicate customer row is created for someone already in the book. Split history across two customer rows is exactly the failure PR #398 fixed on web.
 - **Fix.** Replace the single `.or()` with two sequential `.eq()` lookups (email, then phone), destructure `{ data, error }` and throw on error instead of silently falling through to the insert. Do not hardcode country; drop the field or source it from business settings after confirming the column exists.
- ☐ **[HIGH]** No offline support at all, and list fetches report failure as an empty list
 - *Missing · weeks* — `mobile/src/lib/supabase.ts:14-21` creates a plain client with `AsyncStorage` session persistence and no query cache. `react-native-mmkv` is at `package.json:46` and imported nowhere. Grep for `NetInfo/persistQueryClient` across `app/` and `src/` returns zero hits. Only 4 files use react-query (`app/(tabs)/index.tsx`, `(tabs)/schedule.tsx`, `(tabs)/tasks.tsx`, `app/job/[id].tsx`); the remaining lists do `const { data } = await supabase...; setX(data ?? [])` — verified at `invoices/index.tsx:64`, `work-orders/index.tsx:51`, `leads/index.tsx:88`, `customers/index.tsx:37`.
 - **Impact.** On a roof with no signal every list renders its `EmptyState`, indistinguishable from genuine emptiness, with no retry affordance and no last-known data. Photos cannot be queued. This is the standard reason a trades crew abandons a field app versus ServiceM8/Tradify/Jobber.
 - **Fix.** Step 1 (cheap, do now): destructure `{ data, error }` in the list fetches and render a distinct error/offline state with a retry, instead of `EmptyState`. Step 2: move list screens onto react-query and add `@tanstack/react-query-persist-client` backed by the already-installed MMKV, plus a `NetInfo` banner. Photo queueing is a later phase.
- ☐ **[HIGH]** Tapping the PAID status pill flips the invoice without setting `amount_paid` or writing a payments row

- *Function · hours* — mobile/app/invoices/[id].tsx:79-88 — `setStatus` is `await supabase.from("invoices").update({ status }).eq("id", invoice.id)` and nothing else; the result is not destructured so the error is unreadable. It even fires the confetti on paid (:86). Contrast `recordPayment` at :90-113 which does insert a payments row and set `amount_paid`. The finding's cited line 346 for the tappable pill row is approximate but the wiring is real — `setStatus` is the pill handler.
- **Impact.** An invoice marked paid via the pill has `amount_paid` = 0 permanently and no payments row, so there is no record of how or when it was paid, and reporting understates collections. If it is a deposit/child invoice, `trg_reconcile_parent_invoice` recomputes the parent from children's `amount_paid` and contributes \$0 — reintroducing the exact bug migration 20260510232842 exists to fix.
- **Fix.** In `setStatus`, when `status === 'paid'`, also set `amount_paid = num(invoice.total)` and insert a payments row the way `recordPayment` does — or better, route the pill through the existing `recordPayment` modal. Check the update's error. Recompute `amount_paid` server-side rather than from stale client state.
- ☐ **[HIGH]** Dark mode is broken in the shared `StatusPill` and in module-scope style consts, including the worker's job screen
 - *UI · hours* — mobile/src/components/StatusPill.tsx:18-41 — TONES pairs theme-swapped soft* gradients with hardcoded light-mode foreground hex: `draft/reviewed = softAmber + '#92400e'`, `cancelled/expired/lost = softRose + '#7f1d1d'`, `sent/scheduled/assigned = softBlue + '#1d4ed8'`. mobile/src/lib/theme.ts:107-109 makes those gradients dark in dark mode (`softAmber` → `['#3a2a0f', '#2a1f08']`, `softRose` → `['#3f1a1f', '#2a1015']`, `softBlue` → `['#0e234a', '#08182f']`), so dark text lands on a dark fill. Frozen-style trap confirmed live at mobile/src/components/LinItemsEditor.tsx:150-155 (module-scope `const inputStyle = { ... backgroundColor: colors.canvas, color: colors.text }`) and app/job/[id].tsx:456-472 (`center`, `backBtn`, `openMapsBtn`). `theme-provider.tsx` applies the palette in a `useEffect` after module load, so those consts capture LIGHT permanently.
 - **Impact.** On a dark-mode phone every invoice/quote/lead/job list shows unreadable status pills, the line-item editor used to build quotes renders a white input on the dark canvas, and the worker's job loading and error screens flash a full-screen white background. Dark mode is a shipped, advertised feature.
 - **Fix.** In `StatusPill`, derive `fg` from the live theme tokens for the soft* tones (or add a per-mode fg to TONES). Convert `LinItemsEditor.tsx:150` and `job/[id].tsx:456-472` to `const x = () => ({...})` invoked as `style={x()}` — the pattern already used correctly in `app/(auth)/login.tsx` and `app/customers/new.tsx`.
- ☐ **[HIGH]** Lists are hard-capped at 100 rows with no pagination and no search
 - *Missing · days* — `.limit(100)` with no range/offset/cursor confirmed at exactly the eight cited sites: `customers/index.tsx:37`, `invoices/index.tsx:64`, `leads/index.tsx:88`, `products/index.tsx:48`, `quotes/index.tsx:61`, `recurring/index.tsx:31`, `reports/index.tsx:32`, `work-orders/index.tsx:51`. Grep for `placeholder="Search"` across app/ returns exactly one hit: `customers/index.tsx:91`. Every other list filters client-side over the already-fetched 100.
 - **Impact.** Rows beyond the 100th are unreachable from the phone — no search, no load-more — and status tabs silently under-report because they filter only the fetched window. The dependent roofing business is near that ceiling on work orders.
 - **Fix.** Add a debounced server-side search (using safe filters, not `.or()` string interpolation) and either `useInfiniteQuery` with `.range()` or a Load-more that bumps the range. Start with work-orders and invoices.
- ☐ **[HIGH]** The worker job screen captures no time and no materials, so job costing is blind for field work
 - *Missing · days* — mobile/app/job/[id].tsx offers photos (`photoMutation` at :73-87 wired to camera/library), a free-text worker note (`worker_notes`, referenced :59-61 and :336) and status buttons (`statusMutation` :63-71). Grep for `job_time|timesheet|clock_in|signature` across mobile/app and mobile/src returns zero hits — no timer, no materials picker, no sign-off. The web side exposes `log_time_block`, `add_job_material`, `get_job_costing` and `get_work_order_financials` over MCP with no mobile surface.
 - **Impact.** A crew on site cannot clock on/off or record materials, so labour and material cost is either re-keyed in the office from a free-text note or lost. Job profitability is unknowable for field work — the core promise of a ServiceM8-class product.

- **Fix.** Add a start/stop timer writing to the table `log_time_block` targets, and a materials row picker reusing the product-catalog fetch already present in the `LineItemsEditor ProductPicker`. A customer sign-off signature pad is the natural third addition.

Deferred (4) — medium/low on a feature marked *simplify*

- **[MEDIUM]** Invoice and quote numbers are minted by a racy client-side read-bump in 5 places, bypassing the race-safe RPCs — All five sites verified verbatim: `mobile/app/invoices/new.tsx:37-43`, `app/quotes/new.tsx:36-42`, `app/quotes/[id].tsx:86-94` (duplicate) and `:128-135` (quote→invoice), `app/invoices/[id].tsx:118-126` (duplicate). Each does `select <x>_prefix, <x>_next_number → ${prefix}-${String(n).padStart(4, "0")}` → `update({ <x>_next_number: n + 1 })` with the update result never destructured. Grep for `next_invoice_number / next_quote_number` across `mobile/` returns nothing.
- **[MEDIUM]** Batch photo upload loses the remaining photos on any single failure, and can orphan uploaded files — `mobile/app/job/[id].tsx:75-80` — `for (const uri of localUris) { const { url } = await uploadJobPhoto(uri); await addWorkOrderPhoto(id!, {...}) }` inside one mutationFn, with a single `onError` Alert at :85. The first throw aborts the loop and reports nothing about partial success. `mobile/src/lib/jobs.ts:62-76` confirms the three-step write: storage upload, then `job_photos` insert (throws at :70), then the `work_orders.photos` JSONB mirror update (throws at :76) — so a mid-sequence failure orphans the storage object or desynchronises the two stores. The mobile screen reads the JSONB mirror (`job.photos`), the web reads `job_photos`.
- **[MEDIUM]** `useActiveBusiness` is a per-component hook, not a context — independent copies, and switching business does not propagate — `mobile/src/lib/active-business.ts:88-130` exports a hook holding its own `useState` for `businesses/active/role` and running `load()` in a `useEffect` on mount, which calls `fetchAccessibleBusinesses` plus `fetchRoleForBusiness` (:63-74). `switchTo` at :120-128 sets `AsyncStorage` then mutates only that instance's `setActive/setRole`. It is imported by 36 files (not the 39 claimed), including `app/_layout.tsx` and `app/(tabs)/_layout.tsx`, which therefore hold separate copies from the dashboard's.
- **[MEDIUM]** Zero tests in `mobile/` — the role gate and the money paths are entirely unverified — `mobile/package.json` scripts are `start/android/ios/web/lint/typecheck/release:build/release:submit/release:ota/release` — no test runner, and no `vitest/jest` in dependencies. No `.test.` files or `tests` directories exist under `mobile/`.

Batch 2.12 — Team, roles & worker isolation

Health: rough · **Verdict:** keep-core · **Findings:** 0 blocker · 2 high · 6 medium · 2 low **Estimated:** ~1.7 days (10 scheduled)

What it does. Kirei has five roles: owner (derived from `businesses.user_id`, never stored), plus `admin/editor/viewer/worker` rows in `business_members`. `/team` is one page that stacks two unrelated things: an "Access levels" section (invite by email, change role, remove — `src/components/settings/team-settings.tsx`) and a "Profiles" grid (workforce details: name, phone, skills, bio — `src/components/team/team-page-client.tsx`). Inviting creates a pending `business_members` row plus a `member_profiles` row and emails a link to `/api/activate-invite`, which calls `activate_pending_memberships()` after login; `link_my_member_profile()` then wires `member_profiles.user_id` by email match on every dashboard load. Worker is meant to be a hard-isolation role enforced in the database: migration 20260430000001 adds `is_business_worker()/my_member_profile_ids()` and rewrites RLS on customers, products, invoices, quotes, payments, reports and `work_orders` so workers see only jobs assigned to them. `src/app/(dashboard)/layout.tsx:60` additionally redirects workers away from any path outside `WORKER_ALLOWED_PATHS`, and `src/lib/assistant/scopes.ts:36` denies workers the AI assistant outright because it runs `service-role`.

Why this rating. The management UI works and the role checks in the server actions are correct and well-layered (owner-only admin promotion, admins can't touch admins). But the headline promise — "worker sees no customer data" — is only half-true at the RLS layer: leads, contacts and sites were never given the `_no_workers`

treatment, so a worker's own JWT can read every prospect and every customer contact in the business via PostgREST or the mobile app. The UI also has a broken invite-link path and an unconfirmed destructive action.

If you only do three things:

1. Close the RLS holes: one migration adding AND NOT is_business_worker(business_id) to leads, contacts and sites, then a structural test that enumerates every business_id table and fails CI when one lacks a worker guard. The documented rule in CLAUDE.md has been silently violated by every table added since April 2026 — documentation is not enforcement.
2. Fix the invite recovery path: point "Copy invite link" at /api/activate-invite (not /auth/register), pass the real business_id instead of the empty string, add a Resend button wired to the already-written-but-unused getMemberInviteCode, and stop silently swallowing a failed invite email. Today a Resend outage costs the owner their whole invite flow with no visible symptom.
3. Merge "Access levels" and "Profiles" into one member row on /team. They are one concept split across two tables, two add-forms, two delete behaviours (one confirmed, one not) and an amber callout that exists only to reconcile them. One row per person, role Select inline, expandable panel for phone/skills/bio, one confirmed delete that deactivates both records — rebuilt on the .ch-* and Kirei primitives the leads workspace uses.

Scheduled

- ☐ **[HIGH]** Workers can read every lead in the business
 - *Security · minutes* — supabase/migrations/20260412000001_leads.sql:42-49 — CREATE POLICY "leads_business_access" ON leads USING (business_id IN (owned UNION active memberships)). No FOR clause (defaults FOR ALL), no WITH CHECK (so USING is reused as the check), and no AND NOT public.is_business_worker(business_id). I grepped every migration for policies on leads: 20260412000001 is the only one — nothing later re-creates or tightens it. Contrast supabase/migrations/20260430000001_worker_role_and_isolation.sql:76-93 (customers_no_workers) which spells the guard out on both USING and WITH CHECK, and which covers only customers/products/invoices/quotes/payments/reports/work_orders/work_order_assignments.
 - **Impact.** A worker's JWT hitting PostgREST directly (mobile/ uses @supabase/supabase-js, so RLS is the only filter) can SELECT, UPDATE and DELETE every lead row — name, email, phone, address, source, notes. The web UI hides it, so nothing surfaces the exposure. src/components/settings/team-settings.tsx:288 promises workers see "no invoices, quotes, customers, or financials".
 - **Fix.** New migration: DROP POLICY "leads_business_access" ON public.leads; recreate as leads_no_workers FOR ALL with the same predicate AND NOT public.is_business_worker(business_id) on both USING and WITH CHECK, copying 20260430000001_worker_role_and_isolation.sql:76-93 verbatim.
- ☐ **[HIGH]** contacts and sites are also missing the worker exclusion — customer PII leaks
 - *Security · minutes* — supabase/migrations/20260428000002_unified_contacts.sql:71-79 — CREATE POLICY "contacts_all" ON public.contacts FOR ALL USING (owned UNION active members) WITH CHECK (same); no is_business_worker guard. supabase/migrations/20260418000001_account_site_portfolio.sql:51-61 — CREATE POLICY "sites_all" ON public.sites FOR ALL, identical shape, same omission. A grep of all migrations for "ON public.contacts" / "ON public.sites" returns no later tightening.
 - **Impact.** contacts holds every named person at every customer account; sites holds every serviced property address plus gate_code, access_notes and parking_notes (20260418000001:35-41). A worker gets read and write on all of it. This nullifies the deliberately narrow hole in supabase/migrations/20260512000001_workers_can_see_job_customers.sql, which was written so a worker could see only the customer attached to their assigned job.
 - **Fix.** Same migration: recreate contacts_all and sites_all with AND NOT public.is_business_worker(business_id) on USING and WITH CHECK. Verify against the live DB's pg_policies first — the migration files are the only evidence I could open here, and this repo's migration tracking is

known to drift.

- ☐ **[MEDIUM]** "Copy invite link" sends people to a different URL than the invite email, and usually without the business id
 - *Function · minutes* — `src/lib/actions/members.ts:97-98` builds the emailed link as `${appUrl}/api/activate-invite?biz=...&email=...` `src/components/settings/team-settings.tsx:191-193` builds `${window.location.origin}/auth/register?email=...` and only sets `biz` when `member.business_id` is truthy; `team-settings.tsx:62` hardcodes `business_id: ""` on the optimistic row pushed at :58-70. `src/app/api/activate-invite/route.ts:14-22` handles both signed-in and signed-out.
 - **Impact.** Owner adds a worker, immediately clicks Copy invite link (before any refresh), texts it over. The link has no `biz` param, so the worker registers and lands on `/dashboard` with no membership activated — typically into their own empty business. Recovery is delete-and-re-add.
 - **Fix.** Point the copy button at `/api/activate-invite?biz=...&email=...` so it matches the email, and pass the real `businessId` into the optimistic row — or drop the optimistic insert and call `router.refresh()` after `addMember` resolves.
- ☐ **[MEDIUM]** Removing a team member is a one-click, unconfirmed, irreversible destructive action
 - *UX · minutes* — `src/components/settings/team-settings.tsx:229-239` — ghost icon Button, h-8 w-8, `onClick={() => handleRemove(member.id, member.email)}` with no dialog; `handleRemove` at :92-105 calls `removeMember` immediately. `src/components/team/team-page-client.tsx:201-231` wraps the profile delete on the same rendered page (`TeamSettings` is mounted at `team-page-client.tsx:376`) in a full `AlertDialog` with a named title and "permanent and cannot be undone" copy.
 - **Impact.** A mis-tap next to the role Select revokes a person's access with no confirm and no undo. Two adjacent destructive controls on one page with opposite confirmation behaviour.
 - **Fix.** Wrap the remove button in the same `AlertDialog` pattern used at `team-page-client.tsx:201-231`, naming the member's email in the title.
- ☐ **[MEDIUM]** Removing a member orphans their `member_profiles` row — they stay assignable to jobs
 - *Function · hours* — `src/lib/actions/members.ts:202-206` deletes from `business_members` only; nothing in the file references `member_profiles` after the `addMember` upsert. `src/lib/actions/member-profiles.ts:36-47` `getAssignableProfiles` filters `.eq("is_active", true)` with no membership join. `src/lib/actions/members.ts:78-81` upserts `member_profiles` with `ignoreDuplicates: true`, so re-adding reuses the stale row untouched. `link_my_member_profile (20260430000001_worker_role_and_isolation.sql:54-66)` only fills `user_id` WHERE `user_id IS NULL`, so the departed user's id stays on the profile.
 - **Impact.** A departed worker keeps appearing in the assignable-workers picker and can be scheduled onto jobs nobody will do. On re-hire the stale profile is silently reused.
 - **Fix.** In `removeMember`, also set `member_profiles.is_active = false` and `user_id = null` for that `business_id` + email, and surface a Deactivated filter on the Profiles grid so the row is recoverable.
- ☐ **[MEDIUM]** `/team` silently renders an empty team when its queries fail
 - *Function · minutes* — `src/app/(dashboard)/team/page.tsx:30-33` — `Promise.all([getMemberProfiles().catch(() => []), getMembers().catch(() => [])])`. The empty result renders `src/components/settings/team-settings.tsx:245-249`, "No team members yet. Add someone above to get started."
 - **Impact.** Any RLS denial or transient Postgres failure presents as a genuinely empty team. The rational owner response is to re-add everyone, producing duplicate pending rows and duplicate invite emails. Same failure shape as the `.or()` incident in `CLAUDE.md` — a swallowed error becomes fabricated emptiness.
 - **Fix.** Let the errors propagate to a route `error.tsx` boundary, or catch and render an explicit "Couldn't load your team — retry" state. Do not conflate error with empty.
- ☐ **[MEDIUM]** No resend invite, no pending-invite expiry, and the invite-code action is dead code

- *Missing · hours* — `src/lib/actions/members.ts:132` exports `getMemberInviteCode`; sole grep hit in `src/` is the declaration itself. `members.ts:105` passes `inviteCode` into the email body, so the code is generated and emailed but never surfaced in-app. `members.ts:115-117` — `catch {}` with the comment "Email failure is non-fatal". No expiry column is read by `src/app/api/activate-invite/route.ts`.
- **Impact.** When Resend drops or spam-filters the invite, the owner has a pending row, no resend, no way to read the code the email carried, and a Copy-link button pointing at a different URL. The practical fallback is delete-and-re-add.
- **Fix.** Add `resendInvite(memberId)` reusing the `addMember` email block; surface it and the existing `getMemberInviteCode` on pending rows; return a send-failed flag from `addMember` so the toast can say "added, but the email failed to send".
- ☐ **[MEDIUM]** Zero tests on the role logic and the worker RLS that the whole isolation story rests on
 - *Tests · hours* — Glob of `src/**/*.test.ts` returns 14 files: `booking (3)`, `pg-filter`, `assistant (models/scopes/undo)`, `content (4)`, `mcp (3)`. No `src/lib/tests/permissions.test.ts`, no test importing `src/lib/actions/members.ts`, no structural test over the policies in `20260430000001_worker_role_and_isolation.sql`.
 - **Impact.** Nothing detects a new `business_id` table landing without a `_no_workers` policy — which is exactly how leads, contacts and sites (findings 1 and 2) shipped open. The CLAUDE.md rule "mirror the

`_no_workers` policy" is documented but unenforced.

- **Fix.** (1) `src/lib/tests/permissions.test.ts` — table-driven over `canManageTeam / canSeeFinancials / isPathAllowedForWorker` per role, asserting a worker is denied `/leads`, `/customers`, `/invoices`. (2) A DB-backed test alongside `src/lib/booking/tests/booking-db.test.ts` enumerating every table with a `business_id` column and asserting each has a policy referencing `is_business_worker`, with an explicit allow-list for the intentional exceptions. Decide tasks explicitly and pin it.
- ☐ **[LOW]** Admins see a blank owner row, and every email truncates instead of wrapping
 - *UI · minutes* — `src/app/(dashboard)/team/page.tsx:28` — `const ownerEmail = biz?.user_id === user.id ? (user.email ?? "") : ""`; threaded through `TeamPageClient` into `TeamSettings` at `src/components/team/team-page-client.tsx:376`. `src/components/settings/team-settings.tsx:165` renders


```
{ownerEmail}
```

; member emails at :183 use truncate too.
 - **Impact.** An admin's `/team` shows a first row with an empty name line under an Owner badge. Long emails clip on mobile with no way to read them, against the stated break-words preference.
 - **Fix.** Resolve the owner's email server-side (join `auth.users` or denormalise `businesses.owner_email`) and swap `truncate` → `break-words` on both lines.
- ☐ **[LOW]** Role badges hand-roll Tailwind colours instead of using `.ch-pill / KireiPill`
 - *UI · hours* — `src/components/settings/team-settings.tsx:25-31` — `ROLE_COLORS` literal map (`bg-amber-100 / bg-purple-100 / bg-blue-100 / bg-slate-100 / bg-lime-200` with hardcoded borders), applied via `raw` in `RoleBadge` at :273-281. `src/components/team/team-page-client.tsx:345-361` — `Card` `className="border-amber-200 bg-amber-50/50 dark:bg-amber-900/10 dark:border-amber-800"` with buttons at :353-361 styled `border-amber-300 bg-white text-amber-800` and no `dark: variant`.
 - **Impact.** Role colours share no vocabulary with the `.ch-pill` status system used on every list page, and `bg-lime-200` is a leftover from the abandoned flux accent clashing with the deep-teal Connected Hub palette. The amber callout's `bg-white` pills do not adapt to dark mode.
 - **Fix.** Extend `.ch-pill` in `globals.css` with `owner/admin/editor/viewer/worker` tones from the teal palette, replace `ROLE_COLORS + RoleBadge` with `KireiPill`, and add `dark: variants` to the callout pills.

Batch 2.13 — Leads pipeline (Board / List / Calendar workspace, redesigned in 2501e01 / PR #405)

Health: usable · **Verdict:** keep-core · **Findings:** 0 blocker · 1 high · 5 medium · 3 low **Estimated:** ~2.9 days (9 scheduled)

What it does. /leads loads every lead for the active business in one server call (src/app/(dashboard)/leads/page.tsx:5 → getLeads, no limit) and hands the whole array to a single client orchestrator, LeadsClient, which owns all lead state, the add/edit/delete dialogs and every mutation. Three views render that same array: a 5-column @dnd-kit Board where dragging a card to a column writes its status, a List with stage-filter tabs and 12-per-page in-memory pagination, and a Calendar that buckets leads by the day they arrived. The active view is stored in ?view= so it survives refresh. Per-lead actions (edit, delete, move to stage, add as contact, convert to customer/quote/work order) live in one shared dropdown used by all three views; conversions call the existing server actions which create the downstream record and navigate to it. New leads go through the upsert_lead RPC, which deduplicates on identity_key rather than inserting blindly.

Why this rating. The redesign is well-executed craft — the three views share one vocabulary file, the board's drag is real, the list pages correctly and clamps, the calendar avoids the UTC date-shift trap. But it sits on a data layer with two genuine defects it inherited and did not fix: adding a lead that already exists renders a phantom duplicate card with a duplicate React key, and the leads table has no worker-isolation RLS policy at all despite CLAUDE.md claiming it does. Neither is caused by the redesign; both are reachable by the one business actually using this.

If you only do three things:

1. Close the two data-layer defects, in this order: add the leads_no_workers RLS policy (a worker can currently read and delete the whole lead book through the mobile app's direct PostgREST access), then fix the duplicate-card bug at leads-client.tsx:151 so adding an already-known lead reports the merge instead of faking a second one. Both are minutes of work and both are things a real user hits.
2. Make the workspace tell the truth about its own state: validate ?view= so a bad link doesn't render a blank page, resync leads from the server prop so cron- and form-created leads appear, and show a 'N of M match' chip when search is filtering. Three small changes that together move the page from 'a snapshot that sometimes lies' to 'live'.
3. Put money on the pipeline — add value, follow_up_at and lost_reason to leads and sum value into each board column header. This is the single biggest gap against ServiceM8/Jobber and the reason the board currently informs rather than drives. Fix the mobile touch-none scroll problem at the same time, since the board is the default view and the owner reads it on a phone.

Scheduled

- ☐ **[HIGH]** The leads table has no worker-isolation RLS policy — workers can read and write every lead
 - *Security · minutes* — supabase/migrations/20260412000001_leads.sql:42 is the ONLY policy on leads (`CREATE POLICY "leads_business_access" ON leads USING (business_id IN (owner UNION active business_members))`) — membership-only, no `is_business_worker` term. `grep -rn "POLICY" supabase/migrations/*.sql | grep -i lead` returns that single line and nothing else; `grep -n leads supabase/migrations/20260430000001_worker_role_and_isolation.sql` returns zero hits, while that file spells out customers_no_workers (line 76), products_no_workers (line 97) and the rest. The mobile exposure is real, not theoretical: mobile/app/leads/index.tsx:85 queries `supabase.from("leads").select(...).eq("business_id", active.id)` straight against PostgREST, and mobile/app/leads/[id].tsx:72,99,168 issue `.update()` on leads. Web is masked as the finding says — WORKER_ALLOWED_PATHS (src/lib/permissions.ts:40-47) excludes /leads.

- **Impact.** Any worker's session token can select, update or delete every lead row for that business — names, phones, emails, the whole sales pipeline — via the mobile client or a raw PostgREST call. CLAUDE.md's claim that `is_business_worker` 'denies access to invoices/customers/quotes/leads/etc.' is false for leads, so the gap is invisible to anyone trusting the docs.
- **Fix.** New migration: drop and recreate `leads_business_access` with `AND NOT public.is_business_worker(business_id)` in both USING and WITH CHECK, mirroring `customers_no_workers` at line 76 of the worker-isolation migration. Confirm against the live DB first — migration tracking drifts, so check `pg_policies` on `public.leads` before assuming the remote matches the files.
- ☐ **[MEDIUM]** Adding an existing lead renders a phantom duplicate with a duplicate React key
- *Function · minutes* — `src/components/leads/leads-client.tsx:151` is `setLeads((prev) => [lead, ...prev]);` with no id check. `createLead` (`src/lib/actions/leads.ts:75-89`) calls `rpc("upsert_lead", ...).single()` and returns that row. The RPC (`supabase/migrations/20260503000002_lead_dedup.sql`, header comment at ~line 152 and the `IF v_existing.id IS NULL THEN` branch that follows) selects an existing row by `identity_key` and merges into it, returning `public.leads` — i.e. the pre-existing row. Both consumers key on `lead.id`: `leads-board.tsx:96` `key={lead.id}` and `leads-list.tsx:95` `key={lead.id}`, and the board additionally sets `layoutId={lead-${lead.id}}` (`leads-board.tsx:42`), which `framer-motion` also assumes is unique.
- **Impact.** Manually adding someone who already called shows two identical cards in the same column and two identical rows in the list, with a React duplicate-key warning and a duplicated `framer layoutId`. It clears on refresh — no data is corrupted, the DB correctly merged — but the UI contradicts the dedup system it sits on top of.
- **Fix.** In `handleAdd`, upsert into local state: `setLeads(prev => prev.some(l => l.id === lead.id) ? prev.map(l => l.id === lead.id ? lead : l) : [lead, ...prev])`, and ideally have `createLead` surface a merged flag from the RPC so the toast can say 'Merged into an existing lead'.
- ☐ **[MEDIUM]** An unrecognised `?view=` value renders a completely blank workspace
- *Function · minutes* — `src/components/leads/leads-client.tsx:85` — `const view = (params.get("view") as View) ?? "board";` `??` only fires on null, so any present-but-unknown string passes through the cast. Lines 329-331 render exactly three strict-equality branches with no else. The switcher at line 285 computes `active = view === v.id`, so nothing is highlighted either.
- **Impact.** `/leads?view=kanban` or any typo'd or stale shared link renders header, four KPI tiles, an unhighlighted switcher and empty space where the pipeline should be — no error, no empty state. Reads as a broken app rather than a bad URL.
- **Fix.** `const raw = params.get("view"); const view: View = VIEWS.some(v => v.id === raw) ? (raw as View) : "board";` — `VIEWS` is already declared at line 51.
- ☐ **[MEDIUM]** Every board card sets `touch-none`, so the board is nearly unscrollable on a phone
- *UI · hours* — `src/components/leads/leads-board.tsx:47` — the `DraggableCard` `motion.div` has `className="cursor-grab active:cursor-grabbing touch-none"` and spreads `dnd-kit {...listeners} {...attributes}` on lines 48-49. Sensors are `PointerSensor` only, `activationConstraint: { distance: 6 }` (line 132). Columns are `w-[280px] shrink-0` below xl (line 76) inside `flex gap-4 overflow-x-auto` (line 156), so on a phone the viewport is almost entirely card surface.
- **Impact.** `touch-action:none` hands every touch on a card to JS, so a vertical swipe starting on a card neither scrolls the page nor scrolls the column — the user must find the 8px gaps between cards. Board is the default view, so it is the first thing a phone user hits.
- **Fix.** Add a `TouchSensor` with `activationConstraint: { delay: 200, tolerance: 6 }` alongside `PointerSensor`, and move `touch-none` onto a dedicated drag handle instead of the whole card (`dnd-kit`'s documented pattern for scrollable touch lists).
- ☐ **[MEDIUM]** The pipeline carries no dollar value, no owner and no follow-up date

- *Missing · days* — The Lead interface (src/types/database.ts:827-855) lists id, business_id, user_id, name, email, phone, suburb, address, service, property_type, timing, notes, status, source, utm_*, customer_id, quote_id, sources, source_refs, last_seen_at, identity_key, created_at, updated_at — no value, assigned_to, follow_up_at or lost_reason. Column headers render `{leads.length}` alone (src/components/leads/leads-board.tsx:81-83) and STAGES (src/components/leads/lead-shared.ts:23-29) carries only colours and labels.
- **Impact.** The board shows '7' above Quoted, never '\$84k'. There is no field to record who is chasing a lead or when to chase it, so New leads rot silently — the pain the calendar's own doc-comment names (leads-calendar.tsx:6-8) without giving anywhere to act on it. Value-weighted pipeline is table stakes in ServiceM8/Jobber/Tradify.
- **Fix.** One migration adding `value numeric`, `follow_up_at date`, `lost_reason text`; sum value into each column header; red dot on leads past follow_up_at. Assignment can wait. Mirror the perf index set and add the columns to src/types/database.ts and the MCP lead tools.
- ☐ **[MEDIUM]** The entire leads feature has zero tests, including the pure helpers that are trivial to cover
 - *Tests · hours* — `find src -name "*.test.ts"` returns 14 files, all under src/lib/{assistant,booking,content,mcp}/tests plus src/lib/tests/pg-filter.test.ts. Nothing under src/components/leads/ and nothing covering src/lib/actions/leads.ts. The three named helpers exist and are currently correct: ymd() at src/components/leads/lead-shared.ts:69-71 builds the local Y-M-D by hand with an explicit comment about not using toISOString; monthGrid() at leads-calendar.tsx:30-39 does the Monday-first `(first.getDay() + 6) % 7` offset and returns 42 cells; the pager clamp is leads-list.tsx:49-51.
 - **Impact.** The redesign's most fragile logic — a deliberate anti-UTC date helper, a weekday offset, and a page clamp — has no regression net. A future refactor that 'simplifies' ymd() to toISOString() silently moves every Sydney lead created after ~10am onto the next day's calendar cell, and nothing fails.
 - **Fix.** src/components/leads/tests/lead-shared.test.ts: assert ymd() returns the local day for a late-evening +11:00 timestamp; assert monthGrid(2026, 6) starts Mon 29 June and yields 42 cells; assert STAGE_BY_STATUS covers every LeadStatus. Add a second test for the list clamp.
- ☐ **[LOW]** Server-side lead changes never reach the screen — useState(initial) with no resync
 - *Function · hours* — src/components/leads/leads-client.tsx:73 — `const [leads, setLeads] = useState(initial);` and there is no useEffect anywhere in the file syncing on the prop (I read all 462 lines). Every mutation in src/lib/actions/leads.ts does call revalidatePath("/leads") — lines 102, 119, 135, 243.
 - **Impact.** Once client state diverges from the server it stays diverged until a hard reload. The clearest live case is the merge bug above: the RSC payload after revalidation carries the correct single row, and the client throws it away, keeping the phantom on screen.
 - **Fix.** `useEffect(() => setLeads(initial), [initial])`, or move to useOptimistic over the server prop so the RSC payload stays authoritative while drag/move stays instant.
- ☐ **[LOW]** getLeads is an unbounded select("*") — the whole lead table ships to the browser on every visit
 - *Perf · hours* — src/lib/actions/leads.ts:22-25 — `tbl(supabase, "leads").select("*").eq("business_id", businessId).order("created_at", { ascending: false })`, no .limit(). Contrast mobile/app/leads/index.tsx:85-88, which selects 9 named columns with .limit(100). leads-list.tsx:5-9 explicitly builds on the unbounded load ('the page already loads every lead'). The Lead type (src/types/database.ts:827-855) includes notes, source_refs (append-only JSONB audit) and three utm columns that no view renders.
 - **Impact.** Payload grows linearly with lead count and includes an append-only JSONB audit array nobody displays. In-browser search, paging and calendar bucketing are all O(n) over it.
 - **Fix.** Slim to the ~14 columns the three views read and add .limit(500) — but ship the limit together with a 'showing your most recent 500' affordance, since all three views assume the full set is in memory.

- ☐ [LOW] Searching changes the views but not the KPIs or the header count
 - *UX · minutes* — `src/components/leads/leads-client.tsx:98-107` computes stats from `leads` (unfiltered); line 245 renders the subtitle from stats; lines 261-279 render the four StatTiles from stats; lines 329-331 pass `filtered` to all three views. `leads-list.tsx:57-62` also builds its tab counts from the incoming (already-filtered) `leads` prop, and the only visible count is the pager at `leads-list.tsx:150`.
 - **Impact.** Typing a suburb drops the board from 40 cards to 3 while the header still reads '40 total' and the Total leads tile still says 40. Board and Calendar give no signal that a filter is active.
 - **Fix.** When `search.trim()` is non-empty, render a dismissible '3 of 40 match "x"' chip in the view-switcher row (line 282) so it shows in all three views.

Tier D — secondary

Batch 2.14 — Online booking (public widget, availability engine, appointment management)

Health: rough · **Verdict:** simplify · **Findings:** 0 blocker · 5 high · 4 medium · 1 low **Estimated:** ~1.5 days (5 scheduled, 5 deferred)

What it does. A business configures "booking forms" (each with its own public slug, branding, rules and a subset of services/workers) under Settings → Booking. Services are `appointment_types` (duration + buffer), workers are `booking_resources` optionally linked to a `member_profiles` row, and each resource has weekly working hours plus date-level exceptions. A customer visits `/book/{slug}` (or an embed iframe), picks a service, optionally a worker, then a time. The slot list comes from `src/lib/booking/availability.ts`, which subtracts existing appointments, live 10-minute holds and the linked worker's timed work orders (via the `booking_busy_intervals` SECURITY DEFINER function), applies buffers, min-lead and max-advance, and returns UTC instants rendered in the business timezone. Picking a time creates a soft hold; confirming inserts an `appointments` row guarded by a GiST exclusion constraint, optionally creates a Lead and a Work Order, sends confirmation email/SMS, and fires webhooks. The customer gets a `manage_token` link at `/booking/{token}` to reschedule or cancel themselves. A 15-minute cron expires holds and sends reminder emails. The owner sees bookings at `/bookings` and can mark them done / no-show / cancelled — but cannot create or reschedule a booking from the dashboard at all.

Why this rating. The customer-facing widget and the timezone/DST layer are genuinely well built — the widget has loading, empty, error and skeleton states, hold-locking, idempotency and a honeypot, and `time.ts` is correct and tested. But the write endpoints trust the client for the one invariant that matters: neither `POST /bookings` nor `POST /holds` nor the reschedule `PATCH` ever calls the availability engine, so the only thing standing between a business and a 3am Sunday appointment is the customer using the widget as intended. On top of that, one un-confirmed trash-icon click in Settings cascades away every appointment a worker ever had, and the Work Order the booking generates is written with UTC time-of-day, so it lands on the crew's schedule on the wrong day.

If you only do three things:

1. Make the write path validate the slot. Extract one `isSlotBookable()` helper from `loadAvailabilityData` + `computeSlots` and call it in `POST /holds`, `POST /bookings` and the manage `PATCH` before every insert/update. Right now the availability engine is decoration — it runs only on the GET the widget happens to call, and every write endpoint takes the customer's word for it. Everything else on this list is cosmetic next to a booking calendar that can be filled with arbitrary times.

2. Fix the two data-destroying defaults: change `appointments.resource_id` to `ON DELETE RESTRICT` with soft-delete via the existing `active` flag, and put confirmation dialogs on the four unguarded trash icons in `booking-admin-client.tsx`. Then fix the UTC-vs-local Work Order write in `bookings/route.ts:165-167` so bookings land on the crew's schedule on the right day. These are three small changes that together stop the feature silently corrupting the one thing the business actually acts on — the job calendar.
3. Close the operational loop around reschedule and cancel: enforce the cancellation window on PATCH, add `notifyBookingRescheduled`, and propagate both cancel and reschedule to the linked work order. Then add `createAppointment` / `rescheduleAppointment` server actions plus MCP tools so the owner can book a phone-in customer and the assistant can do it by voice. Until then the calendar is only ever a partial view and the tradie can't trust it.

Scheduled

- ☐ **[HIGH]** Booking confirmation never validates the slot — any time can be booked by POSTing directly
 - *Security · hours* — `src/app/api/public/v1/biz/[slug]/bookings/route.ts:76-87` reads `type/resource_id/start` from the body, selects only `id, duration_minutes, active`, computes end from the duration and inserts at line 90 — no availability call. `grep` across `src/` shows `getAvailability` has exactly one caller: `src/app/api/public/v1/biz/[slug]/availability/route.ts:35`. `holds/route.ts:51-59` re-checks only `booking_busy_intervals` overlap; it selects `buffer_minutes` at line 39 and never references it again, and never consults `form.appointment_type_ids/resource_ids` (`loadAvailabilityData` does that at `availability.ts:40-50`). The comment at `availability.ts:213-214` describes a re-validation helper, but the function below it (`utcToLocalLabel`, line 215) only formats a label string.
 - **Impact.** A crafted POST can create an appointment outside working hours, on a blackout day, in the past, or beyond `max_advance_days`. The owner finds a bogus job on the calendar. Bots hitting the CORS-open endpoint could spam the diary.
 - **Fix.** Extract `isSlotBookable(sb, form, typeId, resourceId, start)` reusing `loadAvailabilityData + computeSlots` for the single day; call it before insert in `holds/route.ts` and `bookings/route.ts` and in the manage PATCH. Return 409 when the start is not in the computed set. Apply the form's `appointment_type_ids/resource_ids` restriction in the holds route too.
- ☐ **[HIGH]** Deleting a booking resource cascade-deletes every appointment ever booked with that worker, with no confirmation
 - *Function · hours* — `supabase/migrations/20260526000001_online_booking.sql:191` — `resource_id UUID NOT NULL REFERENCES public.booking_resources(id) ON DELETE CASCADE` on appointments. `booking_audit_log.appointment_id` is `ON DELETE SET NULL` (line 260). `src/components/booking/booking-admin-client.tsx:392` is a bare `<Button size="icon" variant="ghost" onClick={onDelete}><Trash2/></Button>`; line 156 wires it to `deleteResource(r.id)`. `src/lib/actions/booking.ts:243-247` deletes unconditionally. `grep` for `confirm()` across `booking-admin-client.tsx` returns no confirmation dialog on any delete path (lines 118, 136, 156).
 - **Impact.** An owner tidying up after a worker leaves destroys that worker's entire appointment history — past and future — with no confirmation, no undo, and no surviving audit link.
 - **Fix.** (1) Change the FK to `ON DELETE RESTRICT` and make `deleteResource` soft-delete via `active = false` when appointments exist — the `active` column already exists for this. (2) Put all four delete buttons behind a confirmation dialog that names what is lost.
- ☐ **[HIGH]** Work Orders created from a booking are written with UTC time-of-day, so they land on the wrong day
 - *Function · minutes* — `src/app/api/public/v1/biz/[slug]/bookings/route.ts:165-167` — `scheduled_date: start.toISOString().slice(0,10)`, `start_time: start.toISOString().slice(11,16)`, `end_time: end.toISOString().slice(11,16)`. Migration `20260527000001 (booking_busy_intervals)` reads those

columns as `(wo.scheduled_date + wo.start_time) AT TIME ZONE COALESCE(bs.timezone, 'UTC')`, i.e. as naive local wall-clock in the business timezone.

- **Impact.** For a Sydney business a 09:00 booking becomes a job dated the previous day at 22:00/23:00. The crew's schedule is wrong. The bad work order then feeds back into `booking_busy_intervals`, blocking a 22:00 window nobody wants while the real 09:00 slot still looks free. A booking crossing UTC midnight yields `end_time < start_time` and a negative busy interval.
- **Fix.** Build `scheduled_date/start_time/end_time` with `utcToZonedParts(start, settings.timezone)` from `src/lib/booking/time.ts`, matching what the busy-intervals SQL expects.
- ☐ **[HIGH]** Customer reschedule bypasses the cancellation window, accepts any `resource_id`, and notifies nobody
 - *Function · hours* — `src/app/api/public/v1/bookings/[manage_token]/route.ts:65-117 (PATCH) vs 119-148 (DELETE)`: the `cancellation_window_hours` check exists only in DELETE at lines 130-133. Line 84: `const resourceId = String(body.resource_id ?? appt.resource_id)` passed straight into the update at line 102, with no `business_id`/active verification. Lines 109-116 write an audit row and fire `fireBookingWebhook` but call no `sendEmail`. `src/lib/booking/notify.ts` exports `notifyBookingCreated` (102), `notifyBookingReminder` (145) and `notifyBookingCancelled` (168) — no reschedule function.
 - **Impact.** A customer blocked from cancelling inside the notice window can reschedule to next month instead — same outcome, window defeated. They can move the job to 4am (no slot validation) or onto a resource id that isn't theirs, orphaning it off any real worker's calendar. And because no email fires, the tradie is never told the job moved.
 - **Fix.** In PATCH: enforce the same window check as DELETE; validate `resource_id` with `.eq("business_id", appt.business_id).eq("active", true)`; run the slot validation from finding 1; add `notifyBookingRescheduled` covering customer and team.
- ☐ **[HIGH]** Cancelling or rescheduling a booking leaves the generated Work Order untouched
 - *Function · hours* — `src/app/api/public/v1/biz/[slug]/bookings/route.ts:171-175` creates the work order and stores `work_order_id` on the appointment. The manage DELETE at `route.ts:135-137` updates only `status + cancelled_at`; PATCH at 100-103 updates only `starts_at/ends_at/resource_id/status`. `src/lib/actions/booking.ts:351-358 (setAppointmentStatus)` patches `status (+ cancelled_at)` and nothing else.
 - **Impact.** With `create_work_order` on, a cancelled booking leaves a live job on the crew's schedule and the worker turns up; a rescheduled booking leaves the job at the old time. The stale job also stays in `booking_busy_intervals` (which excludes only `status 'cancelled'`), so it keeps blocking the slot that was just freed.
 - **Fix.** In the manage DELETE, set the linked work order to cancelled when `appt.work_order_id` is present. In PATCH, update its `scheduled_date/start_time/end_time` using the timezone-correct conversion from finding 3. Mirror both in `setAppointmentStatus`.

Deferred (5) — medium/low on a feature marked *simplify*

- **[MEDIUM]** max_per_day caps how many slots are shown, not how many bookings are accepted — and the count it compares against is always zero — src/lib/booking/availability.ts:109 `const perDay = args.perDayCount ?? new Map()`; getAvailability calls computeSlots at lines 201-203 with no perDayCount, and grep confirms the only other callers are the tests. The line-118 gate and the dayBudget at 119-121/153 therefore only limit emitted slots. src/lib/booking/**tests**/availability.test.ts:78-81 asserts `slots.length === 3` with max_per_day: 3 and an empty busy map. bookings/route.ts never reads max_per_day.
- **[MEDIUM]** /bookings hand-rolls status pills and tabs, has no search, and cancels without confirmation — src/components/booking/bookings-page-client.tsx:19-23 is a local PILL map of raw Tailwind (bg-emerald-100 text-emerald-700 ...) rather than .ch-pill or KireiPill, light-mode-only. Lines 82-89 hand-roll the tab row. Line 92 is a bare div where EmptyState exists. Line 142 calls changeStatus(id, "cancelled") straight from a Button with no confirm. No search input or date filter exists in the file; src/app/(dashboard)/bookings/page.tsx passes listAppointments({ limit: 300 }) and all filtering is client-side. src/lib/actions/booking.ts:357 revalidates "/settings/booking".
- **[MEDIUM]** Customer-supplied booking fields are interpolated unescaped into the team notification email — src/lib/booking/notify.ts:132-138 builds the team email with \${appt.customer_name}, \${appt.customer_phone}, \${appt.customer_email}, \${appt.customer_address} and \${appt.notes} interpolated into HTML with no escaping; line 185 does the same for \${appt.customer_name} in the cancellation notice. Those values come from the unauthenticated public POST body with only .trim() applied (bookings/route.ts:53-57 and :99).
- **[MEDIUM]** The owner cannot create or reschedule a booking from the dashboard, and the widget only ever looks 21 days ahead — src/lib/actions/booking.ts has createAppointmentType (line 165) but no createAppointment and no reschedule action. The appointment-related MCP tools in src/lib/mcp/register-tools.ts are list_appointment_types (1806), create_appointment_type (1815), list_appointments (1870), set_appointment_status (1883), update_appointment_type (1894) and delete_appointment_type (1910) — none creates or moves an appointment. The only owner-side reschedule path is the customer manage-token link at src/components/booking/bookings-page-client.tsx:143. Widget window hardcoded at src/app/book/[slug]/booking-widget.tsx:107 (`from=today&to=addDays(today, 21)`); empty-state copy at line 289.
- **[LOW]** "Today" tab and today-count use the viewer's browser timezone, not the business timezone — src/components/booking/bookings-page-client.tsx:40 and :50 both use `new Date(a.starts_at).toDateString() === new Date().toDateString()`, which resolves in the browser locale. The fmt helper at lines 33-35 and the detail row at line 120 correctly pass `timeZone: timezone`.

Batch 2.15 — Contracts & e-signature

Health: rough · **Verdict:** simplify · **Findings:** 0 blocker · 3 high · 4 medium · 3 low **Estimated:** ~0.8 days (3 scheduled, 7 deferred)

What it does. A business creates a contract for one customer, either by pasting raw HTML into a textarea (with {{merge_field}} tokens for customer/business/date) or by uploading a source PDF. "Send to sign" mints/reuses a customer portal token, emails the customer a link to /portal/[token]/contract/[id], and flips the contract to sent; opening it marks it viewed. On the portal the customer types or draws a signature on a canvas, ticks a consent box, and POSTs to the sign route, which captures name + method + IP + user-agent into an **audit** JSONB column, stores the drawn PNG, generates a signed PDF with a signature-certificate block via @react-pdf, and flips status to signed. The dashboard offers edit (draft only), copy link, resend, void and delete. It is a **defaultEnabled: true** plugin (src/lib/plugins/registry.ts:61), so it is live for every business today.

Why this rating. The rich-text happy path works end to end, but there is a live cross-customer document leak in both PDF routes, the uploaded-PDF mode produces a "signed copy" containing none of the contract, the signed PDF is a lossy plain-text re-render of what the customer actually saw with no integrity hash, and there is zero test coverage. It is a legal instrument shipped enabled-by-default with defects that would embarrass the owner in a dispute.

If you only do three things:

1. Close the cross-customer leak in `/api/pdf/contract/[id]` and `/source` by scoping the token path to `link.customer_id` — a two-line fix in each route that stops one business's customers reading each other's signed agreements today.
2. Make the signed artifact defensible: store a sha256 + full HTML snapshot of exactly what was rendered into `audit.signed`, print the hash on the certificate, and render the PDF from that same HTML instead of tag-stripped text. Same change should sanitize the HTML on render, closing the portal XSS.
3. Delete `kind:'pdf'` entirely (upload tab, `uploadContractPdf`, the `/source` route, the empty-certificate branch) and put the saved effort into the rich-text half: a real editor instead of a monospace HTML textarea, a decline path, and search/status tabs on the list rebuilt on the leads workspace pattern.

Scheduled

- ☐ **[HIGH]** Portal-token PDF routes scope to business, not customer — any customer can download any other customer's contract
 - *Security · minutes* — `src/app/api/pdf/contract/[id]/route.ts` lines 17-18 select only `business_id`, `expires_at`, `revoked_at` from `customer_portal_tokens` — `customer_id` is NOT selected (the finding said it was selected but unused; it is not selected at all, a minor evidence inaccuracy that does not change the defect). Line 21 sets `businessId = link.business_id`, line 29-30 queries contracts with `.eq("id", id).eq("business_id", businessId)` only. `source/route.ts` lines 17-18 and 29-30 are byte-identical in structure. The portal page `src/app/portal/[token]/contract/[id]/page.tsx:21` DOES select `customer_id` and line 26 chains `.eq("customer_id", link.customer_id)`; the sign route `src/app/api/portal/[token]/contract/[id]/sign/route.ts:28` does the same. So two of four token consumers scope correctly and two do not. Tokens are per-customer and long-lived: `src/lib/actions/contracts.ts:197-200` inserts with `expires_at = now + 90 days`.
 - **Impact.** A customer holding any live portal token for business X can GET `/api/pdf/contract/?token=` and receive another customer of X's signed PDF — which per `src/components/contracts/contract-pdf-document.tsx:69-77` embeds the signer name, signing timestamp, IP address and full user-agent, plus the drawn signature image (line 61). Contracts is `defaultEnabled` (`src/lib/plugins/registry.ts:61`) so this is live for every business.
 - **Fix.** In both routes change line 18 to select `business_id, customer_id, expires_at, revoked_at`, carry `customer_id` alongside `businessId`, and on the token branch add `.eq("customer_id", link.customer_id)` to the contracts query at line 29-30 — mirroring portal page.tsx:26 and sign/route.ts:28. Leave the `authed-user` branch unchanged.
- ☐ **[HIGH]** Uploaded-PDF contracts produce a "signed copy" that contains none of the contract
 - *Function · hours* — `src/app/api/portal/[token]/contract/[id]/sign/route.ts:45-48`: `let paragraphs: string[] = []` then `if (c.kind === "pdf") { attachmentNote = "This certifies electronic signature of the attached contract document (provided separately)." }` — the else branch (49-55) is the only path that fills paragraphs. `src/components/contracts/contract-pdf-document.tsx:52-55` renders the `attachmentNote` and then maps an empty paragraphs array, so the page is title + rule + note + signature block + certificate. Nothing reads `source_path` during signing. The portal offers the file unconditionally for signed contracts at `src/app/portal/[token]/contract/[id]/page.tsx:104-107` ("Download signed copy"), regardless of `contract.kind`.
 - **Impact.** A business uploads a multi-page agreement (up to 15 MB, `src/lib/actions/contracts.ts:151`), the customer signs, and the permanent artifact both sides download is a one-page certificate asserting that something was signed elsewhere. The signature is never applied to the source PDF and the two files share only a UUID in the certificate's Reference row. In a dispute this evidences the act of signing but not the terms.
 - **Fix.** Merge at sign time: pdf-lib load the object at `c.source_path`, copy its pages into a new doc, append the @react-pdf certificate page, store that as `signed_path`. Alternatively drop the mode entirely — remove the Upload PDF tab (`src/components/contracts/contracts-client.tsx:144`), `uploadContractPdf`, and the

/source route.

- ☐ **[HIGH]** Contract HTML and merge values are rendered unsanitized on the public portal
 - *Security · hours* — `src/components/contracts/contracts-client.tsx:157-158` — a plain Textarea whose placeholder reads "Write the contract. Basic HTML allowed." `src/lib/emails/templates.ts:151-155` `renderTemplateVars` replaces `{{key}}` with `String(v)` and no escaping. `src/lib/contracts/render.ts:15-29` `fillMergeFields` is a thin wrapper over it. The result is injected raw at `src/app/portal/[token]/contract/[id]/page.tsx:93` `dangerouslySetInnerHTML={{ __html: bodyHtml }}` on the unauthenticated portal, and at `src/components/contracts/contract-detail-client.tsx:142-143` in the dashboard. `src/lib/mcp/tools/plugin-form-tools.ts:410-416` `create_contract` takes `content_html: z.string().min(1)` and inserts it verbatim; `update_contract` (421-429) likewise. No sanitizer import exists anywhere in the contracts path.
 - **Impact.** Script injected into `content_html` — by an editor-role member, or by the assistant transcribing attacker-controlled lead text — executes in the customer's browser on the signing page, the one screen where they type their legal name and draw a signature that gets POSTed as a data URL. Separately, any customer whose name or address contains `<` silently corrupts the rendered contract body for every merge-field contract.
 - **Fix.** HTML-escape merge values before substitution in `src/lib/contracts/render.ts` `fillMergeFields`, and run the filled string through an allow-list sanitizer (`p/br/strong/em/ul/ol/li/h1-h4/table/tr/td/a`) in the same file so both the portal and the dashboard inherit it.

Deferred (7) — medium/low on a feature marked *simplify*

- **[MEDIUM]** No integrity hash, and the signed PDF is a lossy plain-text re-render of what the customer saw — `src/lib/contracts/render.ts:36-47` — `.replace(/<[^>+>/g, "")` at line 41 removes every remaining tag; the break-inducing list at line 39 covers `br`, `/p`, `/div`, `/li`, `/h1-6` and nothing else. That array is the PDF body (`contract-pdf-document.tsx:53-55`) while the portal shows the styled original (`portal page.tsx:93`). `sign/route.ts:88` writes `audit.signed = { at, ip, user_agent, method, signature_path, consent }` — no hash, no content snapshot. `content_html` is a mutable TEXT column (`supabase/migrations/20260623100000_contracts.sql:21`) and the `contracts_write` policy at lines 73-84 is **FOR ALL** for owner/admin/editor with no status or column restriction, so the draft-only guard at `src/lib/actions/contracts.ts:114-116` (and its MCP twin at `plugin-form-tools.ts:428-429`) is application-level only.
- **[MEDIUM]** Customer has no way to decline — the status exists but nothing can set it — `supabase/migrations/20260623100000_contracts.sql:23-24` `CHECK (status IN ('draft','sent','viewed','signed','declined','voided'))`. `src/components/contracts/contracts-client.tsx:28` and `src/app/portal/[token]/contract/[id]/page.tsx:132` both style 'declined'. The portal's action area (`page.tsx:111-119`) renders only when the contract is neither signed nor voided, and `src/components/customer-portal/sign-contract.tsx` has one button (line 105) and one fetch target (line 51, the sign route). No decline route exists on disk.
- **[MEDIUM]** Drawn signature is invisible in dark mode — `src/components/customer-portal/sign-contract.tsx:32` sets `ctx.strokeStyle = "#0f172a"` inside the pointermove handler; line 91-93 the canvas carries `bg-background` with no explicit fill. Root layout enables system dark mode app-wide (`layout.tsx:57-61`) and the portal page itself ships dark: variants (`portal page.tsx:100`, 129-133), confirming dark rendering is expected there.
- **[MEDIUM]** Zero test coverage on the only feature that produces a legal artifact — Full test set under `src/`: `tests/pg-filter`, `assistant/tests/{models,scopes,undo}`, `booking/tests/{availability,booking-db,time}`, `content/tests/{live-agents,pipeline,prompts,schedule}`, `mcp/tests/{collect,invoke,live-api}`. No contracts directory, no reference to contracts in any of them.
- **[LOW]** List page hand-rolls a table with no search, filter, status tabs or pagination, and silently truncates at 200 — `src/components/contracts/contracts-client.tsx:99-124` is a raw

with ad-hoc Tailwind (`bg-muted/40`, `border-t border-border/50`) rather than `.ch-table-wrap/.ch-table`, and a local `STATUS_TONES` map at lines 23-30 rather than `.ch-pill`. No search input, filter or Tabs component appears outside the create dialog (the only is the `rich_text/pdf` mode switch at line 141). `src/lib/actions/contracts.ts:66` ends `.order("created_at", { ascending: false }).limit(200)` and the client receives the array as

initial with no count or affordance. Credit where due: line 113 and 116 do use break-words, matching the owner's stated preference.

- **[LOW]** MCP tool description tells the assistant signing requires Dropbox Sign — `src/lib/mcp/tools/plugin-form-tools.ts:409` — `create_contract`'s description ends "Send it for signature separately once Dropbox Sign is configured." `src/lib/actions/contracts.ts:178-181` `isSigningEnabled` is documented "Native in-app signing is always available — no third-party setup required" and returns true unconditionally. `supabase/migrations/20260623100000_contracts.sql:1` ("signed via Dropbox Sign, one platform account") and line 27-28 ("e-signature linkage (Dropbox Sign)", provider comment "'dropbox_sign'") are stale — the sign route writes provider: 'native' (`sign/route.ts:90`).
- **[LOW]** Contract pages use raw `supabase.auth.getUser()` and re-resolve a role the layout already has — `src/app/(dashboard)/contracts/page.tsx:10` `const { data: { user } } = await supabase.auth.getUser();` — the raw `GoTrue` call, not `getUser()` from `src/lib/auth`. Line 13 awaits `getActiveBizId` and throws the value away. `src/app/(dashboard)/contracts/[id]/page.tsx:11` does the same raw call, then lines 17-24 query `businesses.user_id` and, on a miss, `business_members.role` — re-deriving a role the dashboard layout already resolved. The actions these pages call (`getContracts`, `getContract`, `renderContractHtml`) each independently use the cached `getUser()` from `@/lib/auth`, so the raw call is purely additive.

Batch 2.16 — Public form builder & lead capture

Health: rough · **Verdict:** keep-secondary · **Findings:** 0 blocker · 3 high · 6 medium · 1 low **Estimated:** ~3.6 days (10 scheduled)

What it does. A per-business plugin (flag `form_builder_settings.enabled`, togglable from `/agents` or the enable card at `/forms`) that lets a business drag-and-drop a public lead-capture form out of ~20 field types, reusing the onboarding field engine (`OnboardingField[]` stored in `public_forms.schema`, minus the `secure` type). Each form gets a globally-unique slug, published at a hosted page `/f/[slug]` and an iframe embed `/embed/[slug]`, both rendered by the same `PublicFormShell` via the service-role admin client. Anonymous visitors submit to `POST /api/f/[slug]/submit`, which is guarded by a honeypot field and an in-memory per-IP rate limit (8/min), validates required + format rules server-side, writes a `public_form_submissions` row, optionally calls the `upsert_lead` RPC to create a deduped lead (source `form`), and emails a plain HTML digest to any addresses in `settings.notify_emails`. File/image fields upload separately to a private `public-form-uploads` bucket (10MB, 20/min/IP). The owner reads submissions in a third tab of the builder, and the whole surface is mirrored into MCP (`list/get/create/update/delete_public_form`, `list_form_submissions`).

Why this rating. The happy path works: a published form renders, validates, stores a submission and creates a lead. But three defects break the feature's actual purpose rather than its mechanics — the form's answers never reach the lead the salesperson works from, uploaded files are unretrievable in the dashboard, and editing a live form silently blanks the answers of every historical submission. Each of those is invisible to the owner until a real customer has already been lost.

If you only do three things:

1. Make the lead actually contain the enquiry. Map every answer into the `upsert_lead` call as structured notes plus an address slot (`src/app/api/f/[slug]/submit/route.ts:70-81`), and link the submission from the lead detail page. Today the pipeline gets a name and the words 'Submitted via form' — the feature captures leads but not the lead information, which makes it decorative.
2. Stop losing submitted data. Add `schema_snapshot` to `public_form_submissions` and render from it (mirroring the onboarding sibling), and wire the already-written `getFormUploadUrl` into the Submissions tab so uploaded photos are openable. These are two small changes that together turn the submissions store from write-only into a record the business can actually rely on.

3. Harden the public endpoint: escape the answer values interpolated into the notification email (route.ts:119, minutes of work, currently a phishing vector aimed at the owner), and wire the existing verifyCaptcha from src/lib/booking/public.ts into the submit route so the in-memory rate limiter is not the only thing standing between a bot and the sales pipeline.

Scheduled

- ☐ **[HIGH]** Form answers never reach the lead — the pipeline gets a name and nothing else
 - *Function · hours* — src/app/api/f/[slug]/submit/route.ts:70-81 — the upsert_lead RPC is called with p_address: null, p_suburb: null, p_service: null, p_property_type: null, p_timing: null and p_notes: Submitted via form "\${form.name}" . Only three values are extracted from the visitor's answers, at lines 59-61 (name/email/phone), resolved via settings.lead_map or type-detection at lines 56-58. The full answer set is written only to public_form_submissions.answers at lines 107-109. Grep confirms getFormSubmissions is called from exactly one place — src/app/(dashboard)/forms/[id]/page.tsx:19 — and rendered only in the Submissions tab at src/components/forms/forms-builder-client.tsx:362-381. Nothing on the lead links back to the submission.
 - **Impact.** A homeowner fills in address, roof type, urgency and a message; the lead in the pipeline carries only name/email/phone plus the form name as a note. The salesperson must open /forms/[id] → Submissions and match by timestamp, or phone and re-ask. For the one roofing business that actually depends on Kirei, this defeats the point of lead capture.
 - **Fix.** In the submit route, build p_notes from the form name plus a rendered key/value list of every non-empty, non-display answer, and route an address-typed answer to p_address. Add address/service slots to the existing lead_map UI at forms-builder-client.tsx:328-341, falling back to type-detection the way email/phone already do at route.ts:57-58.
- ☐ **[HIGH]** Uploaded files and images are collected but cannot be opened by the business
 - *Function · hours* — src/lib/actions/forms.ts:175-185 exports getFormUploadUrl, which creates a 1-hour signed URL against the private public-form-uploads bucket (declared at supabase/migrations/20260624140000_form_builder.sql:93). A repo-wide grep for getFormUploadUrl across src/ returns exactly one hit — its own definition. The Submissions tab renders every answer through fmtAnswer at src/components/forms/forms-builder-client.tsx:390-398, whose upload branch (line 394) returns the plain string  \${v.name} with no anchor and no handler. The public fill page previews uploads correctly via /api/f/[slug]/file at src/components/forms/public-form-fill.tsx:178-187, and the onboarding sibling ships src/components/onboarding/answer-image-thumb.tsx for exactly this job.
 - **Impact.** 'Upload a photo of the damage' is the single highest-value field a trades business can ask for. The customer uploads it successfully; the owner sees dead grey text and can only reach the file by hand-crafting a signed URL against Supabase Storage.
 - **Fix.** Wire getFormUploadUrl into fmtAnswer's upload branch — a button that fetches the signed URL and opens it, plus an inline thumbnail for image-typed fields. Reuse src/components/onboarding/answer-image-thumb.tsx.
- ☐ **[HIGH]** Anonymous visitors can inject arbitrary HTML into the business's notification email
 - *Security · minutes* — src/app/api/f/[slug]/submit/route.ts:119 string-interpolates both the field label and the raw answer straight into HTML: <tr><td ...>\${f.label}</td><td ...>\${fmt((answers as any)[f.id])}</td></tr> , joined into html at line 120 and passed to sendEmail at line 121-126. fmt() at lines 134-140 is String(v) with no escaping. The request body is parsed at line 16 and the answers object flows to line 119 through missingRequiredFields/invalidAnswerFields (src/lib/onboarding/answers.ts) and processAnswersForStorage (answers.ts:13-27) — none of which sanitise or escape.
 - **Impact.** Anyone who can load the public form types markup into a long_text field and the business owner receives it rendered, in an email sent from their own Kirei sender address (buildBusinessFrom, route.ts:123). Attacker-controlled links and display:none rules hiding the real content make this a credible

phishing vector. Only fires when notify_emails is configured (route.ts:113-114).

- **Fix.** Add an escapeHtml helper (& < > " ') and apply it to both f.label and the fmt(...) output in the row template at route.ts:119.
- ☐ **[MEDIUM]** Editing a published form silently blanks the answers of every past submission
 - *Function · hours* — supabase/migrations/20260624140000_form_builder.sql:35-43 — public_form_submissions is (id, business_id, form_id, answers, lead_id, meta, created_at). No schema_snapshot column. src/components/forms/forms-builder-client.tsx:373-377 iterates (form.schema ?? []) — the CURRENT schema — and reads s.answers?.[f.id] . The onboarding sibling does the opposite deliberately: the column exists at supabase/migrations/20260624100000_client_onboarding.sql:57 with the comment "fields as asked, so edits to the form don't orphan answers", is written at src/app/api/portal/[token]/onboarding/[id]/save/route.ts:65 and read at src/components/onboarding/response-viewer-client.tsx:35 and src/lib/actions/onboarding.ts:267. Field ids are random — newId() at forms-builder-client.tsx:73.
 - **Impact.** Delete a field or rebuild a form and every prior submission's answer to it disappears from the UI with no warning and no export path. Because ids are random, a deleted-and-re-added field is a permanent orphan.
 - **Fix.** Add schema_snapshot JSONB NOT NULL DEFAULT '[]' to public_form_submissions, populate it in the submit route alongside answers (route.ts:107-109), render from s.schema_snapshot ?? form.schema , and backfill existing rows. Mirror the onboarding implementation.
- ☐ **[MEDIUM]** The public lead form has the weakest spam defence of the two public form surfaces — no CAPTCHA
 - *Missing · hours* — src/app/api/f/[slug]/submit/route.ts:19 honeypot (body._hp) and :21-24 rateLimit(form:\${slug}:\${ip} , 8, 60_000) are the only guards. That limiter is defined at src/lib/booking/public.ts:56-68 as an in-memory Map , documented as "Best-effort in-memory IP rate limiter... CAPTCHA + idempotency + per-day caps are the real guards". The same module exports a working verifyCaptcha (hCaptcha + Turnstile) driven by captcha_provider/captcha_secret_key on booking_forms. public_forms has no equivalent columns (supabase/migrations/20260624140000_form_builder.sql:16-31) and the submit route never calls verifyCaptcha.
 - **Impact.** The Map resets on cold start and is per-instance, so a distributed or slow-drip bot walks through the 8/min window. Each accepted submission creates a pipeline lead and fires a Resend send. Jotform, Typeform and GHL all ship CAPTCHA on public forms.
 - **Fix.** Add captcha_provider / captcha_secret_key to public_forms.settings, surface them in the Settings tab of forms-builder-client.tsx, render the widget in public-form-fill.tsx, and call the existing verifyCaptcha before any write. Add a per-form daily cap as a backstop.
- ☐ **[MEDIUM]** A required consent tick can be submitted unchecked once the visitor unticks it
 - *Function · minutes* — src/lib/onboarding/answers.ts:139-144 — missingRequiredFields only treats an answer as missing when v == null || v === "" or an empty array; boolean false falls through and returns false (present). invalidAnswerFields skips it as well: answers.ts:87 if (v == null || v === "" || typeof v !== "string") continue; . The consent control at src/components/forms/public-form-fill.tsx:148 sets the answer to e.target.checked , so unticking stores the boolean false rather than clearing the key.
 - **Impact.** A form with a required 'I consent to be contacted' tick accepts submissions where the visitor explicitly unticked it. The owner assumes consent is on record; the stored answer says the opposite.
 - **Fix.** In missingRequiredFields (answers.ts:141) treat v === false as missing for boolean-answer field types. Shared with onboarding, so it fixes both surfaces.
- ☐ **[MEDIUM]** The builder has no unsaved-changes guard — navigating away discards the whole form

- *UX · hours* — `src/components/forms/forms-builder-client.tsx:94-98` computes `dirty` by JSON-comparing name/description/status/fields/settings against the loaded form. Its only consumer is `disabled={saving || !dirty}` on the Save button at line 154. The back link at lines 136-138 is `<Link href="/forms">` with no interception, and there is no `useEffect` registering a `beforeunload` listener in the file. All builder state is local `useState` (lines 83-90); the only persistence is the explicit `save()` at lines 116-124.
- **Impact.** Field list, every label/option/conditional rule, and all three Settings cards are local state until Save. Clicking the back arrow, a sidebar item, or the browser back button after twenty minutes of building discards everything with no prompt.
- **Fix.** Register a `beforeunload` handler while `dirty` and intercept the back link with a confirm. Better: autosave via the existing `updatePublicForm` action.
- ☐ **[MEDIUM]** Embed snippet is a fixed 700px iframe with no auto-resize
- *UX · minutes* — `src/lib/forms/embed.ts:2-5` returns a single hard-coded `<iframe src="${baseUrl}/embed/${slug}" title="Form" width="100%" height="700" style="border:0;max-width:640px" loading="lazy">`. `src/components/forms/public-form-shell.tsx:53` returns `<div className="min-h-screen bg-background">{body}</div>` for the embed branch. Neither `public-form-shell.tsx` nor `src/components/forms/public-form-fill.tsx` contains any `postMessage` or `ResizeObserver` height reporting, and `embedSnippet` ships no companion resizer script.
- **Impact.** A three-field contact form sits in 700px of whitespace; a fifteen-field quote form gets a scrollbar nested inside the host page's scrollbar. The thank-you state (`public-form-fill.tsx:71-79`) and inline validation change the content height and the frame does not follow, so the confirmation can land below the fold and the visitor resubmits.
- **Fix.** Post document height from the embed page on mount, resize and state change; ship a short listener in `embedSnippet` that sets `iframe.style.height`. Drop `min-h-screen` from the embed branch at `public-form-shell.tsx:53`.
- ☐ **[MEDIUM]** Zero tests over the only unauthenticated write path that creates business records
- *Tests · hours* — A find across `src` for test files returns only `src/lib/assistant/tests/{models,scopes,undo}.test.ts`, `src/lib/booking/tests/{availability,booking-db,time}.test.ts`, `src/lib/content/tests/{live-agents,pipeline,prompts,schedule}.test.ts`, `src/lib/mcp/tests/{collect,invoke,live-api}.test.ts` and `src/lib/tests/pg-filter.test.ts`. Nothing covers `src/lib/onboarding/answers.ts`, `src/app/api/f/{slug}/submit/route.ts`, `src/lib/forms/slug.ts`, `src/lib/forms/embed.ts`, or the `uniqueSlug` collision loop at `src/lib/actions/forms.ts:57-67`.
- **Impact.** The public submit route is an anonymous service-role write that creates leads and sends email. A regression silently drops real enquiries or admits spam, and CI catches neither.
- **Fix.** Three pure-function test files, no DB: `answers.test.ts` (required consent set to false — passes today, should fail; `show_if` visibility skipping required fields; `isValidAbn/isValidAcn` checksums; `invalidAnswerFields` against a malformed `validation.pattern`), `slug.test.ts` (`slugifyFormName` over `unicode/punctuation/empty` plus the -2/-3 loop in `uniqueSlug`), `embed.test.ts` (slug escaping in the `src` attribute). Add an escaping assertion on the notification HTML once the `escapeHtml` fix lands.
- ☐ **[LOW]** Submissions tab is an unbounded card list — no search, no filter, no pagination, no CSV export
- *UX · days* — `src/lib/actions/forms.ts:158-161` selects submissions ordered by `created_at` with `.limit(500)`. `src/components/forms/forms-builder-client.tsx:361-383` maps every returned row to a full Card with a definition list of every field; the file contains no search input, date filter, pagination control or export button. By contrast `src/components/leads/leads-client.tsx:309-316` renders a 'Search leads...' Input, and `src/components/leads/leads-list.tsx:37-54` holds filter + page state with a `pageCount` clamp for page-stranding.
- **Impact.** Once a form gets real volume the tab becomes an unsearchable wall of cards, the 501st submission is silently invisible, and there is no way to get the data out of Kirei.
- **Fix.** Reuse the leads pattern — search over answer values, a date filter, and the pagination from `leads-list.tsx` — plus an Export CSV button generating columns from `schema_snapshot` once that column exists.

Batch 2.17 — Tasks & Messages/SMS

Health: rough · **Verdict:** simplify · **Findings:** 0 blocker · 3 high · 5 medium · 1 low **Estimated:** ~1.7 days (3 scheduled, 6 deferred)

What it does. Two unrelated features share this audit. **Tasks** (`src/app/(dashboard)/tasks/page.tsx` , `src/components/tasks/kanban-board.tsx`) is a per-business kanban board with four fixed columns (todo / in_progress / in_review / done), drag-and-drop via @dnd-kit, and a click-to-edit modal covering title, description, priority, assignee, due date, free-text tags, and optional links to a work order / customer / contact. It has client-side search plus assignee/priority/tag filters, a dashboard "Your todos" widget (`src/components/tasks/tasks-widget.tsx`), and 3 MCP tools (`register-tools.ts:491-530`). It genuinely works for an owner. **Messages** (`src/components/messages/messages-client.tsx`) is a two-pane SMS inbox — conversation list plus a chat thread — that sends outbound texts through ClickSend (`src/lib/actions/sms.ts:27-53`) using an 11-character alphanumeric Sender ID derived from the business name. Despite the UI (reply bubbles, unread counts, Supabase Realtime subscriptions, an inbound webhook at `src/app/api/sms/webhook/route.ts`), the inbound half does not function: the webhook is Twilio-shaped, is not whitelisted in middleware, and an alphanumeric Sender ID cannot receive replies at all. In practice Messages is a one-way SMS blaster with an inbox-shaped UI wrapped around it.

Why this rating. Tasks is functional for an owner but silently broken for any 'editor'-role member because the RLS policy (20260428000001_tasks.sql:36-44) grants write to roles ('manager','staff') that do not exist in the app's role model, and omits 'editor' — with zero client-side error handling on top, so the failure is invisible. Messages is worse: the inbound path is three independent layers of dead (wrong provider format, unreachable route, unreceivable sender ID), so the product presents a two-way inbox that can only ever be one-way. Neither feature has a single test.

If you only do three things:

1. Decide Messages is one-way and act on it. Delete /messages, the sms_conversations/sms_messages tables, and /api/sms/webhook; keep only clicksendSend() for the booking notifications in src/lib/booking/notify.ts that actually use it. This removes three blocker-severity defects, the logged RLS hole, and ~700 lines of UI in one commit — and stops the product promising a customer inbox it cannot deliver.
2. Fix the tasks RLS role list (migration replacing ('admin','manager','staff') with ('admin','editor') on all three write policies, plus worker exclusion on select) AND add try/catch + toast to the four kanban handlers in the same PR. Either fix alone is insufficient: the RLS bug is what breaks editors, the missing error handling is what makes it invisible.
3. Add a confirmation to task delete and a test file for moveTask's renumbering loop. Both are under an hour and cover the two ways a business's task board can silently lose or scramble data.

Scheduled

- ☐ **[HIGH]** Inbound SMS webhook is unreachable — middleware 307s it to the login page
 - *Function · minutes* — `src/lib/supabase/middleware.ts:24-63` builds `isPublicRoute`; I read every clause — `/api/stripe/webhook (:59)`, `/api/cron/ (:48)`, `/api/f/ (:39)`, `/api/portal/ (:31)` are all listed, `/api/sms/webhook` is not. `:62` only exempts bearer routes (`/api/mobile/`, `/api/ai/transcribe` at `:21-22`). `:68-70` returns early for public routes; `:102-106` redirects everything else without claims to `/auth/login`. `src/app/api/sms/webhook/route.ts:5` is a bare `export async function POST` with no signature or bearer check of its own.
 - **Impact.** Inbound SMS POSTs are redirected to the HTML login page and dropped. No customer reply can ever land in the inbox. /messages (plugin registry `:77`, `defaultEnabled true`) therefore only ever shows outbound messages the business itself sent.

- **Fix.** Do not whitelist the path as-is. Treat findings 1 and 2 as one decision: either delete the route, or rewrite it (ClickSend inbound JSON, real per-business number, signature verification) and whitelist it in the same change.
- ☐ **[HIGH]** Alphanumeric Sender ID makes replies physically impossible — the whole inbox premise is false
- *Missing · days* — `src/lib/actions/sms.ts:14-17` `deriveSenderId` strips non-alphanumerics and slices to 11 chars; `:123` `business?.sms_sender_id?.trim() || deriveSenderId(...)` feeds `:148-151` as the ClickSend `from`. The settings validator at `src/components/settings/settings-client.tsx:86` is `.max(11).regex(/^[A-Za-z0-9]*$/)` — it forbids `+`, so the field cannot hold an E.164 number. UI: `messages-client.tsx:315` 'No messages yet. Say hello!'; `:319-345` renders inbound/outbound reply bubbles, `:353-360` a composer. I grepped `sms.ts` and `booking/notify.ts` for opt-out/unsubscribe/STOP — zero hits.
- **Impact.** An owner sends a text from a two-way-looking inbox and waits for a reply the customer's handset has no number to send to. No opt-out text is appended to any send path.
- **Fix.** Relabel `/messages` as an outbound send log, drop the conversational framing at `messages-client.tsx:315` and the reply composer, and append an opt-out line in `clicksendSend` (`sms.ts:39-42`) so it covers booking notifications too. Real two-way needs provisioned long codes.
- ☐ **[HIGH]** Task write RLS grants roles that do not exist and omits 'editor' — editors silently cannot use the board
- *Function · hours* — `supabase/migrations/20260428000001_tasks.sql` — insert `:36-44` and update `:47-55` gate on `m.role in ('admin', 'manager', 'staff')`; delete `:58-66` gates on `m.role in ('admin', 'manager')`. `src/types/database.ts:1591` defines `MemberRole = 'admin' | 'editor' | 'viewer' | 'worker'` — 'manager' and 'staff' do not exist. `src/lib/permissions.ts:6-8` `canEdit` includes 'editor'. I grepped `supabase/migrations` for policies on `public.tasks`: only this file (`20260428110001`, `20260511020000`, `20260522184355` touch tasks but add columns/indexes, no policies). `src/lib/actions/tasks.ts:41-47` uses the cookie client, so RLS is live on every task call.
- **Impact.** An 'editor' — the app's normal can-edit role — is blocked by the database from creating, updating or deleting any task while the UI shows full controls. 'manager'/'staff' grant nothing to anyone. The SELECT policy at `:26-33` admits any active member with no worker exclusion and no `tasks_no_workers` policy exists, so a worker querying tasks directly reads the whole board.
- **Fix.** New migration replacing the write policies' role lists with ('admin','editor') (delete may stay admin-only if that is intended), and add a worker exclusion to `tasks_select_business` mirroring the

`_no_workers` pattern in `20260430000001_worker_role_and_isolation.sql`.

Deferred (6) — medium/low on a feature marked *simplify*

- **[MEDIUM]** Webhook parses Twilio but the app sends via ClickSend, and falls back to the oldest business in the database — `src/app/api/sms/webhook/route.ts:10-13` reads `From/To/Body/MessageSid` as urlencoded Twilio params; `:108-111` returns TwiML `<Response/>`. Sending is ClickSend — `src/lib/actions/sms.ts:33` POSTs to `https://rest.clicksend.com/v3/sms/send` with Basic auth. Tenant match is `.eq("twilio_phone", to)` at `route.ts:30`; I grepped the whole repo for `twilio_phone` and got exactly two hits: that line and `supabase/migrations/20260413000002_sms.sql:33` (the ALTER). Nothing ever writes it. `route.ts:36-43` then falls back to `.order("created_at", {ascending:true}).limit(1)` — the oldest business — and writes the inbound message into it (`:65-105`).
- **[MEDIUM]** Kanban create, save and drag failures are swallowed — the board silently lies about what was saved — `src/components/tasks/kanban-board.tsx:186-192` — `onDragEnd` applies the optimistic reorder at `:175-184` then `catch (err) { console.error("moveTask failed", err) }` with no revert. `:219-227` `handleDelete` removes the card from state and closes the modal BEFORE awaiting `deleteTask`, then `console.error` only. `:195-200` `handleCreate` and `:202-217` `handleSave` have no try/catch at all. There is no `toast` import anywhere in the file (grep: zero hits), while `src/components/tasks/tasks-widget.tsx:36-39` does toast on both success and failure.
- **[MEDIUM]** Deleting a task destroys it instantly with no confirmation and no undo — `src/components/tasks/kanban-board.tsx:782-787` — the Delete button's `onClick` is `onDelete` with no intermediate step; it is wired at `:329` to `() => handleDelete(editing.id)`, which at `:220-226` filters state then calls `deleteTask`. `src/lib/actions/tasks.ts:217-228` is a real `.delete().eq("id").eq("business_id")` — no soft-delete column, no `cleanup_runs` entry.
- **[MEDIUM]** Kanban hand-rolls a neutral-* palette and a raw modal instead of the Connected Hub design system — `src/components/tasks/kanban-board.tsx:43-48` COLUMNS tones are `bg-neutral-100 / bg-blue-50 / bg-amber-50 / bg-emerald-50`; `:50-55` `PRIORITY_TONE` is a private neutral/amber/red map, not `.ch-pill`. `:245-250` the search input and `:252-283` three native ☐ **[HIGH]** One unreachable photo URL kills the entire report PDF
 - `Function · hours` — `src/components/reports/report-pdf-document.tsx:226` renders `<Image style={styles.photoImage} src={photo.url} />` against the raw remote URL, inside `renderToStream` at `src/app/api/pdf/report/[id]/route.ts:74`. The logo IS pre-fetched and base64-embedded at `route.ts:52-61` with the comment at `49-50` ("react-pdf can't reliably load remote images during a server-rendered stream"), and the Props comment at `report-pdf-document.tsx:71-75` repeats it. Photos get no equivalent. The catch at `route.ts:90-94` returns `NextResponse.json({error}, {status:500})`, and the only callers open it in a new tab: `reports-client.tsx:94` and `report-detail-client.tsx` (Downloads card) both use `<a href={ /api/pdf/report/${id} } target="_blank">`.
 - **Impact.** If any photo URL fails to resolve, the whole PDF render rejects and the customer/user sees a raw JSON error blob in a new tab. There is no partial degradation and no user-readable error. The logo fetch at `route.ts:54` also has no timeout, so a slow logo host stalls every report PDF.
 - **Fix.** Pre-fetch every `photo.url` in parallel with an `AbortSignal` timeout and convert to data URLs exactly as the logo already is (`route.ts:52-61`), passing them into `ReportPdfDocument`; skip individual failures instead of aborting. Return an HTML error page rather than JSON from the catch, since the URL is always opened in a browser tab.
- ☐ **[HIGH]** Deleting a report orphans its photos permanently in a public bucket
 - `Function · hours` — `src/lib/actions/reports.ts:224` `const user = await getUser();` is the DELETING user; `:230-236` lists and removes `${user.id}/${id}` under the report-images bucket. Upload keys the path to the CREATING user: `report-generator.tsx:109` `${user.id}/${report.id}/${crypto.randomUUID()}.${ext}`. `supabase/migrations/006_report_images_bucket.sql` restricts DELETE to `auth.uid()::text = (storage.foldername(name))[1]`, and the SELECT policy is `using (bucket_id = 'report-images')` with the bucket created `public` — so a different user's `list()` returns nothing to remove AND anyone with the URL can still read the files. The try/catch at `reports.ts:238-240` swallows it with the comment "Storage cleanup failure should not block DB delete". The confirmation dialog meanwhile promises the opposite: `reports-client.tsx:108` "This will permanently delete the report and all associated images."
 - **Impact.** When anyone other than the original inspector deletes a report — the normal case on the multi-user roofing account — the DB row goes, the UI claims the images were deleted, and every inspection photo remains publicly resolvable by URL forever. Storage grows unbounded.

- **Fix.** Read the report row's `user_id` before deleting and list/remove under that prefix, or do the storage cleanup with the service-role admin client (`src/lib/supabase/admin.ts`) which is not bound by the folder-owner policy. Also stop swallowing the error silently.
- ☐ **[MEDIUM]** Going Back in the wizard creates a duplicate report row and re-uploads every photo
 - *Function · hours* — `src/components/reports/report-generator.tsx:78` `handleStep1` calls `createReport(...)` at :86 with no check of the existing `reportId` state (declared :52, set :95). It is the sole handler on the Continue button at :385. Step 2's Back button at :413 is `onClick={() => setStep(1)}` with no reset. The upload loop at :106-134 re-runs against the NEW report.id, and `images` state is never cleared, so every file uploads again to a fresh `${user.id}/${newReportId}/...` path (:109).
 - **Impact.** Every Back-then-Continue leaves an orphaned draft report row plus a full duplicate copy of the photos in the report-images bucket. The reports list has no bulk delete and no CleanupButton (verified: `reports-client.tsx` has neither), so the owner deletes them one at a time.
 - **Fix.** Guard the top of `handleStep1`: if `reportId` is set, call `updateReport(reportId, {...})` instead of `createReport`, and skip photos already present in the saved report rather than re-uploading the whole `images` array.
- ☐ **[MEDIUM]** `/api/ai` is the only AI route in the repo with no `maxDuration`, and it runs the longest call
 - *Function · minutes* — `src/app/api/ai/route.ts` has no `maxDuration` export anywhere. The `generate_report` branch at :68-160 builds `imageContent` from every photo as a `type: "url"` image block (:75-78) and calls `messages.create` with `max_tokens: 8192` (:125). The client shows fake rotating progress (`report-generator.tsx:33-41`, interval at :152-155) and on any failure just toasts and stays put (:226-230).
 - **Impact.** On a long generation the function is killed mid-call, the user gets a generic 'Generation failed' toast after minutes of fake progress, and the draft is left in the list with empty sections. There is no regenerate action on the detail page (verified: `report-detail-client.tsx` has section/caption editing only), so recovery means re-running the whole wizard.
 - **Fix.** Add `export const maxDuration = 300;` to `src/app/api/ai/route.ts` to match its siblings, and add a 'Regenerate with AI' action on the report detail page so a failed draft is recoverable without re-uploading photos.
- ☐ **[MEDIUM]** Site reports have zero MCP tools — the assistant cannot see or create them
 - *Missing · hours* — `register-tools.ts:1386` `{ customer_id: UUID.optional(), invoice_id: UUID.optional(), quote_id: UUID.optional(), report_id: UUID.optional() }` and :1405 `out.report_url = ...` are the only report references in the entire MCP surface. `src/lib/mcp/collect.ts` feeds the same registry to the in-app assistant, so the gap is identical on both. `CLAUDE.md` marks this a hard standing rule ("every new feature MUST get an MCP tool").
 - **Impact.** Against the owner's stated voice/AI-first principle, 'show me the inspection report for the Woy Woy job' or 'mark that report complete' cannot be answered — the assistant has no concept that site reports exist. Reports are one of the few entities with no coverage at all.
 - **Fix.** Add `list_reports / get_report / create_report / update_report_section / set_report_status / get_report_pdf_link` to `src/lib/mcp/register-tools.ts` under a `reports:read / reports:write` scope pair, running through the admin client scoped by `business_id` like every neighbouring tool.
- ☐ **[MEDIUM]** The whole feature is hardcoded to roofing with no per-trade templating
 - *Missing · days* — `src/lib/templates/roof-inspection.ts:10-49` is a fixed 7-entry `ROOF_INSPECTION_SECTIONS` array (`executive_summary`, `tile_condition`, `biological_contamination`, `ridge_hip_capping`, `valleys_flashings`, `solar_panel_mounting`, `structural_assessment`) returned verbatim by `getDefaultSections()`, used at `reports.ts:81` on every create. The AI prompt hardcodes the same seven keys at `src/app/api/ai/route.ts:95-103` and the system prompt at :126 says 'expert roofing inspector'. `report-pdf-document.tsx:118` prints the literal `<Text style={styles.coverSubtitle}>ROOF INSPECTION REPORT</Text>` and :165 iterates `ROOF_INSPECTION_SECTIONS.slice(1)` rather than `report.sections`. `report-detail-client.tsx` does the same, mapping `ROOF_INSPECTION_SECTIONS.slice(1)` and looking each id up in `sectionMap`.
 - **Impact.** Any non-roofing business installing the plugin gets a document titled ROOF INSPECTION REPORT with a mandatory 'Solar Panel Mounting Interface' section. Because both the PDF and the detail page iterate the constant and not the row, the stored sections array is effectively write-only — a section not in the constant can never be displayed.

- **Fix.** Move the section list into a per-business report_templates row (or reuse the field-schema engine in src/lib/onboarding), drive the AI prompt and the PDF cover subtitle from it, and iterate report.sections rather than intersecting with the constant.
- ☐ **[MEDIUM]** Reports list and generator ignore the design system — purple accents, hand-rolled cards, truncated titles, no search
 - *UI · hours* — report-generator.tsx:282 `accent="linear-gradient(180deg, #a78bfa 0%, #6d28d9 100%)"`, :293 the same purple gradient on the progress bar, :419 `className="gap-1.5 bg-purple-600 hover:bg-purple-700 text-white"`, :237-241 `bg-purple-50` tile with `text-purple-500` icons, :351 `hover:border-purple-400/60`. reports-client.tsx:73-74 hand-rolls `bg-blue-50` / `text-blue-600`, :21-24 hand-rolls `statusColor` instead of `.ch-pill`, :78 `className="font-semibold truncate"`. The list renders reports.map with no search input, no filter, no sort, no pagination and no CleanupButton; getReports (reports.ts:19-27) applies no .limit at all.
 - **Impact.** Site Reports reads as a different product from the leads workspace that is the stated quality bar. Titles are built as `${title} - ${propertyAddress}` (report-generator.tsx:87) and then truncated, clipping the exact field that distinguishes two reports — directly against the owner's wrap-don't-truncate preference. With no search and no limit, finding a report on the roofing account is an unbounded scroll.
 - **Fix.** Rebuild reports-client.tsx on PageHeader + kirei CardListRow/StatTile with break-words, add a search box and a draft/complete tab strip, route status through .ch-pill, and replace every purple literal in report-generator.tsx with the teal primary token.
- ☐ **[MEDIUM]** Risk assessment table and status badges are hardcoded light-mode colours
 - *UI · minutes* — report-detail-client.tsx:26-31 `ratingColors = { Critical: "bg-red-100 text-red-700", High: "bg-orange-100 text-orange-700", Medium: "bg-yellow-100 text-yellow-700", Low: "bg-green-100 text-green-700" }` with a `bg-gray-100 text-gray-700` fallback — no dark variants. :258 `<tr className="bg-slate-800 text-white">`; :267 `className={i % 2 === 0 ? "bg-white" : "bg-slate-50"}`; the advisory banner `bg-yellow-50 border-l-4 border-yellow-400` with `text-yellow-800`; and the sidebar status badge `bg-green-100 text-green-800` / `bg-yellow-100 text-yellow-800`.
 - **Impact.** In dark mode the risk assessment table — the most consequential content in the document — renders as white and near-white rows inside a dark card, and the Critical/High badges wash out. The dark-mode toggle ships in Settings → Appearance, so this is reachable today.
 - **Fix.** Move the four rating tones into a shared const with dark: variants mirroring lead-shared.ts, and swap bg-white / bg-slate-50 striping for bg-card / bg-muted tokens.
- ☐ **[MEDIUM]** No way to email the report to the customer, and no way to add photos after generation
 - *Missing · days* — No sendReportEmail exists anywhere in src (grep). The detail page's only outbound path is `<ShareWithCustomerDialog ... docType="report" docId={report.id} />` plus two download anchors in the Downloads card. The photo file input exists only in report-generator.tsx:356; report-detail-client.tsx's Photographic Record card renders `report.photos.map(...)` as `<img>` with an editable caption and offers no add, delete or reorder control.
 - **Impact.** After an inspection the roofer copies a link into their own mail client, so the report never goes out from the business's branded from-address and no email_events row records it — unlike invoices and quotes, which both have full templated senders. If the inspector missed a photo the only options are re-running the wizard or leaving it out.
 - **Fix.** Add a sendReportEmail action reusing getResolvedEmailTemplate with a new `report` template type and the PDF attached, mirroring src/lib/actions/invoices.ts, and add an upload dropzone plus delete/reorder to the Photographic Record card.
- ☐ **[LOW]** Zero tests on the only feature that produces a client-facing legal document
 - *Tests · hours* — src/app/api/ai/route.ts:151-157 builds `validRatings = new Set(["Low", "Medium", "High", "Critical"])` then maps `rating: validRatings.has(item.rating) ? "" : item.rating` — any unrecognised rating is silently rewritten to Medium with no log and no flag on the stored row. Nothing tests src/lib/actions/reports.ts, that parser, or ReportPdfDocument.

- **Impact.** An unrecognised severity string is downgraded to Medium inside a document the business hands to a client as an inspection finding, with nothing recording that a substitution happened. Separately there is no coverage that a report with zero photos, null sections or a missing meta key renders rather than 500ing.
- **Fix.** Extract the extract-JSON / pad-captions / validate-rating block from the route into a pure helper and test it (including logging rather than silently coercing an unknown rating), and add a renderToBuffer test for ReportPdfDocument against empty-report, no-photos and unknown-rating fixtures.
- ☐ **[LOW]** Parallel Telegram report generator hardcodes one business id and is unreachable from the app
 - *Function · hours* — `src/app/api/report-sessions/generate/route.ts:21` `const AGENT_BUSINESS_ID = process.env.AGENT_BUSINESS_ID ?? "ff3a47f3-54b0-45e3-b7a9-69ddc9fa787e"`, used at the logo fetch `.eq("id", AGENT_BUSINESS_ID)` regardless of which session is generating. The prompt hardcodes 'You are an expert roofing inspector for Crown Roofers, a professional roofing company in Sydney, Australia'; reference numbers are minted as `CRR-${year}-...`; the PDF uploads to a separate `roof-reports` bucket. It renders `src/components/reports/roof-inspection-pdf.tsx`, a second PDF template. Grepping `src` for `report-sessions / roof-inspection-pdf` outside that directory returns nothing.
 - **Impact.** Two report data models, two PDF templates and two storage buckets ship in every build, which misleads anyone reading the feature. If a second business ever reached this path it would be branded Crown Roofers and billed against one hardcoded business id.
 - **Fix.** Do NOT delete on the strength of a repo grep. First confirm via Vercel logs whether `/api/report-sessions/*` is receiving traffic. If it is live, point it at the shared reports table and ReportPdfDocument and derive the business from the session instead of AGENT_BUSINESS_ID. Only if it is confirmed cold should the route, `roof-inspection-pdf.tsx` and the `roof-reports` bucket be removed.

Batch 2.19 — MCP server & API surface (/api/mcp, /api/v1, OAuth AS, API keys)

Health: usable · **Verdict:** simplify · **Findings:** 0 blocker · 2 high · 6 medium · 2 low **Estimated:** ~0.4 days (2 scheduled, 8 deferred)

What it does. Kirei exposes one shared tool registry — 217 tools (counted across `src/lib/mcp/register-tools.ts` and `src/lib/mcp/tools/*.ts`) — through two surfaces: the MCP server at `/api/mcp` (`src/app/api/mcp/route.ts`) for Claude Code and the claude.ai connector, and `src/lib/mcp/collect.ts` + `invoke.ts` for the in-app assistant. Every tool reads business context off the authenticated key, calls `assertScope()`, and runs against the service-role Supabase client manually filtered by `business_id` — RLS is bypassed here, so `assertScope` is the only gate. Two ways in: a per-business `inv_*` key from Settings → API (validated by `src/lib/api-auth.ts`), or a full OAuth 2.1 authorization server (`src/app/api/oauth/{register,authorize,token}`) whose token endpoint mints an `inv_*` admin key as the access token. Separately there is a small legacy REST API at `/api/v1/{customers,leads,agent}`.

Why this rating. The MCP core is the best-engineered part of this codebase: I scripted every one of the 217 tool registrations and all 217 call `assertScope`, and all scope every query by `business_id`. But the REST `/api/v1` surface is broken for any admin key, there is a cross-tenant auth fallback in `src/lib/api-auth.ts:86-113`, the OAuth tokens are immortal and always full-admin, and the entire feature is invisible in the product UI.

If you only do three things:

1. Fix `src/lib/api-auth.ts:121` to expand `admin` (unbreaks every REST call from every OAuth-minted key), delete the cross-tenant `INTERNAL_API_KEY` fallback at lines 83-114, and add `src/lib/tests/api-auth.test.ts` so neither can regress. Half a day, removes the only cross-tenant path I found.
2. Harden the OAuth server: set `expires_at` + return `expires_in` on the minted key (`token/route.ts:68-85`), give each connection a distinct label and revoke the prior one, and let the consent screen grant a scope subset instead of hardcoding admin (`authorize/route.ts:183`). Add `rateLimit()` to the unauthenticated `/api/oauth/register` insert.
3. Delete `/api/v1` entirely (`customers, leads, agent`) and surface the MCP server in Settings → API instead — the connect URL, the `claude mcp add` snippet, and a Connect-claude.ai button. That trades three broken, unrate-limited endpoints for making the 217-tool surface something a customer can actually find.

Scheduled

- ☐ **[HIGH]** OAuth consent ignores the requested scope and always grants full admin, forever

- *Security · hours* — Every cited line checks out. `src/app/api/oauth/authorize/route.ts:38` reads `scope` from the query (`sp.get("scope") ?? "admin"`), threads it through the hidden form fields at :112, and then line 183 hardcodes `scope: "admin"` on the `oauth_codes` insert — the requested value is discarded. `src/app/api/oauth/token/route.ts:74` inserts `scopes: ["admin"]`; the insert at :68-75 sets no `expires_at`, and the response at :78-85 returns only `access_token/token_type/scope` — no `expires_in`, no `refresh_token`. `ls src/app/api/oauth/` shows only `authorize`, `meta`, `register`, `token` — no revocation endpoint. The consent copy at `authorize/route.ts:143` is the fixed string "Full admin access". `src/types/database.ts:1571-1575` confirms `expandApiScopes` turns `admin` into every concrete scope. The dangerous tools are real: `delete_customer` at `register-tools.ts:685` (`assertScope customers:write`) and `email_prospects` at `src/lib/mcp/tools/prospects-tools.ts:122` (max 500 recipients).
- **Impact.** A claude.ai connect grants a non-expiring, non-downgradeable credential covering all 40 concrete scopes on a business with live Stripe — `charge_saved_card`, `delete_customer`, `email_prospects`, `update_settings`. The only revocation path is manually finding the right row in Settings → API. One correction to the finding's arithmetic: `ALL_API_SCOPES` has 41 entries, and `expandApiScopes` drops `admin` itself, so it is 40 concrete scopes, not 41.
- **Fix.** As proposed. Two independent pieces: (a) set `expires_at` on the `business_api_keys` insert in `token/route.ts:68-75` and return `expires_in` — cheap and immediate; (b) honour a scope subset by rendering `ALL_API_SCOPES` groups as checkboxes in the authorize consent HTML and persisting the selection to `oauth_codes.scope`, then reading it at `token/route.ts:74` instead of the literal. Do (a) first.
- ☐ **[HIGH]** `INTERNAL_API_KEY` fallback grants cross-tenant access to whichever business is oldest
- *Security · minutes* — `src/lib/api-auth.ts:83-114` is exactly as described: after the per-business key lookup fails, `const legacyKey = xApiKey ?? authHeader?.slice(7)` (line 86) accepts the secret via either header, constant-time-compares it to `process.env.INTERNAL_API_KEY`, then resolves `.from("businesses").select("id, user_id").order("created_at", { ascending: true }).limit(1).single()` (lines 97-102) and returns that business with scopes `leads:read/write`, `customers:read/write`, `agent:access` (line 109). The comment at line 85 does say "Remove this block once all integrations have migrated." I additionally confirmed the var is actually populated — `.env.local:6` sets `INTERNAL_API_KEY` to a 64-hex value, so this is not dead configuration. The finding's note about the other consumers is accurate: `src/app/api/report-sessions/route.ts:13` and `.../generate/route.ts:25` compare the header to the same var but never resolve a business, so they are unaffected.
- **Impact.** One shared secret yields a service-role-backed context on an arbitrary tenant — the platform's first-created business — with read AND write on leads and customers plus agent access. RLS cannot catch it because the context is handed out above RLS. Not an open door (the holder must know the secret), but it is a latent misconfiguration with no per-tenant binding and no audit trail beyond a console.warn.
- **Fix.** Remove lines 83-114 of `src/lib/api-auth.ts`. Keep the env var itself — `report-sessions` still needs it.

Deferred (8) — medium/low on a feature marked *simplify*

- **[MEDIUM]** No rate limiting on /api/mcp, /api/v1, or the unauthenticated OAuth client registration — src/app/api/oauth/register/route.ts:16-44 parses JSON, requires only a non-empty redirect_uris array (line 30), and inserts into oauth_clients (line 37) with no authentication and no throttle. src/app/api/mcp/route.ts has no limiter — withMcpAuth at :49-70 only validates the key. Neither /api/v1/customers nor /api/v1/leads calls any limiter (verified by reading both files end to end). The reusable helper does exist: src/lib/booking/public.ts exports `rateLimit(key, limit, windowMs)` and `clientId(req)`, described in its own comment as best-effort in-memory. The ad-hoc limiter in src/app/api/v1/agent/route.ts:26-40 (30 req / 5 min) is confirmed.
- **[MEDIUM]** requireScope() never expands `admin`, so admin keys get 403 on every REST endpoint — src/lib/api-auth.ts:121-123 is verbatim `return scopes.includes(required)` with no expansion, and the file imports only ApiScope from @/types/database — expandApiScopes is not in scope. The divergence with src/lib/mcp/context.ts:22-29 (which does `expandApiScopes(ctx.scopes)`) is real. Grep for requireScope returns exactly the five call sites cited: v1/customers/route.ts:22,66; v1/leads/route.ts:21,81; v1/agent/route.ts:866. src/app/api/oauth/token/route.ts:74 does insert `scopes: ["admin"]`, and `admin` is the first entry of ALL_API_SCOPES (types/database.ts:1528), so it is the most likely single scope a user picks in the create dialog.
- **[MEDIUM]** The MCP server is completely undiscoverable inside the product — Grepping src/app/(dashboard)/, src/components/settings/ and src/components/layout/ case-insensitively for "api/mcp", "claude mcp" and "connector" returns zero matches. The only integration guidance in the UI is src/components/settings/api-keys-settings.tsx:199-205, a Usage block whose entire content is the line `Authorization: Bearer inv_your_key_here`. The connect one-liner exists solely as a source comment at src/app/api/mcp/route.ts:5-8.
- **[MEDIUM]** REST `limit` param is unvalidated — NaN 500s and unbounded full-table dumps — src/app/api/v1/leads/route.ts:85 and src/app/api/v1/customers/route.ts:70 both read `parseInt(searchParams.get("limit") || "50")` with no isNaN guard and no ceiling, feeding it straight into `.limit(limit)` (leads:94, customers:80). leads/route.ts:91 does `.select("*")`; customers/route.ts:76 correctly selects a named column list, and the finding does not claim otherwise. The MCP contrast holds — register-tools.ts:152 constrains limit with `z.number().int().min(1).max(200).optional()`.
- **[MEDIUM]** API-keys panel ignores the design system and its scope picker has no admin feedback, search, or post-hoc editing — src/components/settings/api-keys-settings.tsx imports and uses generic shadcn Card/Table/Badge/Checkbox (lines 10-39) with no .ch-table-wrap/.ch-table/.ch-pill from globals.css and nothing from src/components/ui/kirei/. The create dialog at lines 230-250 maps ALL_API_SCOPES grouped by `scope.group` into a flat checkbox list — no search input, no per-group toggle. `admin` is the first array entry (types/database.ts:1528) so it renders as the first checkbox; toggleScope (109-113) treats it identically to every other scope, giving no signal that it subsumes the rest. src/lib/actions/api-keys.ts exports only createApiKey, listApiKeys, revokeApiKey — no scope editing. Eye and EyeOff are imported at line 6 and appear nowhere else in the file.
- **[MEDIUM]** The auth boundary for the whole API surface has zero tests — No src/lib/**tests**/api-auth.test.ts exists (the directory holds only pg-filter.test.ts) and there is no src/app/api/oauth/**tests** directory at all. src/lib/mcp/**tests**/ holds collect.test.ts, invoke.test.ts and live-api.test.ts — and I read collect.test.ts in full: its six cases cover tool count floor, duplicate names, name indexing, per-registrar presence, Anthropic input_schema validity and zod→JSON-Schema fidelity. Not one of them executes a handler or exercises a scope refusal. The finding's characterisation is accurate.
- **[LOW]** Every OAuth reconnect mints another permanent key with an identical label — src/app/api/oauth/token/route.ts:71 sets `label: "Claude (claude.ai connector)"` as a literal on every insert. The insert at :68-75 does no lookup for an existing key on the same (business_id, client_id) and sets no expires_at. The Settings list (api-keys-settings.tsx:160-192) renders label plus `inv_{key_prefix}...`, so duplicates are visually distinguishable only by the 8-hex prefix and the created/last-used timestamps.
- **[LOW]** API keys can never expire — the column exists but nothing ever sets it — expires_at is declared on BusinessApiKey (src/types/database.ts:1596) and enforced at authentication time (src/lib/api-auth.ts:65: `if (data.expires_at && new Date(data.expires_at) < new Date()) return null;`), and it is

selected back on both create and list (api-keys.ts:75, :93). But the insert at api-keys.ts:66-74 never sets it, and the create dialog (api-keys-settings.tsx:219-251) has only a label field and the scope checkboxes. No caller anywhere sets expires_at.

Batch 2.20 — Settings, email templates & integrations

Health: rough · **Verdict:** simplify · **Findings:** 0 blocker · 2 high · 6 medium · 2 low **Estimated:** ~0.4 days (2 scheduled, 8 deferred)

What it does. A single admin-gated /settings page with seven tabs (Business, Payment, Documents, Appearance, API, Email, Webhooks) rendered by one 613-line client component, plus three sub-routes: /settings/email-templates (five email types, field editor + sandboxed iframe preview + reset), /settings/booking and /settings/work-order-templates. It stores business identity/branding/bank details/invoice numbering on the **businesses** row, live-applies accent/sidebar/pattern theming, mints and revokes hashed **inv_*** API keys with per-scope checkboxes, registers outbound webhooks with an optional HMAC secret, configures an IMAP mailbox for the AI lead scanner, and links to Stripe Connect onboarding. Every action file re-implements the same inline role lookup and gates on canManageSettings (owner/admin), except email-templates.ts which has no code-level check and relies on RLS (editor+).

Why this rating. The owner-only happy path works, which is why the one dependent roofing business hasn't hit it. But the moment a second person with the **admin** role touches Settings, every save fails — updateBusiness filters on **user_id = auth user**, which only the owner satisfies — and logo upload fails silently because its update error is discarded entirely. Separately, customer mailbox passwords are stored in cleartext despite two AES-GCM helpers already existing in the repo, and the account menu's Profile link 404s for every user in the product. The webhooks tab is half-built: a delivery-log query was written and never wired to any UI, so a failing integration is undebuggable.

If you only do three things:

1. Fix the owner-only write filter in src/lib/actions/business.ts:127 and 209 so admin-role members can actually save business details, branding and logos — and make uploadLogo check its update error instead of discarding it and toasting success. This is a two-line fix that turns a silently broken role into a working one, and it must land before a second person is ever invited into an account.
2. Deal with the plaintext IMAP password (email-config.ts:103 / migration 20260417000002). Either encrypt it with the existing src/lib/crypto.ts on write and decrypt only in the cron reader, or delete the Email Lead Scanner outright. At 2 active users and with ANTHROPIC_API_KEY 401ing in production the feature is inert anyway, so holding customer mailbox credentials in cleartext is pure downside risk.
3. Build /settings/profile — it is linked from the account menu (app-header.tsx:91) for every user and 404s, and it is the only settings path workers are allowed (permissions.ts:45). While there, add the "send test email" button to the template editor; together these close the two gaps a new customer would notice in their first hour.

Scheduled

- ☐ **[HIGH]** Admin-role members cannot save any business setting
 - *Function · minutes* — src/lib/actions/business.ts:127-132 — .update(payload).eq("id", businessId).eq("user_id", user.id).select().single(); line 133 throws on error. For a non-owner the WHERE matches 0 rows and .single() returns PGRST116, so error is truthy and the action throws. src/app/(dashboard)/settings/page.tsx:22-35 resolves userRole from business_members and admits anyone passing canManageSettings; src/lib/permissions.ts:22-24 includes 'admin'. settings-client.tsx:120 destructures userRole and it appears nowhere else in the 613-line file (verified by full read). Catch blocks at 173/181/189 emit a bare "Failed to save".
 - **Impact.** Any admin added to a business gets a Settings page that renders every form and fails every write with an unexplained toast. Business details, bank details, invoice numbering and all three appearance settings are affected.
 - **Fix.** Drop the .eq("user_id", user.id) filter and let RLS on businesses gate the write (the pattern the rest of the codebase uses), or resolve the role and permit owner+admin. Surface the real error message in the toast.
- ☐ **[HIGH]** Customer mailbox passwords stored in plaintext

- *Security · hours* — supabase/migrations/20260417000002_business_email_config.sql:10 — `imap_pass text not null`, no encryption. `src/lib/actions/email-config.ts:103` writes `imap_pass: password` straight into the payload; :92 reads existing?.`imap_pass` raw; :57-62 reads the raw value to build the mask; :141 reads it raw again for the connection test. `src/lib/crypto.ts` and `src/lib/onboarding/crypto.ts` both exist as AES-256-GCM helpers.
- **Impact.** Each configured business's mailbox app-password is readable in Postgres to anything holding the service-role key or a backup. That mailbox is typically the password-reset destination for the rest of the business's accounts.
- **Fix.** Encrypt via `src/lib/crypto.ts` on write in `saveEmailConfig`; decrypt only in `testEmailConnectionAction` and the email-leads cron reader; migrate existing rows in the same PR. Or delete the feature and the column if unused.

Deferred (8) — medium/low on a feature marked *simplify*

- **[MEDIUM]** Logo upload silently discards its database error — `src/lib/actions/business.ts:209-212` — `await tbl(supabase,"businesses").update({ logo_url: urlData.publicUrl }).eq("id", businessId).eq("user_id", user.id)` with no `{ error }` capture; line 219 returns `urlData.publicUrl` unconditionally. `src/components/settings/settings-client.tsx:221-223` then sets local state and toasts "Logo uploaded".
- **[MEDIUM]** `updateBusiness` accepts an unvalidated Partial from the client — `src/lib/actions/business.ts:121-131` — `updateBusiness(payload: Partial)` in a "use server" file passes payload directly to `.update(payload)` with only `.eq("id", businessId).eq("user_id", user.id)` as constraints. No key filtering anywhere in the function.
- **[MEDIUM]** The account menu's Profile link is a 404 for every user — `src/components/layout/app-header.tsx:91` — Profile. `find src/app -path "*profile*"` returns nothing; `src/app/(dashboard)/settings` contains only `page.tsx`, `loading.tsx`, `booking/`, `email-templates/`, `work-order-templates/`. `src/lib/permissions.ts:45` lists `"/settings/profile"` in `WORKER_ALLOWED_PATHS`. `src/app/(dashboard)/settings/page.tsx:35` redirects non-admins to `/dashboard`.
- **[MEDIUM]** Webhooks tab has no delivery log, no edit, and no test-fire — `src/lib/actions/webhooks.ts:103` (`getRecentDeliveries`, no importers); `src/components/settings/webhooks-settings.tsx:5` (unused status icons), :24 (imports); `src/lib/webhooks.ts:80-91` (deliveries written with full status detail).
- **[MEDIUM]** Email templates cannot be sent as a test to a real inbox — `src/components/settings/email-templates-client.tsx` — preview state is set at :86-88 from `previewEmailTemplate` and rendered into an `iframe`; no send path exists in the file. No `sendTest*` symbol anywhere under `src/components/settings/` or `src/lib/actions/email-templates.ts`.
- **[MEDIUM]** Switching template tabs silently discards unsaved edits — `src/components/settings/email-templates-client.tsx:67-73` — `useEffect(() => { const s = initial.find(...); if (s) setForm(formFromSummary(s)); ... }, [active, initial]);` tab strip at :143-157 with no guard.
- **[LOW]** Settings deviates from the design system the leads workspace sets — `src/components/settings/stripe-settings.tsx:64` `if (!confirm("Disconnect Stripe? ...")) return;` ; `src/components/settings/api-keys-settings.tsx:30-34` and `webhooks-settings.tsx:20-23` import `AlertDialog` for equivalent destructive actions; `webhooks-settings.tsx:151` `truncate max-w-[200px]` ; identical 6-line tab-trigger class at `settings-client.tsx:259` and `email-templates-client.tsx:149`; `src/app/(dashboard)/settings/loading.tsx` re-exports `../loading`.
- **[LOW]** The template engine that formats every outbound email has zero tests — `find src -name "*.test.tsx"` returns 14 files: `tests/pg-filter`, `assistant/{models,scopes,undo}`, `booking/{availability,booking-db,time}`, `content/{live-agents,pipeline,prompts,schedule}`, `mcp/{collect,invoke,live-api}`. `src/lib/emails/templates.ts:147-155` (`renderTemplateVars`) and :158-175 (`resolveEmailTemplate`) are pure and untested; the trim-vs-?? asymmetry sits at :165 vs :166, `custom_html` nulling at :173.

Batch 2.21 — Dashboard & analytics

Health: rough · **Verdict:** simplify · **Findings:** 0 blocker · 2 high · 6 medium · 1 low **Estimated:** ~0.4 days (2 scheduled, 7 deferred)

What it does. Two separate surfaces. `/dashboard` (`src/app/(dashboard)/dashboard/page.tsx`) auth-gates, branches to a worker view, and otherwise renders four KPI tiles (Revenue this month, Outstanding, Overdue, Paid this month) from a single Postgres RPC `dashboard_stats`, plus a hand-drawn 6-month CSS bar chart, recent invoices, today's/tomorrow's schedule, and four client-side widgets (Briefing, New leads, Tasks, Operations) that each fetch their

own data after hydration. `/analytics` (`src/lib/actions/analytics.ts`) is a separate read-only page with a 30d/90d/12m/YTD range switch that pulls every invoice, quote, lead, work order and customer for the business into Node and reduces them in JS to produce a KPI strip, a recharts 12-month revenue bar chart, top-5 customers, a quote-outcome donut, an A/R aging panel and a lead funnel. Neither surface reads the `payments` table, and neither knows anything about costs or expenses — so there is no profit anywhere in the product.

Why this rating. It renders well and is visually on-system, but the numbers are not trustworthy: `/dashboard` and `/analytics` compute "outstanding" and "overdue" from different status sets and will disagree on the same data, "revenue" is keyed to invoice creation date rather than when money arrived, and `/analytics` silently renders all-zeros if any of its six queries errors because it never checks `error`. One chart (A/R aging) renders bars that carry no information at all. Nothing in the whole feature has a test.

If you only do three things:

1. Make the numbers agree and mean what the labels say. One shared SQL definition of 'open invoice' (including the 'overdue' status) used by `dashboard_stats`, `analytics.ts` and `briefing.ts`; revenue computed from the `payments` table by payment date instead of `invoices.created_at`; error checked on all six `analytics` queries so a failure throws instead of rendering zeros; currency default changed from GBP to AUD. This is roughly a day and it is the difference between a dashboard the owner trusts and one he double-checks against the invoice list.
2. Add cost so the page answers a question the invoice list can't. Join the existing `expenses/job-time/materials` data into the `analytics` payload and put gross margin on the KPI strip. Right now `/analytics` reports what was billed, which the owner already knows — profit is the only reason he opens a reporting screen weekly, and it is what `ServiceM8` and `Tradify` give him.
3. Delete the decoration and fix what's left. The A/R aging bars are literally non-functional (100%/0% width), the quote donut and lead funnel duplicate what the leads workspace already shows better. Cut them, fix the aging bars to be proportional, and fold the 30d/90d/12m range switch onto the dashboard so there is one reporting surface instead of two that disagree.

Scheduled

- ☐ **[HIGH]** Dashboard and Analytics disagree on Outstanding and Overdue — the 'overdue' status is invisible to the dashboard
 - *Function · minutes* — `supabase/migrations/20260716130000_dashboard_stats_rpc.sql:49` `where status in ('sent','partial')` for 'outstanding' and :51-52 the same set for 'overdue'. `src/lib/actions/analytics.ts:108` `const isOpen = ["sent", "partial", "overdue"].includes(inv.status)` and :182 the same three-status list for A/R aging. 'overdue' is in the Invoice status union at `src/types/database.ts:159`. `src/lib/actions/cleanup.ts:694-701` emits `patch: { status: "overdue" }` for any sent/partial invoice past `due_date`. Confirmed the RPC is called at `src/lib/actions/invoices.ts:458`.
 - **Impact.** Once any invoice is flipped to 'overdue' (Smart Organise on invoices, or the status dropdown / MCP `set_invoice_status`), it drops out of the dashboard's Outstanding and Overdue figures while `/analytics` still counts it. The dashboard then under-reports what is owed and the two pages show different money for the same business.
 - **Fix.** Add 'overdue' to both status IN-lists in the RPC (lines 49 and 52) and re-apply the migration. Preferably define one shared SQL predicate for 'open invoice' used by the RPC and `analytics.ts`.
- ☐ **[HIGH]** 'Revenue' and 'Paid this month' are keyed to invoice creation date, not when the money arrived
 - *Function · hours* — `supabase/migrations/20260716130000_dashboard_stats_rpc.sql:53-54` `'paidThisMonth' ... where status = 'paid' and created_at >= date_trunc('month', now())` ;:36 the monthly series joins `on date_trunc('month', i.created_at) = m.m` and :33 sums total filtered on `status='paid'`. `src/lib/actions/analytics.ts:110-112` `if (inRange(created)) { invoiced += total; if (isPaid) { revenue += total; ... } }` and :143-148 buckets months by `new Date(inv.created_at)`. Neither reads `payments`. `Analytics`' six queries (:69-86) do not include the `payments` table.
 - **Impact.** An invoice raised in March and paid in July is counted as March revenue and never appears in July's 'Paid this month'. For multi-month roofing jobs the revenue chart is a chart of when invoices were written, labelled as when they were paid. Cash collected on part-paid invoices is invisible entirely.

- **Fix.** Compute revenue from the `payments` table bucketed by payment date; keep the `created_at` series as a separately-labelled 'Invoiced' line.

Deferred (7) — medium/low on a feature marked *simplify*

- **[MEDIUM]** /analytics silently renders all-zeros when a query fails — `error` is never checked —
`src/lib/actions/analytics.ts:61-68` destructures `{ data: invoicesRaw }, { data: quotesRaw }, { data: leadsRaw }, { data: workOrdersRaw }, { data: customersRaw }, { data: business }` from `Promise.all` — `error` is never bound. Lines 88-96 then coalesce each to `?? []`. Contrast `src/lib/actions/leads.ts:29-30` which does `if (error) throw error`.
- **[MEDIUM]** No profit anywhere — analytics has revenue but no cost, despite an Expenses module existing —
`src/lib/actions/analytics.ts:12-31` `AnalyticsPayload` contains `revenue/invoiced/outstanding/overdue/monthly/top_customers/quote_funnel/ar_aging/lead_funnel/job_status` — no cost, expense, labour or margin field. The six parallel queries at :69-86 hit invoices, quotes, leads, work_orders, customers, businesses only. `src/lib/actions/expenses.ts` exists and is never imported by `analytics.ts`.
- **[MEDIUM]** A/R aging bars carry no information — every non-zero bucket renders a full-width bar —
`src/components/analytics/analytics-client.tsx:220` `style={{ width: amount > 0 ? "100%" : "0%", ... }}`. Lines 207-209 are dead: `const totalAll = Math.max(amount, 1); // for the bar; we'll compute pct against the row's max separately` followed by `void totalAll;`. Compare `LeadFunnel` at :235 `const max = Math.max(...stages.map((s) => s.value), 1)` and :257 `width: ${(s.value / max) * 100}%`.
- **[MEDIUM]** Briefing widget hangs on 'Preparing your briefing...' forever if the fetch throws —
`src/components/briefing/briefing-widget.tsx:39-46` `setLoading(true); try { setSummary(await getMyBriefing()); } finally { setLoading(false); }` — no catch. Lines 60-69 `if (!summary) { return (... 'Preparing your briefing...') }` returns before the header block at :76-84 that hosts the controls.
- **[MEDIUM]** Dashboard widgets turn fetch failures into reassuring empty states —
`src/components/dashboard/new-leads-widget.tsx:52` `} catch { setLeads([]); }`.
`src/components/dashboard/outlook-widget.tsx:28-31` `getMembers().catch(() => []), getRecurringJobs().catch(() => []), listAgentInstalls().catch(() => [])`.
`src/components/tasks/tasks-widget.tsx:27` `} catch { /* swallow - empty state is fine */ }`.
- **[MEDIUM]** Every money calculation in the feature is untested — `find src -name "*.test.ts"` returns 14 paths: `assistant/{models,scopes,undo}`, `booking/{availability,booking-db,time}`, `content/{live-agents,pipeline,prompts,schedule}`, `mcp/{collect,invoke,live-api}`, and `lib/tests/pg-filter.test.ts`. No analytics, briefing or dashboard test exists. The reducers in question are `src/lib/actions/analytics.ts:102-120` (totals), :135-152 (monthly bucketing), :180-192 (aging).
- **[LOW]** New-leads widget downloads the entire leads table client-side to show five rows —
`src/components/dashboard/new-leads-widget.tsx:46-53` `const all = await getLeads(); const filtered = all.filter((l) => l.status === "new" || l.status === "contacted").slice(0, 5);`. `src/lib/actions/leads.ts:22-25` `.select("*").eq("business_id", businessId).order("created_at", { ascending: false })` — no `.limit()` and no column list.

Batch 2.22 — Client onboarding forms

Health: rough · **Verdict:** simplify · **Findings:** 0 blocker · 2 high · 3 medium · 3 low **Estimated:** ~0.4 days (2 scheduled, 6 deferred)

What it does. A per-business plugin (flag in `onboarding_settings.enabled`, toggled from `/agents` or the enable card) that lets a trades/agency business design custom client-intake forms and send them to a customer through the existing customer-portal token system. The builder at `/onboarding-forms/[id]` is a palette/canvas/settings three-pane editor over ~25 field types (text, contact, ABN/ACN with real checksums, opening-hours grid, image/file upload, rating, consent, and an AES-256-GCM-encrypted `secure` credential type), plus a ~45-entry ready-made preset library and one-level `show_if` conditional visibility. Sending mints or reuses a 90-day `customer_portal_tokens` row and

emails the customer a login-free link to `/portal/[token]/onboarding/[requestId]`, where they fill the form with debounced autosave; a final submit runs required-field and format validation server-side, snapshots the schema alongside the answers so later form edits don't orphan them, and flips the request to `completed`. The business reads answers in a response viewer (secure fields redacted, owner/admin-only one-at-a-time reveal) and in an inline tab on the customer profile. It ships with 10 MCP tools so the AI assistant can draft and send forms.

Why this rating. The architecture is sound — schema snapshotting, token gating, redaction-by-default, server-side validation, and RLS mirroring are all done correctly. But the customer-facing autosave loses data silently while displaying "✓ Saved" (onboarding-fill.tsx:68-79), the headline encrypted-credential field is dead in production because `ONBOARDING_SECRET_KEY` is unset, the `settings` object the portal reads can't be written by any UI or MCP tool, and there is no reminder/resend so sent forms just decay. Zero tests cover any of it, including the pure ABN/ACN checksum and crypto round-trip logic that is trivial to test.

If you only do three things:

1. Fix the autosave data-loss path in `src/components/customer-portal/onboarding-fill.tsx:68-79` — retain the dirty patch until `res.ok`, gate the "✓ Saved" indicator on it, show a persistent failure banner, and flush on tab close via `sendBeacon`. Right now the feature can silently lose a customer's answers while telling them they're safe, which is worse than not having autosave at all.
2. Set `ONBOARDING_SECRET_KEY` on Vercel and make the save route degrade per-field instead of 500-ing the whole payload. Until this is done the encrypted-credential field — the one thing this has that a generic form builder doesn't, and the thing the enable card advertises — is unusable in production.
3. Collapse the two builders and two field renderers into one. `src/components/onboarding/form-builder-client.tsx` (476 lines) and `src/components/forms/forms-builder-client.tsx` (398), plus `onboarding-fill.tsx` (385) and `public-form-fill.tsx` (199), already share the `OnboardingField` model, `presets.ts` and `answers.ts`. Keep one of each with a `portal/public` mode prop and delete ~900 lines. Do this before adding features to either, or every new field type has to be built twice.

Scheduled

- ☐ **[HIGH]** `ONBOARDING_SECRET_KEY` unset in production — the advertised encrypted-credential field cannot be used
 - *Missing · minutes* — `src/lib/onboarding/crypto.ts:16-24` throws on a missing/non-64-hex key; `secureFieldsAvailable()` lines 27-30. Builder gates on the flag at `src/components/onboarding/form-builder-client.tsx:116-119` and `206-209` (toast only) and dims the palette button at `235-239`. Pitch copy at `src/components/onboarding/onboarding-forms-client.tsx:64-65` ('even securely-stored credentials'). Save route returns 500 for the whole payload at `src/app/api/portal/[token]/onboarding/[id]/save/route.ts:53-60`. Verified absent from Vercel production env.
 - **Impact.** The one capability separating this from a generic form builder is unreachable in production, and the only feedback is a toast naming a server env var the owner cannot set from the UI. If a form with a secure field were ever created (e.g. via the MCP `create_onboarding_form` tool, which does NOT check the flag — `plugin-form-tools.ts:279-291`), the customer's entire submit fails with a raw internal error string.
 - **Fix.** Set `ONBOARDING_SECRET_KEY` (64 hex) on Vercel and redeploy. Independently: make the save route persist non-secure answers and return a per-field error, and have the MCP create/update tools reject `type: 'secure'` when the key is absent rather than authoring an unusable form.
- ☐ **[HIGH]** Form `settings` (thank-you message, edit-after-submit) can't be set by any UI or MCP tool
 - *Function · hours* — Action accepts settings at `src/lib/actions/onboarding.ts:120-135` (line 123). Portal consumes both: `src/app/portal/[token]/onboarding/[id]/page.tsx:42` and `:77`; save route at `src/app/api/portal/[token]/onboarding/[id]/save/route.ts:33`. Builder's `save()` sends only `name/description/status/schema` — `src/components/onboarding/form-builder-client.tsx:144-146`. MCP `update_onboarding_form` zod shape has no `settings` key — `src/lib/mcp/tools/plugin-form-tools.ts:292-297`. Repo-wide grep for `thank_you_message/allow_edit_after_submit` finds no other onboarding writer.
 - **Impact.** `allow_edit_after_submit` is permanently falsy, so a customer who submits with a typo gets a 409 forever (`save/route.ts:33-35`) and the business must delete the request and re-send. The thank-you screen is the same hardcoded copy for every business (`onboarding-fill.tsx:115`).

- **Fix.** Add a settings section to the builder passing `settings` through the existing action, and add `settings` to the MCP tool's zod shape. Both plumbing already exists end-to-end; only the two write surfaces are missing.

Deferred (6) — medium/low on a feature marked *simplify*

- **[MEDIUM]** Autosave discards answers on failure and reports "✓ Saved" anyway — `src/components/customer-portal/onboarding-fill.tsx:68-79` — `const patch = dirtyRef.current; dirtyRef.current = {};` precedes `await fetch(...)`; no `res.ok` check; `catch { /* autosave is best-effort */ }` swallows errors; `setSavedTick(true)` at line 76 is unconditional. Line 83: `useEffect(() => () => { if (saveTimer.current) clearTimeout(saveTimer.current); }, [])` — clears, never flushes, despite the comment 'Flush pending autosave on unmount.' Line 131 renders '✓ Saved' / 'Progress saves automatically'. Counter-evidence: line 91 sends the full `answers` state on submit.
- **[MEDIUM]** No reminders, resend, or expiry nudge — sent forms decay silently — `grep -rn onboarding vercel.json src/app/api/cron/` → no matches. Sent-tab actions: `src/components/onboarding/onboarding-forms-client.tsx:230-248` (View answers when completed, else Copy link at :236, plus delete). Token expiry 90 days: `src/lib/actions/onboarding.ts:162`. Expired/revoked token dead-ends at `notFound()`: `src/app/portal/[token]/onboarding/[id]/page.tsx:20-21`.
- **[MEDIUM]** Draft and archived forms can be sent and filled — status is decorative — `src/lib/actions/onboarding.ts:176` selects `id, name, status, schema`; validation at 180-182 checks only existence and `schema.length`. Portal page selects `name, description, schema, settings` and never reads status — `src/app/portal/[token]/onboarding/[id]/page.tsx:29`. Send button enabled on any form with fields — `src/components/onboarding/onboarding-forms-client.tsx:184`. Profile picker excludes archived only — `customers/[id]/page.tsx:31`.
- **[LOW]** Select-then-insert race on the first save can surface a raw Postgres duplicate-key error to the customer — `src/app/api/portal/[token]/onboarding/[id]/save/route.ts:38-39` (`.maybeSingle()`), :68-70 (update-vs-insert branch), :71 (`return NextResponse.json({ error: error.message }, { status: 500 })`). Unique constraint: `supabase/migrations/20260624100000_client_onboarding.sql:53` `request_id UUID NOT NULL UNIQUE`. `src/components/customer-portal/onboarding-fill.tsx:85-105` — `submit()` never touches `saveTimer`.
- **[LOW]** None of the onboarding UI uses the `.ch-*` or `kirei` design vocabulary — it is entirely hand-rolled — `grep` for `ch-` and `ui/kirei` over `src/components/onboarding/` and `src/components/customer-portal/onboarding-fill.tsx` → no matches. Bare table: `onboarding-forms-client.tsx:211-253`. `STATUS_TONES` duplicated at `onboarding-forms-client.tsx:25-32` and `customer-onboarding-card.tsx:25-30`. Hand-rolled tabs: :146-155. Counter-evidence: `src/components/leads/leads-client.tsx:30` imports `kirei` primitives; `src/components/leads/leads-board.tsx` contains no `.ch-*` class.
- **[LOW]** Zero tests across the entire feature, including pure logic that is trivial to test — No onboarding test files exist (`src/lib/onboarding/` contains only `answers.ts`, `crypto.ts`, `presets.ts`). Pure functions: `isValidAbn` `src/lib/onboarding/answers.ts:48-54`, `isValidAcn` :58-63, `invalidAnswerFields` :74-125 (regex rules :98-113, pattern guard :116-122), `missingRequiredFields` :129-146. `crypto.ts:32-46` encrypt/decrypt round-trip.

Batch 2.23 — Field-services plugins: Products, Inventory, Expenses, Assets, Timesheets

Health: rough · **Verdict:** simplify · **Findings:** 0 blocker · 1 high · 5 medium · 4 low **Estimated:** ~0.0 days (1 scheduled, 9 deferred)

What it does. Five modules registered in the plugin system (`src/lib/plugins/registry.ts:68-76`). **Products** (`defaultEnabled: true`) is the price-list catalogue used to fill quote/invoice line items — full CRUD, CSV bulk import, search, Smart Organise. The other four are `defaultEnabled: false` and are each a single list page plus a couple of dialogs: **Inventory** adds four columns to `products` (`track_stock/stock_qty/reorder_point/unit_cost`) and a `stock_movements` ledger you adjust by hand; **Expenses** records costs with a category, an optional work-order link, a "rebill to customer" flag and a receipt upload to a private bucket; **Assets** tracks tools/vehicles with a status dropdown, an assigned member and a service log; **Timesheets** sums `job_time_entries` per worker for a UTC Mon-Sun week, multiplies by

`member_profiles.hourly_rate` and exports CSV. All four were shipped in one batch on 2026-07-10 (supabase/migrations/20260710140000...20260710200000). None has a mobile screen (mobile/app contains products only), none has a cron, and none has a test.

Why this rating. The plumbing is clean — RLS with worker isolation is correct on every new table, UUID/empty-string coercion is done properly, and the MCP tools exist. But the four newer modules are islands: nothing in the app writes to them and nothing reads out of them. Job materials never move stock, billable expenses can never reach an invoice, reorder points never alert anyone. Timesheets produces a payroll number that is wrong for an Australian business because it buckets hours by UTC day. And /products — the one module every business has on — renders its "Catalog value" KPI as NaN because of the exact PostgREST numeric-as-string trap documented in CLAUDE.md.

If you only do three things:

1. Delete Timesheets and Assets outright. Timesheets computes a pay figure that is wrong for every Australian Monday morning and after-5pm shift (timesheets.ts:41-70) and exports it to CSV for payroll — that is a liability, not a feature, and doing it properly means timezone-correct bucketing, approval, overtime and a mobile screen. Assets is a table nobody reads, whose own RLS denies the workers who hold the tools. Both are default-off, so nothing breaks. That is 3 tables, 3 routes, ~370 lines and 2 sidebar entries gone.
2. Fix the three defects that a paying user hits today: the \$NaN catalog value on /products (products-client.tsx:80 — one `Number()` call, and /products is on for everyone), the receipt IDOR (expenses.ts:125 — scope the path to the caller's business), and the delete-without-confirm in expenses and assets (expenses-view.tsx:136, assets-view.tsx:107).
3. Decide Expenses in or out, and if in, finish it in one pass: pull billable expenses into invoiceUnbilledForWorkOrder (work-orders.ts:492) so the rebill checkbox is true, relax the RLS so workers can capture their own fuel and materials receipts, and add a mobile capture screen. An expense tracker the field crew cannot use and that cannot rebill is not competing with ServiceM8 — it is a data-entry chore.

Scheduled

- ☐ **[HIGH]** /products "Catalog value" KPI string-concatenates prices and renders NaN
 - *Function · minutes* — src/components/products/products-client.tsx:80 `products.reduce((s, p) => s + (p.unit_price ?? 0), 0)` — no `Number()`. `unit_price` is `numeric(12,2)` (supabase/migrations/001_initial_schema.sql:67) and `getProducts` does a plain `select("*")` (src/lib/actions/products.ts:18) with no coercion, so PostgREST returns it as a string. The sum is passed to `formatCurrency(amount: number)` (src/lib/utls.ts:8) which calls `Intl.NumberFormat.format`. Rendered at products-client.tsx:111 as the "Catalog value" StatTile. `products` is `defaultEnabled: true` in src/lib/plugins/registry.ts:75 — the only one of the five that is.
 - **Impact.** With one product the tile is coincidentally right (`0 + "150.00" = "0150.00" → Number 150`). With two or more it becomes `"0150.0075.00" → Number(...)` is NaN → the tile reads \$NaN. Every business with a price list of 2+ items sees a broken KPI on a default-on page.
 - **Fix.** `s + Number(p.unit_price ?? 0)` at products-client.tsx:80. Consider whether "sum of unit prices" is a metric worth showing at all.

Deferred (9) — medium/low on a feature marked *simplify*

- **[MEDIUM]** Timesheets bucket hours by UTC day, so Monday-morning work in Australia lands in the previous week — `src/lib/actions/timesheets.ts:41-42` builds the window from `new Date(weekStartISO + "T00:00:00Z")`; `:46` filters `started_at` between those UTC bounds; `:70` `(new Date(e.started_at).getUTCDay() + 6) % 7` picks the day column.
`src/app/(dashboard)/timesheets/page.tsx` `mondayOf()` is UTC-based too. No timezone is read anywhere in the file. The sub-claim also holds: a running entry has null `duration_seconds` until `stopTimeEntry` writes it (`src/lib/actions/job-time.ts:60-79`), and `timesheets.ts:69` `num(null)` silently yields 0 hours.
- **[MEDIUM]** `getReceiptUrl` signs any storage path with the service-role client, with no business scoping — `src/lib/actions/expenses.ts:125-130` — `await biz()` is called for its side effect only, the resolved `businessId` is discarded, and the caller-supplied `path` goes straight into `createAdminClient().storage.from("expense-receipts").createSignedUrl(path, 3600)`.
`Upload` namespaces the path at `:117` `${businessId}/${randomUUID()}.${ext}` so the tenant id is present and simply never compared. `supabase/migrations/20260710140000_expenses.sql:51` creates the bucket `private`; `grep` across `supabase/migrations` for 'expense-receipts' returns only that one line — zero `storage.objects` policies.
- **[MEDIUM]** Billable expenses can never reach an invoice — the "Rebill to the customer" checkbox does nothing — `src/components/expenses/expenses-view.tsx:176` renders the "Rebill to the customer" checkbox; it persists as `expenses.billable` (`src/lib/actions/expenses.ts:67`). `src/lib/actions/work-orders.ts:495-502` — the only `job-costs-to-invoice` path — queries `job_time_entries` and `job_materials` only; `grep` for 'expenses' in that file returns nothing. `supabase/migrations/20260710140000_expenses.sql` declares status `CHECK IN ('recorded','reimbursed','invoiced')`; `grep` for 'invoiced' across `src` returns only `src/types/database.ts:580` and the MCP enum in `src/lib/mcp/tools/expenses-tools.ts:58` — no code sets it.
- **[MEDIUM]** Inventory is a manual ledger — nothing in the job flow moves stock and no reorder point ever alerts — `grep -rl 'stock_qty|adjustStock|track_stock' src/` returns exactly 4 files, none of them `src/lib/actions/job-materials.ts`. `grep -rn 'reorder_point' src/` returns only `inventory-view.tsx` (32, 88, 93), `actions/inventory.ts` (37, 45, 55, 61) and `mcp/tools/inventory-tools.ts` (15, 20, 25, 32, 39) — no scheduled job reads it. `src/lib/plugins/registry.ts:76` has `inventory.defaultEnabled:false`.
- **[MEDIUM]** Products can be archived but never un-archived, and archived items are invisible with a permanently-zero counter — `src/components/products/product-form.tsx:42` `await onSubmit({ ...data, archived: product?.archived ?? false })` — the zod schema at `:13-19` has no archived field and the form body renders no control for it. `src/app/(dashboard)/products/page.tsx` calls `getProducts()` with no arg → `src/lib/actions/products.ts:19` applies `.eq("archived", false)`. So `products-client.tsx:81-82` derives `archivedCount` from a list archived rows were filtered out of, and the subtitle at `:88` is structurally always "N active · 0 archived".
- **[LOW]** `adjustStock` is a non-atomic read-modify-write that can lose adjustments and desync the ledger — `src/lib/actions/inventory.ts:77-88` — reads `stock_qty` (77), computes `next = num(prod.stock_qty) + delta` in JS (79), inserts the `stock_movements` row (81), then writes the absolute `next` back (87).
`src/lib/mcp/tools/inventory-tools.ts:56-67` is the same sequence line for line. Contrast the atomic RPC pattern `CLAUDE.md` established for number minting (`next_invoice_number / next_quote_number`).
- **[LOW]** Expenses and Assets delete on a single click with no confirmation, while Products correctly confirms — `src/components/expenses/expenses-view.tsx:136` wires the `Trash2` button directly to `handleDelete` → `deleteExpense` with no dialog. `src/components/assets/assets-view.tsx:107` does the same for `deleteAsset`.
`src/components/products/products-client.tsx:167` sets `deleteId` and routes through the `AlertDialog` at `:210-221`. The `CleanupEntity` union (`src/lib/actions/cleanup.ts:29-35`) lists `customers/contacts/leads/invoices/quotes/products...` — no expenses, no assets — so there is no undo path either. `supabase/migrations/20260710200000_assets.sql:24` `asset_id UUID NOT NULL REFERENCES public.assets(id) ON DELETE CASCADE` confirms service history goes with the asset.
- **[LOW]** The four new pages hand-roll a design system instead of composing the existing one — Private Stat components: `src/components/expenses/expenses-view.tsx:196`, `src/components/inventory/inventory-view.tsx:153`, `src/components/assets/assets-view.tsx:166` — versus `products-client.tsx:110-112` using `StatTile` from `@/components/ui/kirei`. Identical `selectCls` string duplicated at `expenses-view.tsx:21`, `inventory-view.tsx:17`, `assets-view.tsx:22`. Raw status colours: `inventory-view.tsx:91` `bg-amber-100 text-amber-700`,

expenses-view.tsx:131 `bg-emerald-100 text-emerald-700`, assets-view.tsx:21 STATUS_TONE map — none use `.ch-pill` or `KireiPill`. KPI strips are bare `grid grid-cols-3` at expenses-view.tsx:108, inventory-view.tsx:69, assets-view.tsx:77 versus products' `grid-cols-2 lg:grid-cols-3` at products-client.tsx:109. No search input in any of the three. src/lib/actions/expenses.ts:33 `.limit(filters?.limit ?? 300)` with no pagination and no date-range control.

- **[LOW]** Zero test coverage across all five modules, including the two pure functions that are trivially testable and currently wrong — find src for test files returns only src/lib/**tests**/pg-filter.test.ts, src/lib/assistant/**tests**/models,scopes,undo.test.ts, src/lib/booking/**tests**/availability,booking-db,time.test.ts, src/lib/content/**tests**/live-agents,pipeline,prompts,schedule.test.ts and src/lib/mcp/**tests**/collect,invoke,live-api.test.ts. Nothing covering products, inventory, expenses, assets or timesheets, and no src/lib/actions/**tests** directory exists.

Batch 2.24 — Quoting Agent

Health: broken · **Verdict:** cut · **Findings:** 0 blocker · 0 high · 7 medium · 3 low **Estimated:** ~0.0 days (0 scheduled, 10 deferred)

What it does. A separate AI chat surface at /quoting-agent where a trades business briefs an agent in plain English ("8m of fence repair, palings + posts, urgent") and the agent builds a draft quote. It is backed by two tables: quoting_agent_settings (one row per business — enabled flag, industries, baseline hourly rate / margin / tax / call-out fee / emergency multiplier, an estimation_mode of manual|ai_estimate|skip, and free-text house notes) and quoting_agent_knowledge, a per-business key/value "knowledge bank" of material prices, labour rates and scope templates that the agent both reads from and writes to during conversation. A 4-step onboarding wizard collects the industries, rates, estimation mode and optional seed prices; a settings page edits the rates and the knowledge bank by hand. The API route (src/app/api/quoting-agent/route.ts) runs its own 12-iteration Anthropic tool loop with 7 tools (5 knowledge CRUD + search/create customer + create_quote). It is a plugin, default OFF, gated by nav, route and the plugin registry.

Why this rating. The feature is unreachable end-to-end for any business that has not already been enabled through the Plugins store: the plugin route gate in src/app/(dashboard)/layout.tsx:85-89 redirects /quoting-agent to /dashboard whenever the plugin resolves disabled, which makes the "Get started" enable card at src/app/(dashboard)/quoting-agent/page.tsx:13-33 and the entire onboarding entry point dead code. Even for an enabled business, ANTHROPIC_API_KEY returns 401 in production and the route has no try/catch around anthropic.messages.create (route.ts:376), so the failure surfaces as a raw 500 HTML page rendered inside a chat bubble. Zero tests exist for any of it.

If you only do three things:

1. Decide the surface question first, and decide to cut. Expose quoting_agent_knowledge as MCP tools under a quoting:read/quoting:write scope in src/lib/mcp/tools/, teach the assistant to consult it before pricing, keep the rates form as a settings page, and delete src/app/api/quoting-agent/route.ts plus src/components/quoting-agent/chat-client.tsx. This single move fixes the unreachable-route blocker, the missing persistence, the missing error handling, the missing streaming and the off-palette UI at once — because the assistant already solved all of them — and it makes the pricing bank usable by voice, which is the owner's stated first-class requirement.
2. If the surface is kept instead, fix the three things that make it unusable today, in this order: the route gate that redirects /quoting-agent to /dashboard before the enable card can render (layout.tsx:85-89 vs page.tsx:13-33); the unhandled Anthropic throw that prints raw HTML into the chat (route.ts:376, chat-client.tsx:55-58); and the ignored PostgreSQL errors at route.ts:169, 181 and 225 that let the agent invent prices and duplicate customers.
3. Make margin real in code rather than a prompt instruction. Apply default_margin_percent inside the create_quote handler (route.ts:252-290), return the applied figure in the tool result, extract the arithmetic into src/lib/quoting/line-items.ts, and unit-test it. A margin the model silently forgets on a \$327k pipeline is the one defect in this feature that costs the dependent user actual money.

Deferred (10) — medium/low on a feature marked cut

- **[MEDIUM]** The Quoting Agent enable card and onboarding are unreachable — route gate redirects before they render — Confirmed at `src/app/(dashboard)/layout.tsx:82-87` — `const gate = ROUTE_GATES.find((g) => path === g.prefix || path.startsWith(g.prefix + "/")); if (gate && !plugins[gate.pluginId]) redirect("/dashboard")`. `ROUTE_GATES` is built at `src/lib/plugins/registry.ts:99-100` from `OPTIONAL_PLUGINS`, and the quoting-agent entry at `registry.ts:81` has `defaultEnabled: false, settingsTable: "quoting_agent_settings", routePrefixes: ["/quoting-agent"]`. `resolveEnabledPlugins` (`registry.ts:122-125`) takes `settingsFlags[p.settingsTable] ?? byId.get(p.id) ?? p.defaultEnabled`; `getPluginFlags` (`src/lib/layout-data.ts:84`) returns `quoting: quoting ? !!quoting.enabled : null`. A business with no row or `enabled=false` therefore resolves false and is redirected. The disabled branch at `src/app/(dashboard)/quoting-agent/page.tsx:13-33` is consequently unreachable — `getQuotingAgentSettings` (`src/lib/actions/quoting-agent.ts:56-57`) even lazily inserts `enabled: false`, guaranteeing the gate fires.
- **[MEDIUM]** `search_customers` ignores the PostgreSQL error — the exact half-fixed trap that caused duplicate customers — `src/app/api/quoting-agent/route.ts:221-226` — `const { data } = await tbl(supabase, "customers").select(...).or(iLikeAcross(["name", "email", "company"], q)).limit(10); return data ?? []`; . Same omission at `route.ts:169` (`const { data } = await query; return data ?? []`;) and `route.ts:181`. By contrast `save_knowledge` (`route.ts:199`), `update_knowledge` (`route.ts:210`) and `delete_knowledge` (`route.ts:216`) all return `{ error: error.message }`. The `.or()` itself is safe — `iLikeAcross` from `src/lib/pg-filter.ts` is used correctly.
- **[MEDIUM]** No error handling around the Anthropic call — a failed request surfaces the raw error string in the chat bubble — `src/app/api/quoting-agent/route.ts:12` `const anthropic = new Anthropic()`; at module scope; `route.ts:376-382` calls `await anthropic.messages.create({...})` inside the while loop with no try/catch in the POST handler (verified across the whole function, lines 303-426). `src/components/quoting-agent/chat-client.tsx:55-58` does `const err = await res.text(); throw new Error(err || \ Server returned ${res.status} )`, and lines 63-67 catch it and push `{ role: "assistant", text: 'Error: ${e.message}' }` into the message list.
- **[MEDIUM]** The pricing knowledge bank has no MCP tool, so the main assistant cannot see it — `grep -ril quoting src/lib/mcp/` returns only `src/lib/mcp/tools/plugin-form-tools.ts`. `grep -rn quoting_agent_knowledge src/` returns only `src/app/api/quoting-agent/route.ts` (lines 162, 174, 186, 208, 214) and `src/lib/actions/quoting-agent.ts` (lines 121, 149, 176, 188, 215) — no `register-tools.ts` or `src/lib/mcp/tools/*` entry. `src/lib/mcp/collect.ts` feeds the same registry to `/api/assistant`, so the omission propagates to the main assistant.
- **[MEDIUM]** The quoting conversation is memory-only — a refresh destroys it, and Clear wipes it with no confirmation — `src/components/quoting-agent/chat-client.tsx:31-32` — `const [messages, setMessages] = useState<DisplayMessage[]>([])` and `const [apiHistory, setApiHistory] = useState<ApiMessage[]>([])`, with no write to any table anywhere in the file. `chat-client.tsx:87-91` — `<Button variant="ghost" size="sm" onClick={() => { setMessages([]); setApiHistory([]); }}>Clear</Button>` with no confirm, in contrast to `src/components/quoting-agent/settings-client.tsx:78` which gates deletion behind `if (!confirm("Forget this fact?")) return`;
- **[MEDIUM]** Default margin is collected, displayed and prompted but never applied by any code — `default_margin_percent` is captured at `src/components/quoting-agent/onboarding-client.tsx:58` and `88`, edited at `src/components/quoting-agent/settings-client.tsx:153-154`, and injected into the prompt at `src/app/api/quoting-agent/route.ts:339`. In the `create_quote` handler, `route.ts:256` computes `const sub = qty * price`, `route.ts:257` `const tax = sub * (taxRate / 100)`, `route.ts:273-274` recompute subtotal and `tax_total` the same way, and `route.ts:286-288` hardcode `discount_type: null, discount_value: 0, discount_amount: 0`. No margin appears. `route.ts:365` instructs the model: 'Apply margin where appropriate (build into unit_price)'.
- **[MEDIUM]** Decimal rates cannot be typed in the settings editor — `parseFloat` runs on every keystroke — `src/components/quoting-agent/settings-client.tsx:148-149, 153-154, 158-159, 163-164, 168-169` all follow `<Input value={settings.X ?? ""} onChange={(e) => setSettings({ ...settings, X: e.target.value ? parseFloat(e.target.value) : null })} />` — the parsed number is the

rendered value, so `parseFloat("1.") === 1` re-renders as "1" and swallows the separator.

`src/components/quoting-agent/onboarding-client.tsx:57-61` keeps the same five fields as strings (`useState(...?.toString() ?? "")`) and parses once at submit, `onboarding-client.tsx:87-91`.

- **[LOW]** Knowledge bank renders every row with no search or filter, although the action already supports both — `src/lib/actions/quoting-agent.ts:116-128` — `listKnowledge(filters?: { kind, category, search })` applies `.eq("kind"), .eq("category"), .ilike("key", \ %${search}%)` .
`src/app/(dashboard)/quoting-agent/settings/page.tsx:8` calls `listKnowledge()` with no arguments. `src/components/quoting-agent/settings-client.tsx:239-290` renders every row grouped by kind with no search input, filter chips, pagination or sort; the heading at `settings-client.tsx:215` prints the raw count; the unconfirmed badge is at `settings-client.tsx:260-264`.
- **[LOW]** Chat and onboarding surfaces are off-palette and hand-rolled rather than composed from the design system — `from-purple-500 via-fuchsia-500 to-rose-500` appears at `src/components/quoting-agent/chat-client.tsx:77`, `chat-client.tsx:98`, `src/app/(dashboard)/quoting-agent/page.tsx:18` and `src/components/quoting-agent/onboarding-client.tsx:122`, against the deep-teal Connected Hub accent. `chat-client.tsx:76-92` hand-builds a header div rather than using `PageHeader`, which the sibling `settings-client.tsx:110-122` does import and use. `chat-client.tsx:74` hard-codes `className="flex flex-col h-[calc(100vh-7rem)] max-w-3xl mx-auto"` .
- **[LOW]** Zero tests for the money math, the plugin gating, or the knowledge upsert — `find src -name "*.test.ts"` returns exactly 14 files: `src/lib/assistant/tests/{models,scopes,undo}.test.ts`, `src/lib/booking/tests/{availability,booking-db,time}.test.ts`, `src/lib/content/tests/{live-agents,pipeline,prompts,schedule}.test.ts`, `src/lib/mcp/tests/{collect,invoke,live-api}.test.ts`, `src/lib/tests/pg-filter.test.ts`. None mention quoting. The line-item arithmetic at `src/app/api/quoting-agent/route.ts:252-290` is inline in the route handler and not importable. Key normalisation is `input.key.trim().toLowerCase()` at `src/lib/actions/quoting-agent.ts:153`, mirrored at `route.ts:190` and `quoting-agent.ts:205`, controlling collisions on the `(business_id, kind, key)` unique index.

Batch 2.25 — Plugin system, agents store & admin

Health: rough · **Verdict:** simplify · **Findings:** 0 blocker · 0 high · 6 medium · 3 low **Estimated:** ~0.0 days (0 scheduled, 9 deferred)

What it does. Kirei's modules are declared in a static registry (`src/lib/plugins/registry.ts` , 30 plugins, 6 marked `core`). Per-business on/off state lives in `business_agent_installs` , resolved by `resolveEnabledPlugins()` with a deliberate data-safe fallback chain: legacy settings table → install row → `defaultEnabled` , so a business with no rows behaves exactly as it did pre-plugin-system. Three industry presets (trades / agency / seo-agency-local) write explicit enable+disable rows for every optional module and set `businesses.industry_preset` , which also drives a small sidebar vocabulary override. Gating is claimed to be three layers but is really two: the sidebar filters nav items on `features[plugin]` (`app-sidebar.tsx:102`) and the dashboard layout redirects direct URL hits via `ROUTE_GATES` (`(dashboard)/layout.tsx:85-86`). The MCP/assistant tool surface has no plugin check at all — only API scopes. The `/agents` page ("Plugins") is a single client component combining preset cards, a module toggle grid, and a separate catalog of 13 AI "agents" that share the same table with different semantics. `/admin` is an internal operator console (separate `admin_operators` table, 4 roles, audit log, tenant list, metrics) with a tenant-impersonation feature that, as wired today, does not actually work.

Why this rating. The registry and resolver are well-designed and genuinely data-safe — that part is solid. But the gating is incomplete in the two places it matters most: background crons ignore plugin state entirely (including the one that charges customer cards), and the MCP/assistant surface has no gate at all. The flag read swallows query errors and then caches the wrong answer for five minutes. The admin panel's headline feature (impersonation) renders no banner in the tenant app, never enforces its read-only flag, and lands the operator on their own dashboard. Zero tests cover any of it.

If you only do three things:

1. Close the gating holes where money and AI spend happen: gate every cron on the owning plugin (recurring-invoices is charging saved cards for a module the business turned off, recurring-invoices/route.ts:31), and filter the assistant's collected tool list by the enabled map. Then check **error** in getPluginFlags/getEnabledPlugins so a transient DB failure stops silently rewriting a business's configuration for 5 minutes.
2. Decide what /admin is for and act on it. Impersonation renders no banner in the tenant app, never enforces read_only, and drops the operator on their own dashboard — with 7 tenants, deleting it is a defensible answer, and shipping a broken escalation path labelled "read-only" is not. If it stays, wire the banner into (dashboard)/layout.tsx and enforce the flag.
3. Collapse the /agents page to one concept and one write path. Delete the duplicate agent cards for client-onboarding and form-builder, validate ids in installAgent/toggleAgent so the agent path cannot bypass the core and dependency guards, then rebuild the page on PageHeader + category-grouped, searchable module cards to the standard src/components/leads/ now sets.

Deferred (9) — medium/low on a feature marked *simplify*

- **[MEDIUM]** Recurring-billing cron charges saved cards for businesses that disabled the module — `src/app/api/cron/recurring-invoices/route.ts:30-34` selects `recurring_invoices` on `active=true` + `next_run_on<=today` with no plugin/business filter, then line 49-58 calls `generateRecurringInvoice` with `autoCharge: r.auto_charge`. `src/app/api/cron/recurring-jobs/route.ts:38-41` has the same shape. By contrast `src/app/api/cron/invoice-reminders/route.ts:43-47` does gate on `business_agent_installs` (`agent_id='invoice-reminders'`, `enabled=true`). The module is route-gated at `src/app/(dashboard)/layout.tsx:85-86` via `ROUTE_GATES`, so `/recurring-invoices` does redirect to `/dashboard` when disabled.
- **[MEDIUM]** Plugin-flag query ignores `error` and caches the wrong answer for 5 minutes — `src/lib/layout-data.ts:76-81` destructures only `{ data }` from four parallel queries; the result is returned from inside `unstable_cache` with `{ tags: [...], revalidate: 300 }` at line 90. `src/lib/actions/plugins.ts:36-41` repeats the pattern for `getEnabledPlugins` (uncached). Line 83 coerces `installRows` to `[]` on failure, and `resolveEnabledPlugins` (`registry.ts:124-127`) then falls through to `p.defaultEnabled` for every plugin.
- **[MEDIUM]** Admin impersonation renders no banner in the tenant app and never enforces read-only — `grep` for `ImpersonationBanner/getActiveImpersonation` across `src/` returns only `src/app/admin/layout.tsx:3,34,38` and the component itself — the (dashboard) layout never calls it. `startImpersonationAction` (`src/app/admin/actions.ts:27-29`) sets `readOnly:true` then redirects to `/dashboard`. `grep` for `read_only` returns only the operator role enum (`actions.ts:44,137`; `roles.ts:6`), the banner label (`impersonation-banner.tsx:27`) and the type/field in `impersonation.ts:8,27,64,94` — no tenant write path reads it. (dashboard)/layout.tsx:44 picks the business from `allBusinesses` (operator's own owned+memberships), so the impersonated id is not found and it falls back to `allBusinesses[0]`, while `getActiveBizId` (`active-business.ts:21-26`) returns the impersonation cookie value.
- **[MEDIUM]** MCP and the AI assistant have no plugin gating — the claimed third layer does not exist — `src/lib/mcp/context.ts:22-29` `assertScope` is the only check; it validates the API key's scopes, nothing about enabled plugins. Tools collected by `src/lib/mcp/collect.ts` are handed to the assistant unfiltered (`src/app/api/assistant/route.ts:169-170` resolves `role` → scopes only).
- **[MEDIUM]** Dashboard quick actions link to modules the business has disabled — `src/components/dashboard/dashboard-client.tsx:150-153` hard-codes `HubTiles` to `/invoices/new`, `/quotes/new`, `/work-orders/new`, `/customers/new`; lines 130 and 134 hard-code stat-tile hrefs to `/invoices?status=pending` and `?status=overdue`; line 93 a New-invoice CTA. Props at line 35 carry no features map even though (dashboard)/layout.tsx:93 already passes `features={plugins}` to `DashboardShell`.
- **[MEDIUM]** The resolver that gates the entire nav and route surface has zero tests — find over `src/**/*.test.ts` returns 14 files, none referencing `plugins/registry`, `presets`, `admin_operators` or `impersonation`. The logic under test is `resolveEnabledPlugins` (`src/lib/plugins/registry.ts:114-130`) and `dependentsOf` (103-107), consumed by (dashboard)/layout.tsx:74-86 for both nav and route gating.
- **[LOW]** /agents hand-rolls its layout and ignores the design system the leads workspace sets — `src/components/agents/agents-store.tsx:240-254` (hand-built header + gradient rail div), 264-279 (chips), 163-166 + 359 (chips filter only the AI-agent grid), 303 and 336 (line-clamp-2), 326-353 (25-card module grid with no search or grouping despite `PluginCategory` existing on every registry entry).
- **[LOW]** Applying a preset disables modules holding live data with no warning about what is in them — `src/lib/plugins/presets.ts:67-74` `presetInstallRows` writes `enabled:false` for every `OPTIONAL_PLUGINS` entry not in the bundle; `applyIndustryPreset` (`src/lib/actions/plugins.ts:102-105`) upserts them all. The confirm dialog (`agents-store.tsx:399-414`) lists hidden module names only, with the reassurance at line 398 and 319 that nothing is deleted. Agency bundle at `presets.ts:35-39` contains jobs, quotes, invoicing, recurring-billing, contracts.
- **[LOW]** Admin tenant list swallows query errors and renders them as an empty state — `src/app/admin/tenants/page.tsx:28` `const { data: tenants } = await query` with no error check; line 36 coerces to `[]` and lines 71-76 render "No tenants match your search"/"No tenants yet". Line 25 already uses `likeAcross(['name','email'], q)`, so the injection half of the CLAUDE.md trap is closed.

Phase 3 — Delete

~2 days. Do this **before** Tier D, so you are not polishing code you are about to remove.

Batch 3.1 — Quoting Agent (4h)

Rated **broken** and **unreachable**: the route gate redirects `/quoting-agent` whenever the plugin is disabled, and it reads the same `quoting_agent_settings` flag the page checks — so the "Get started" card can only render in a state where you would already have been redirected. Verified independently.

- ☐ **Keep** `quoting_agent_knowledge` — per-business pricing memory is the good idea
- ☐ Expose the knowledge bank to the main assistant as MCP tools (it is currently invisible to it)
- ☐ Delete `/quoting-agent` pages, `/api/quoting-agent`, the chat client, the registry entry
- ☐ The main assistant already has `create_quote` and ~199 tools, streaming, persistence and undo

Batch 3.2 — Dead enable-cards (1h)

Same root cause, verified across three plugins: the route gate reads the authoritative settings table, so every in-page "enable" screen is unreachable. Only the Plugins store can turn these on.

- ☐ `/quoting-agent` `page.tsx:13-33`
- ☐ `/forms` `page.tsx:16-17` (`enabled={false}` branch)
- ☐ `/onboarding-forms` `page.tsx:16-17` (`enabled={false}` branch)
- ☐ Either delete the branches, or exempt these routes from the gate so the cards work

Batch 3.3 — Merge the two form builders (1d)

`/forms` and `/onboarding-forms` already share the field engine but ship two builders, two viewers, two MCP tool sets, two buckets and two mental models.

- ☐ Keep public lead-capture as the primary (it acquires customers)
- ☐ Fold onboarding in as a delivery mode

Batch 3.4 — The strategic cut (your call)

- ☐ **SEO Production plugin** — largest body of code in the repo, different market, zero users, live operational burden
- ☐ **Content Studio** — same argument, newer, less sunk cost
- ☐ **Prospects / outbound** — agency tooling; also drags in spam-compliance obligations

I am not making this call for you. But every hour spent maintaining these is an hour not spent on Xero or subscription billing.

What is deliberately not scheduled

84 findings (~31.1 days) — medium and low severity on features marked *simplify* or *cut*. They are listed under each feature in collapsed sections above, so nothing is lost. Revisit only if a feature graduates back to *keep*.

Design-system unification — folded into each feature's PR rather than run as a standalone project.

Pagination on desktop lists (silent `.limit()` caps) — not urgent at 42 invoices. **Except mobile's 100-row cap**, which the roofing business is one busy year from hitting; that one is scheduled under Mobile.

Effort summary

Two numbers, because one of them is misleading on its own.

Sum of every individual estimate: ~63.6 days. That is what you get by adding up all 157 scheduled findings. It is an over-count, and I do not believe it, for three reasons: findings within a feature overlap heavily (one fix often closes three findings), Phase 1 sweeps remove a large share of Phase 2 before it starts, and per-finding estimates are made without knowing the file is already open.

Realistic, sequenced: ~4–5 focused weeks.

Phase	Work	Estimate
0	Safety — RLS, crons, worker isolation	~1 day
1	Six cross-cutting sweeps	~4–5 days
2	25 feature batches (Tiers A–C carry the value)	~2–3 weeks
3	Deletions and merges	~2 days
	Total	~4–5 weeks focused

If you only get through **Phase 0, Phase 1 and Tier A**, that is ~2 weeks and it covers the security hole, every repeating bug class, and the entire money path. That is the version to do if time is short.

Two conditions on that estimate.

It holds only if **nothing new is built** during it. In a single recent session the project shipped an AI assistant rebuild, a Content Studio plugin, a typeface change and a Leads redesign — all decent work, none of which moved the product closer to being sold.

And none of this makes Kirei **sellable**. Subscription billing (there is currently no way to charge anyone for Kirei) and Xero sync (the integration Australian bookkeepers veto deals over) are separate tracks. They decide whether there is a business; this plan decides whether it is a good product.